ThoraxDSS: A Negation-Aware Multimodal RAG System for Chest X-ray Decision Support

Kaliopi Oikonomou

DIPLOMA THESIS

Supervisor: Evaggelia Pitoura

Department of Computer Science and Engineering

School of Engineering

University of Ioannina

February 2026

ACKNOWLEDGEMENTS

I would like to express my sincere appreciation to Professor Evaggelia Pitoura for supervising this thesis and for her guidance and insightful feedback throughout this work.

Beyond her academic guidance, I am deeply grateful for her mentorship. Her clarity of thinking, professionalism, and integrity have been a constant source of inspiration. I feel fortunate to have learned from her, not only through her ideas and teaching, but also through her character and dedication to her work.

Finally, I would like to thank my friends and family for their encouragement and constant support throughout this process.

TABLE OF CONTENTS

ABSTRACT

Kaliopi Oikonomou, Diploma, Department of Computer Science and Engineering, School of Engineering, University of Ioannina, Greece, February 2026.
ThoraxDSS: A Negation-Aware Multimodal RAG System for Chest X-ray Decision Support.
Supervisor: Evaggelia Pitoura.

Chest X-ray interpretation is a common clinical task in which findings may be subtle, radiographic patterns may overlap, and image interpretation often depends on the radiologist's experience. Reliable computational support is further complicated by the linguistic structure of radiology reports, where the same clinical concept may appear under different expressions and polarity (present versus absent findings), directly affecting clinical meaning. This thesis presents *ThoraxDSS*, a multimodal retrieval-augmented decision support system for chest X-ray interpretation. The system accepts (i) a free-text clinical description and (ii) a patient chest X-ray image, and retrieves clinically relevant prior cases through a multi-pathway retrieval pipeline that captures textual semantics, visual morphology, and cross-modal compatibility within a late-fusion ranking framework. To improve retrieval reliability, the text-based retrieval component is further refined with a negation-aware and synonym-aware adjustment that normalizes surface-form variation to canonical clinical concepts and incorporates polarity-aware term matching during ranking. The retrieved cases are then used as explicit evidence for multimodal reasoning and comparison through a vision-language model, producing a clinician-facing output that summarizes patient findings, highlights similarities and differences with prior cases, and supports a final assessment grounded in retrieved evidence. When designed with explicit evidence grounding and clinically informed retrieval constraints, such systems can serve as valuable tools to improve diagnostic alignment, transparency, and confidence in computer-assisted chest X-ray interpretation.

CHAPTER 1

Introduction

1.1 Motivation and Problem Statement

Chest radiography (CXR) is one of the most widely performed imaging examinations in routine clinical practice worldwide, due to its low cost, clinical utility and low radiation exposure. As a two-dimensional projection of a three-dimensional anatomical volume, chest radiographs inherently involve overlapping structures, which can obscure or mask clinically significant findings [1]. Findings may be subtle or non-specific, anatomical appearances vary between patients and age groups, and reliable interpretation is often based on the experience of the radiologist. Beyond anatomical and technical complexity, chest X-ray interpretation is also constrained by fundamental limits of human visual attention: experimental studies have shown that even experienced radiologists may fail to detect salient abnormalities when attention is directed toward a different visual task, a phenomenon known as inattentional blindness [2]. Such perceptual limitations contribute to missed findings in routine practice and further motivate the use of clinical decision support systems (DSS) that aim to complement human interpretation rather than replace it.

A primary obstacle for reliable computational support is that radiology reports contain linguistic and terminological phenomena that directly affect clinical meaning. Negation (e.g., "no pleural effusion") and uncertainty (e.g., "possible atelectasis") are frequent, while the same concept is often expressed using multiple surface forms (e.g., "haziness" or "increased density" used for *opacity*; "pulmonary atelectasis" or "atelectases" mapped to *atelectasis*). If handled naively, retrieval and comparison become clinically unreliable: synonymous descriptions may be missed, while polarity differences (affirmed vs. negated findings) may be treated as equivalent. This can lead to evidence selection that appears similar at the keyword level but is clinically inconsistent, undermining trust and traceability in downstream reasoning.

1.2 Overview of the Proposed System

The proposed system, **ThoraxDSS**, is a clinician-oriented decision support system (DSS) powered by a multimodal retrieval-augmented generation (RAG) pipeline. It accepts as input: (i) a free-text clinical query describing the patient (e.g., symptoms, suspected findings, impression or a draft report fragment), and (ii) a patient chest X-ray image.

Using these inputs, the system retrieves a small set of clinically relevant prior cases from a reference archive, where each case consists of a radiograph and its associated narrative report text (findings and impression). Retrieved cases serve as explicit evidence providing comparable imaging patterns and documented interpretations that can support clinical reasoning, highlight alternative explanations, and increase transparency in how the final output is grounded.

To retrieve relevant cases, the system performs multimodal similarity search across parallel pathways (text-based and image-based) and then combines them with a late-fusion ranking scheme. Because clinical meaning depends strongly on language phenomena such as negation and terminology variation, the fused ranking is additionally modulated using lightweight clinical term signals (canonical terms, synonym–aware matching, and polarity), so that semantically equivalent terms expressed with different wording can still align, while cases with polarity conflicts (e.g., a finding affirmed in the patient query text but negated in the retrieved case report) are discouraged during ranking.

Finally, the top-ranked cases are used in a structured retrieval-augmented generation stage where a vision–language model produces a structured summary grounded in the retrieved evidence and emphasizes consistency or discrepancies between the patient and the retrieved cases. Model outputs are constrained to a predefined schema and validated to preserve deterministic behavior and maintain stable, safe outputs.

To demonstrate and evaluate the system in a representative clinical retrieval setting, we rely on the Indiana University Chest X-ray Collection distributed via the

NLM Open-i platform [3]. The dataset includes 7,470 chest radiography images and 3,955 associated radiology reports, along with metadata linking each report to one or more images. In this work, we use the PNG images and the provided XML links to construct image–report associations (one row per linked image) which form the retrieval corpus and the unit of indexing. We also use the provided vocabulary and term-mapping resource to normalize mentions to canonical concepts (with synonyms) and to support polarity-aware (negation-sensitive) concept processing in retrieval and comparison.

All data are used for non-commercial research within the scope of this thesis.

1.3 Thesis Structure

The remainder of this thesis is organized as follows.

Chapter 2 establishes the clinical and methodological context, reviewing chest radiography reporting practices, related work in multimodal medical learning and clinical NLP, and the foundations used in this thesis.

Chapter 3 details the proposed system design, including data preparation, multimodal retrieval with late fusion, polarity- and synonym-aware ranking refinement, and the structured VLM prompting stages used for patient extraction, case comparison, and final assessment.

Chapter 4 formalizes the validation layer for VLM outputs, including strict JSON schema enforcement and the deterministic fallback policy applied when an output does not conform to the expected contract.

Chapter 5 presents the system implementation and the Streamlit user interface for configuring runs and inspecting retrieved evidence and structured outputs.

Chapter 6 reports the experimental protocol and evaluation methodology, and analyzes the system performance across the studied settings.

Chapter 7 closes with a discussion of the main findings, limitations, and directions for future work.

CHAPTER 2

Background

This chapter summarizes the clinical and methodological context underlying the proposed system. We briefly review (i) the clinical setting of chest radiography and radiology reporting, (ii) related work in multimodal medical representation learning and clinical NLP, and (iii) the methodological foundations used throughout the thesis: retrieval-augmented generation (RAG), vision–language models (VLMs), and vector similarity search.

2.1 General Knowledge and Clinical Context

Thoracic imaging can be performed with multiple modalities, including chest radiography (X-ray), computed tomography (CT), ventilation–perfusion (V/Q) scanning, echocardiography, magnetic resonance (MR), fluoroscopy, and positron emission tomography (PET). Among these modalities, the chest X-ray (CXR) is one of the most frequently performed radiological study, serving as the first line imaging modality in standard clinical workflows, due to its wide availability, speed, and comparatively low radiation exposure [4].

2.1.1 What a CXR report typically contains

CXR interpretation is primarily communicated through the radiology report. In routine practice, reports are often written in free-form prose rather than a fully standardized template, and reporting style can vary across institutions and radiologists, leading to inconsistencies in how findings are described. Recent work therefore proposes structured reporting desiderata in which reports are organized into distinct sections (such as history, technique, comparison, Findings, and Impression), with the latter two serving as the core components in structured report generation settings [5].

In this thesis, we focus on the two sections that are consistently available in our dataset and explicitly labeled as such: *Findings* and *Impression*. In typical clinical usage, Findings records imaging observations (often organized by anatomical region), while Impression summarizes the key conclusions, usually emphasizing the most clinically important points.

2.1.2 Why clinical DSS requires grounded evidence and safety checks

Clinical decision support systems (CDSS) can influence clinical actions; therefore, their design must address not only algorithmic performance but also safety, workflow integration, and human factors. Practice guidance emphasizes delivering the right information at the right time while minimizing unintended consequences [6]. In this view, a CDSS is a socio-technical intervention whose outputs must be interpretable, actionable, and aligned with clinical processes.

These principles motivate two practical requirements for radiology DSS. First, outputs should be grounded in evidence that clinicians can inspect, such as retrieved prior cases (images and associated reports), so that the system functions as an assistive tool rather than an opaque authority. Second, the system should include safety checks to reduce avoidable failure modes, including inconsistent terminology, negation-related errors, and format violations that make outputs difficult to validate or integrate into downstream workflows.

2.2 Related Work in Multimodal Radiology and Clinical NLP

Recent progress in representation learning and vision–language pretraining has enabled more effective joint modeling of medical images and associated clinical text, supporting tasks such as chest X-ray classification, cross-modal retrieval, and report generation from paired radiographs and reports. These methods suggest that shared visual–textual representations can capture clinically meaningful structure beyond unimodal baselines. However, generative systems can produce fluent yet unsupported statements (hallucinations) and may lack traceability to explicit supporting evidence—limitations that are particularly critical in clinical decision support [7, 8]. In response to these limitations, recent work explores multimodal retrieval-augmented pipelines for radiology, where similar prior cases or reference reports are retrieved to ground generation and improve factual reliability (e.g., Fact-Aware Multimodal RAG for Accurate Medical Radiology Report Generation) [9].

Beyond generation, evidence from clinical NLP benchmarks shows that robust processing of clinical text remains challenging even for extraction tasks. In benchmark evaluations, top-performing systems still misclassify a substantial fraction of clinical relations, with errors often attributed to limited explicit contextual cues and complex linguistic expression. This highlights the value of lightweight domain knowledge—canonical term matching with synonym mapping and negation-aware polarity handling—as a robustness layer when working with radiology reports [10].

This thesis builds on these lines of work while addressing a key gap for clinical decision support: reducing unsupported generation and improving traceability without sacrificing multimodal coverage. We pursue a retrieval-augmented generation (RAG) approach in which relevant prior cases are selected as explicit evidence and a vision–language model produces structured clinician-facing outputs constrained by that evidence. To better align retrieval and comparison with clinical meaning, we incorporate lightweight domain knowledge—terminology normalization (canonical terms with synonym mapping) and negation-aware polarity handling—and emphasize reliability mechanisms (structured outputs and validation) suitable for DSS use.

2.3 Methodological Background

2.3.1 Retrieval-Augmented Generation (RAG)

Retrieval-augmented generation (RAG) combines an explicit retrieval step with neural generation, so that model outputs are conditioned on external evidence rather than produced solely from the model's internal parameters [11]. This grounding is particularly relevant in clinical decision support, where evidence-backed outputs improve traceability and can reduce unsupported generations by guiding the generator to reason with retrieved context.

In the formulation introduced by Lewis et al. [12], a retriever selects relevant documents for a given query, and a generator produces an answer conditioned on both the query and the retrieved evidence. This setup grounds generation in explicit context, improving traceability and reducing unsupported content.

Preprocessing and retrieval units. In many RAG settings, large documents are segmented into smaller, semantically coherent units serving two purposes. First, it limits the amount of irrelevant context retrieved alongside a relevant finding. Second, it increases retrieval resolution, allowing the retriever to localize specific observations rather than returning an entire multi-page report. While chunking is a common design choice in large-scale RAG systems, in this thesis each radiology report is treated as a single retrieval unit containing both Findings and Impression of the reports. This choice is guided by the typical report length in the dataset (often only a few sentences), making further segmentation unnecessary and potentially harmful by separating clinically linked statements. Retrieval quality is sensitive to chunk granularity: overly large chunks may introduce irrelevant context, while overly small chunks can lose semantic coherence [13].

Retrieval component. Given the user's input(s), a retriever searches an external corpus to identify the top-k most relevant retrieval units. In neural RAG systems, this is typically implemented using dense embeddings, where both queries and corpus items are mapped into one (or more) vector spaces and ranked by a similarity metric. In practice, this similarity search can be implemented using either approximate (for low latency at scale) or exact nearest neighbor methods (for determinism). The specific embedding spaces and indexing choices used in this thesis are described in Section 2.3.3.

Augmentation component. Retrieved units are appended to the original query and formatted as part of the model's input context. This in-context grounding exposes the generator to concrete evidence at inference time, effectively conditioning generation on non-parametric memory. In addition to the retrieved units themselves, the augmented context may also include lightweight structured signals extracted from them, to guide the generator's focus and improve interpretability.

Generation (VLM integration) and structured outputs. The final input—combining the user query with retrieved evidence and, in multimodal settings, visual features—is passed to a generator. In this thesis, generation is performed with a vision–language model (VLM), enabling the output to be conditioned jointly on the patient radiograph and the retrieved report context. To support downstream use, the system elicits structured outputs that can be parsed and validated before being consumed by later stages of the pipeline.

2.3.2 Vision–language models

Vision–language models (VLMs) extend language models by incorporating visual inputs, enabling joint reasoning over images and text. A common approach is contrastive pretraining, which aligns image and text representations in a shared embedding space; CLIP is a canonical example [14]. Such aligned representations are useful for tasks that require matching images with clinical text, including retrieval and multimodal conditioning.

For clinical decision support, it is often preferable to produce structured outputs rather than unconstrained free text, because they are easier to validate, compare, and integrate into downstream workflows. Recent work studies the reliability of schema-constrained outputs for language models [15], motivating the use of structure and validation as guardrails in clinical pipelines. In this thesis, outputs are constrained to JSON so they can be deterministically parsed and validated before downstream use.

2.3.3 Vector similarity search

Vector similarity search retrieves semantically related items by ranking dense embeddings under a similarity measure. This thesis uses FAISS for indexing and retrieval [16]. Text reports and radiographs are embedded into vector spaces and ranked by vector similarity.

In our implementation, the retrieval archive consists of paired radiology reports and radiographs, where each case links a report to one or more images. Both report text and images are encoded into dense vector representations, and retrieval is performed using exact inner-product search with `IndexFlatIP` over L2-normalized embeddings, so that ranking corresponds to cosine similarity. This design prioritizes determinism and stable ranking behavior over sub-millisecond retrieval latency.

While vector similarity captures board semantic relatedness, it does not explicitly model clinically important language phenomena such as synonymy and negation. Therefore, we augment similarity-based retrieval with lightweight terminology normalization (canonical terms with synonym mapping) and negation-aware polarity handling, described in Chapter 3. In the proposed pipeline, these clinical term signals are not applied inside the embedding similarity computation. Instead, they are used as post-retrieval adjustment and reranking signals after the initial similarity scores are computed and before the final fused ranking is produced.

CHAPTER 3

System Design and Architecture

This chapter presents the system design and architectural choices used to implement the proposed multimodal DSS. It follows the end-to-end execution flow of the pipeline. First, it defines the retrieval corpus and preprocessing artifacts used to construct the image–report archive (Section 3.1). Next, it introduces the overall RAG workflow (Section 3.2) and the representation/indexing layers used for retrieval, including the text and image embedding spaces and FAISS-based similarity search (Section 3.3). It then details the multimodal retrieval stage, late-fusion ranking, and the polarity- and synonym-aware refinement applied to the Text→Text pathway (Section 3.4.1–Section 3.4.4). Finally, it describes how retrieved evidence is packaged and consumed by the vision–language model through staged, schema-constrained prompts, together with the sequential orchestration of VLM calls (Section 3.5.1–Section 3.5.3).

Chest X-ray (CXR) interpretation is a common clinical task that remains nontrivial in practice, where findings can be subtle and radiographic patterns may overlap across conditions. In this setting, clinical decision support systems are designed to assist clinicians by improving consistency, surfacing relevant evidence, and supporting transparent reasoning rather than replacing expert judgment. A key implication for DSS in radiology is that usefulness depends not only on predictive performance, but also on traceability (why a case was retrieved or a conclusion was suggested) and robust handling of clinical language, where negation and uncertainty can be as informative as presence.

3.1 Data preparation and retrieval artifacts

3.1.1 Building image–report pairs

In the Open-i Indiana University chest X-ray collection [3], report text and report–image associations are provided as individual XML report files, while the corresponding radiographs are stored separately as image files. For each report, narrative content is extracted from the Findings and Impression sections and combined into a single retrieval text field by concatenation (Findings + Impression). Report–image linking is obtained by collecting the image identifiers referenced in the report metadata and mapping them to local image filenames. The output of this step is a tabular dataset represented in-memory as a Pandas DataFrame and optionally persisted as a CSV artifact, where each row corresponds to one report-image association and includes the report identifier, image filename, extracted Findings/Impression text, and the combined retrieval text.

The resulting dataset contains the fields `report`, `image`, `findings`, `impression`, and `text`; a precise definition of each field is provided in Appendix A.

In subsequent stages, `text` is the report-side representation used for text-based retrieval and prompting.

For efficiency and reproducibility, the pipeline follows an artifact-first workflow. When precomputed artifacts are available (sampled tables, embeddings, and indices), they are reused. Otherwise, the pipeline rebuilds them from the raw Open-i reports and images and stores them for subsequent runs, reducing end-to-end runtime significantly. Details of the stored artifacts and their roles are provided in Appendix B.

3.1.2 Handling reports linked to multiple images

A single Open-i report may reference more than one radiograph. During dataset construction, the table stores one image per row. As a result, the same report identifier and report text can appear in multiple rows, each paired with a different image file. For retrieval and UI presentation, these image-level rows are additionally aggregated into a report-level gallery, so that when a report is retrieved the user can inspect the complete set of radiographs linked to that report rather than only a single file.

When candidates are merged during multimodal fusion, a single representative image is selected for visualization. The exact selection order is described in Appendix C.

3.1.3 Term resource loading and configuration

The system uses the Open-i term mapping file as an external clinical term resource to provide a controlled list of radiology concepts for rule-based matching in the report *text*. The resource provides (i) a list of canonical terms and (ii) synonym variants for each canonical term. Canonical terms are loaded from the Term column, while synonym variants are collected from all spreadsheet columns whose header begins with Synonym (the file contains multiple synonym columns with the same header, so synonyms are collected by scanning the full column set rather than assuming a single unique synonym field).

Synonym values are read cell-by-cell. If a cell contains multiple variants written together (e.g., separated by ";" or ","), they are split into separate entries and normalized before storage. In total, the resource defines **177 canonical terms**, each optionally associated with a set of synonym variants. This vocabulary is used later to normalize report phrasing and to derive concept-level polarity signals for retrieval-time adjustments (Section 3.4.4).

Before introducing the negation- and synonym-aware refinement used later in retrieval, we quantify how frequently these phenomena occur in the dataset. The goal of this analysis is not to claim dataset-specific uniqueness, but to motivate, with corpus-level evidence, why polarity handling and synonym normalization are treated as first-class design requirements in the proposed pipeline.

3.1.4 Corpus statistics for negation and synonym variability (Open-i subset)

This subsection summarizes corpus-level properties of the Open-i subset that directly affect retrieval reliability. All statistics are computed on the cleaned report *text* field used throughout the pipeline (the concatenated Findings+Impression string), after removing non-clinical placeholder tokens (e.g., anonymization markers such as "XXXX") and normalizing whitespace. The analysis uses the same curated term resource and rule inventory later applied by the backend for term matching and negation scope resolution.

Definitions. A *negation cue* is a word or multi-word phrase that explicitly introduces absence (e.g., *no*, *there is no*, *without*), as defined by the fixed cue inventory used in the system. A *cue trigger* is each occurrence of a negation cue in a report. Importantly, cue triggers are *surface-level*: the presence of a cue does not guarantee that a clinical finding is negated; it means the report contains a negation phrase that may negate one or more terms depending on the scope rules.

Negation frequency in reports. Because the dataset is represented in our pipeline as an *image–report association* (one row per linked image, with the report text duplicated when a report links to multiple images; Section 3.1.1), the counts below are computed over the 7,430 report-text instances seen during retrieval rather than the 3,955 unique reports. Explicit negation cues are widespread in radiology reporting. In the Open-i subset, **6,989/7,430 report-text instances (94.1%)** contain at least one negation cue trigger, with a **median of 2** and a **mean of 2.20** cue triggers per instance. This indicates that absence statements are routine rather than exceptional, and therefore polarity must be handled explicitly to avoid clinically misleading "matches" driven by shared keywords.

Figure 3.1 illustrates how frequently negation cues occur across report-text instances in the Open-i subset.

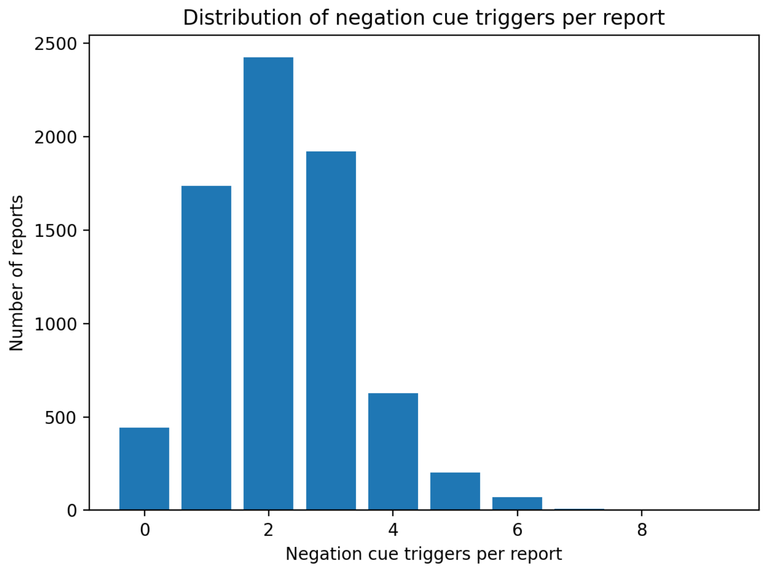


Figure 3.1: Distribution of negation cue triggers per report in the Open-i subset (Findings+Impression). The x-axis shows the number of cue triggers in a report, and the y-axis shows the number of reports with that trigger count.

Why it matters for a DSS. Negation is not an edge case: it appears in the majority of reports and often occurs multiple times within the same report. Therefore, similarity based on raw keyword overlap is likely to be clinically unreliable unless polarity is explicitly modeled.

Synonym variability of canonical concepts (vocabulary resource). Radiology language exhibits substantial surface-form variability (e.g., near-synonyms, abbreviations, alternative phrasing), which can reduce retrieval recall and distort similarity estimates when matching is purely lexical. To cope with this, the system uses the curated mapping from canonical concepts to synonym variants (previously introduced in Section 3.1.3) and applies it during rule-based concept matching in report text.

Definitions used in this analysis. We compare two concept-matching settings on the same cleaned report text: (i) *canonical-only matching*, which detects a concept only when the report contains the canonical surface form of that concept, and (ii) *synonym-enabled matching*, which detects a concept when the report contains either the canonical surface form or any curated synonym variant mapped to that concept. In both settings, matches are counted at the level of *canonical concepts*. For each report, we define the *synonym gain* as the number of additional canonical concepts detected with synonym-enabled matching compared with canonical-only matching.

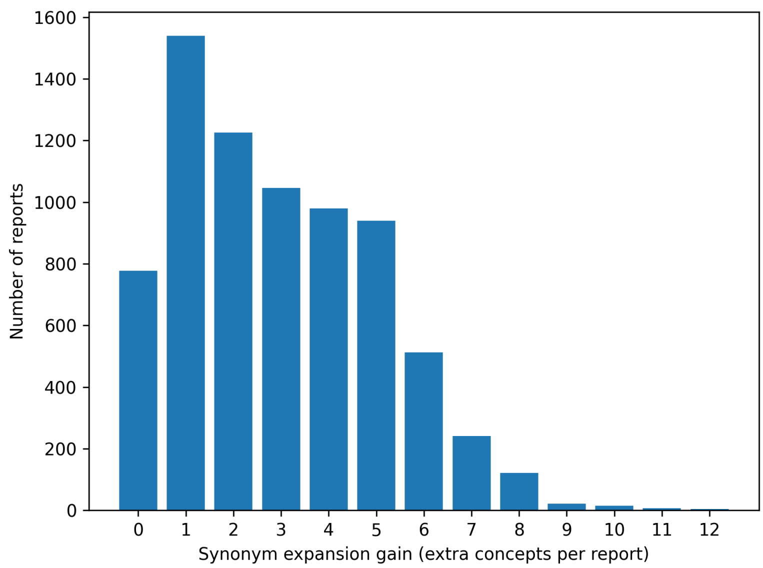


Figure 3.2: Distribution of synonym gain per report.

Corpus-level effect of synonym expansion. Synonym expansion has a substantial practical impact in this dataset. Under synonym-enabled matching, **6,653 out of 7,430 image–report associations (89.5%)** yield a *positive synonym gain*, meaning that at least one additional canonical concept is detected compared with canonical-only matching. Across all associations, the **median synonym gain is 3** concepts per instance; the **mean synonym gain is 2.95** concepts per instance, and the **maximum observed synonym gain is 12** concepts in a single instance. These results indicate that synonym variants are common in report language and that canonical-only matching systematically underestimates concept coverage.

Why it matters for a DSS. These findings show that synonym variability is not an edge case: limiting matching to canonical surface forms misses clinically relevant mentions of findings and underestimates true concept overlap between reports. This can harm retrieval and comparison by assigning lower similarity to cases that describe the same underlying findings using different wording. Synonym-enabled matching improves robustness by increasing concept coverage and making comparisons less sensitive to surface-form variation.

3.1.5 Practical implication for the pipeline

These corpus-level statistics motivate two concrete requirements for clinically meaningful similarity in the proposed DSS:

(i) *polarity-aware concept handling*, since negation cues are present in most reports and often occur multiple times within the same report, and

(ii) *synonym-enabled concept matching*, since canonical-only matching frequently misses concept mentions expressed via alternative surface forms (as reflected by the observed synonym gain).

In the proposed DSS pipeline, these signals are not used to exclude candidate cases; instead, they are used as lightweight, post-retrieval adjustments that *softly modulate* similarity scores and evidence ranking. This helps the ranking reflect whether reports describe the same concepts with the same polarity, even when phrasing differs. Further details on negation scope resolution, polarity signals, and synonym-enabled concept matching are presented in Section 3.4.4

3.2 Overview of the RAG pipeline

In this thesis, retrieval-augmented generation is implemented in a case-based manner. At query time, the system accepts:

1. a free-text clinical query q describing the patient (e.g., suspected findings, impression or a draft report fragment), and

2. a patient chest X-ray image I_p.

Figure 3.3 provides a high-level overview of the pipeline components and data flow, from patient inputs and the reference archive to retrieval, evidence fusion, and VLM-based structured generation.

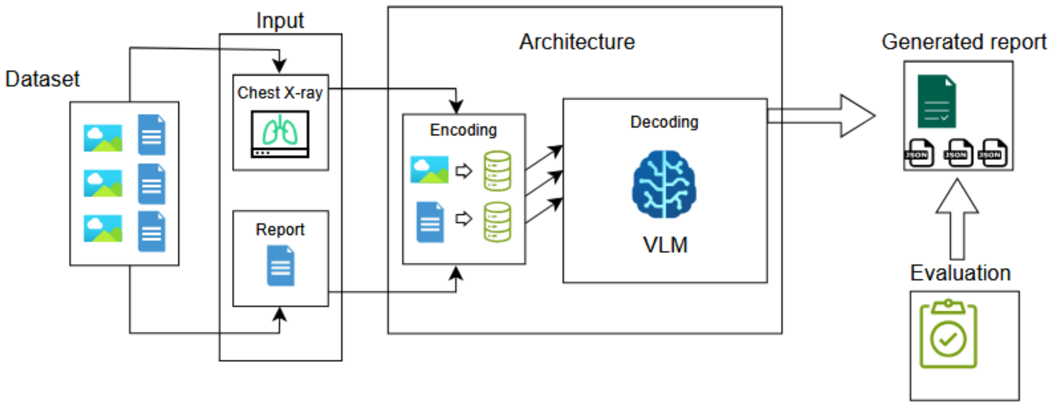


Figure 3.3: High-level workflow of the proposed multimodal RAG-based DSS.

Using these inputs, the system retrieves a small set of clinically related prior cases from a reference archive of paired images and reports (Section 3.1). Retrieved cases are treated as explicit evidence and are subsequently used to condition a vision–language model (VLM) that performs three tasks: (i) extracting patient findings, (ii) comparing the patient to each retrieved case, and (iii) synthesizing a final structured assessment grounded in the retrieved evidence.

Retrieval follows a multimodal design that combines heterogeneous similarity signals. The system performs similarity search through three pathways:

1. **Text→Text retrieval**: the clinical query q is matched against report-text representations to retrieve semantically similar reports.

2. **Text→Image retrieval**: the clinical query q is mapped into an image-aligned representation space to retrieve radiographs consistent with the described findings.

3. **Image→Image retrieval**: the patient radiograph I_p is matched against the archive radiographs to retrieve visually similar cases.

The resulting candidates are combined using late fusion: scores from each modality are merged via a weighted sum, producing a single ranked list of retrieved cases. This design allows different modalities to contribute complementary similarity signals, while keeping the fusion mechanism simple and controllable.

Negation- and synonym-aware refinement (Text→Text pathway only). To improve clinical consistency without changing the retrieved candidate set, the pipeline computes lightweight concept-level signals from report text: (i) synonym-enabled mapping of surface forms to canonical terms, and (ii) polarity labels indicating affirmed vs. negated mentions. These signals are used only to compute a bounded adjustment to the Text→Text similarity term before late fusion, while Text→Image and Image→Image scores remain unchanged.

3.3 Retrieval representations and indexing

This system indexes both report text and radiographs using dense vector representations, enabling efficient nearest-neighbor retrieval. Two embedding spaces are constructed: a sentence-level text space for report retrieval and a CLIP-style shared image–text embedding space for image and cross-modal retrieval. In both cases, embeddings are L2-normalized so that similarity can be computed using inner product search, which is equivalent to cosine similarity for unit-length vectors.

3.3.1 Text representation layer (TextEncoder, 768D)

The text pathway represents each report (Findings+Impression) and each user text query as a fixed-length dense vector using a Sentence-Transformers bi-encoder (`pritamdeka/S-PubMedBert-MS-MARCO`) [17].[1] The encoder produces a 768-dimensional

[1]Model card: `https://huggingface.co/pritamdeka/S-PubMedBert-MS-MARCO`

embedding per text input, yielding an embedding matrix of shape $(N, 768)$ for the archive.

In this implementation, text embeddings are produced with `normalize_embeddings =True`, resulting in L2-normalized (unit-length) vectors. These normalized embeddings are then used directly for similarity search with inner-product scoring.
Stored artifact: `text_emb.npy` (size $N \times 768$).
Used for: Text→Text retrieval (query text embedding vs. report text embeddings).

3.3.2 Image representation layer (ImageEncoder, 512D)

The image pathway represents each chest radiograph as a dense vector using a CLIP-style vision–language model (`microsoft/BiomedCLIP-PubMedBERT_256-vit_base_patch16_224`) [18].[2]. Each radiograph is preprocessed using the model's evaluation transform and encoded into a 512-dimensional embedding, yielding an embedding matrix of shape $(N, 512)$ for the archive.

In this implementation, image embeddings are explicitly L2-normalized to unit length after encoding. These normalized embeddings are then used directly for similarity search with inner-product scoring.
Stored artifact: `image_emb.npy` (size $N \times 512$).
Used for: (i) Image→Image retrieval (patient radiograph vs. archive radiographs).
(ii) Text→Image retrieval (text query embedded into the same multimodal space).

3.3.3 FAISS indexing with inner product (shared retrieval principle)

Both embedding spaces are indexed using FAISS with an exhaustive inner-product index (`IndexFlatIP`) [16]. Since all vectors are L2-normalized during encoding, the inner product is mathematically equivalent to cosine similarity. This exact search design yields deterministic retrieval behavior and always returns the nearest neighbors under the chosen similarity metric, providing a reproducible foundation for clinical evidence selection.

[2]Model card: `https://huggingface.co/microsoft/BiomedCLIP-PubMedBERT_256-vit_base_patch16_224`

3.4 Multimodal retrieval, late fusion and reranking

3.4.1 Query processing and Multimodal Representation

At query time, the system follows a branched control flow to transform raw user inputs into searchable representations across multiple embedding spaces. This multi-pathway approach enables the retrieval of archive cases based on textual semantics, visual morphology, and cross-modal compatibility.

Inference Inputs The retrieval process is initiated via three primary inputs:

1. **Clinical text query (q)**: A free-form patient description or suspected pathology, entered by the user.
2. **Patient CXR image (I_p)**: An optional radiograph (PNG/JPEG) uploaded via the UI to enable visual similarity search, entered by the user.
3. **Retrieval Controls**: Hyperparameters that regulate the search breadth and precision (see Appendix D.1 for more information on the parameters that control the behavior of retrieval.

3.4.2 Parallel Retrieval Pathways (T2T, T2I, I2I)

The system executes three distinct pathways to capture different facets of clinical similarity. These pathways serve as an initial retrieval stage, producing per-modality ranked candidates that are later consolidated and reranked by late fusion and the negation-aware adjustment.

Text-to-Text (T2T) Pathway The query string is mapped to a 768D semantic space using the sentence-level bi-encoder. This pathway identifies archive reports whose narrative content (Findings and Impression) is linguistically closest to the query. To ensure diversity and avoid redundancy, the system employs a seen-set deduplication filter, which returns only the highest-scoring unique report per candidate case.

Text-to-Image (T2I) Pathway The system performs cross-modal retrieval by embedding the query into a 512D shared image-text space via a prompt-engineered wrapper: "Chest X-ray showing query". This aligns natural language descriptions with the visual features of the radiographs.

Image-to-Image (I2I) Pathway When a patient image (Ip) is provided, it is processed through the visual encoder to generate a 512D query vector. The system then performs an exhaustive search in the image index to identify archive cases with similar morphology/visual markers. This allows the system to surface cases that share visual similarities which may not be explicitly detailed in the associated textual reports.

The fields returned by each pathway (`rank`, `score`, `report`, `text`, and, for image pathways, `image` and `path`) are summarized in Appendix D.2 (Table D.1).

3.4.3 Pathway Synthesis and Multimodal Late Fusion

The results returned by the parallel retrieval pathways are merged into a single candidate set at the *report* level, where each candidate corresponds to one unique radiology report (case) and aggregates all modality-specific evidence associated with it. This step consolidates overlapping hits across modalities, combines their modality-specific similarity scores, and produces a unified ranked list for downstream use.

Candidate consolidation and score alignment. The system first deduplicates candidates by report identifier, forming a single table with one row per report. It then attaches the available similarity scores from each pathway to the corresponding report. If a report is not returned by a particular pathway, the score for that pathway is set to zero, so that every candidate has the same set of score fields available for fusion (see Appendix D.3.1).

Representative Image Selection In the clinical archive, a single diagnostic report may be associated with *multiple radiographs* (e.g., frontal and lateral views). For components that require a single image (e.g., patient–case comparison and UI visualization), the system selects a representative radiograph per candidate using a fixed priority rule. The selection hierarchy favors images surfaced via the image-related pathways, since these are most directly aligned with the visual similarity signal. The exact selection hierarchy and associated fields are specified in Appendix D.3.2.

Late Fusion Scoring and Reranking The final ranking is determined by a weighted linear combination of three modality scores (Text→Text, Text→Image, Image→Image).

After consolidation, reports may enter the fused candidate set via image-based retrieval even if they were not in the top-k retrieval of Text→Text results. For this reason, the system computes a separate Text→Text score for all fused candidates to ensure every report has a consistent set of components for late fusion and reranking (see Appendix D.3.3).

The system first computes a *raw* fused score using the unadjusted Text→Text similarity for each entry. For analysis and debugging, this quantity is retained as

$$S_{\text{fused,raw}} = w_{t2i}\, S_{T2I} + w_{i2i}\, S_{I2I} + w_{t2t}\, S_{T2T}, \tag{3.1}$$

which isolates the impact of the negation-aware adjustment.

Next, a bounded negation-aware adjustment is computed from the query text and the candidate report text and is applied *only* to the Text→Text component. In addition, the adjustment computation uses lightweight terminology normalization: mentions in the query and candidate report are mapped to canonical terms via a synonym map, so that concept overlap and polarity are compared at the canonical level when forming the adjusted text score $S_{T2T_{\text{adj}}}$. Further details of the negation-aware adjustment and the canonical-term matching used to compute $S_{T2T_{\text{adj}}}$ are provided in Section 3.4.4.

The final fused score used for ranking is then:

$$S_{\text{fused}} = w_{t2i}\, S_{T2I} + w_{i2i}\, S_{I2I} + w_{t2t}\, S_{T2T_{\text{adj}}} \tag{3.2}$$

The fusion weights $\mathbf{w} = (w_{t2t}, w_{t2i}, w_{i2i})$ control the relative contribution of textual, cross-modal, and visual similarity signals. Candidates are sorted in descending order of S_{fused} and assigned ranks $1..N$.

Score-level traceability in late-fusion retrieval (illustrative example)

To support interpretability in the DSS setting, the system exposes the per-modality similarity components used in late fusion. Figure 3.4 shows an illustrative query under the default fusion weights $(w_{t2t}, w_{t2i}, w_{i2i}) = (0.50, 0.25, 0.25)$, plotting the top-$K = 10$ candidates in the final fused ranking (distinct from the downstream reasoning depth k used for VLM conditioning).

Specifically, the plot visualizes the three modality scores (S_{T2T}, S_{T2I}, S_{I2I}) used in late fusion, and makes the negation-aware refinement traceable by showing both the original text similarity S_{T2T} and the adjusted score $S_{T2T_{\text{adj}}}$ (labeled "text2text (+negation)"). It also contrasts the corresponding fused scores: the raw fusion $S_{\text{fused,raw}}$

(Eq. 3.1, computed with S_{T2T}) versus the final ranking score S_{fused} (Eq. 3.2, computed with $S_{T2T_{\text{adj}}}$ and labeled "fused (+negation)").

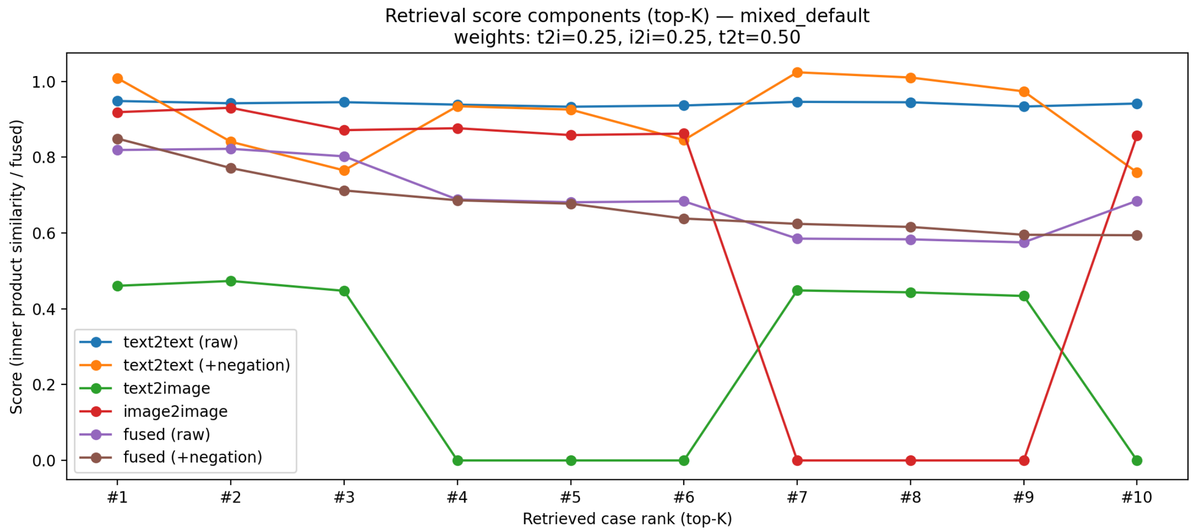


Figure 3.4: Retrieval score breakdown for an illustrative query under the default fusion weights $(0.50, 0.25, 0.25)$. Lines show modality-specific similarities (T2T, T2I, I2I), the fused score before adjustment, and the fused score after the bounded negation-aware text adjustment. Zeros indicate missing modality-specific scores after candidate union and score alignment (filled as 0.0 for modalities that did not return that candidate in the plotted list).

3.4.4 Polarity-aware and Synonym-aware reranking

Motivated by the corpus-level negation frequency and synonym variability reported in Section 3.1.4, the retrieval stage incorporates lightweight clinical term signals. The goal is not to replace embedding-based retrieval, but to make ranking more clinically consistent when the same concept appears under different surface forms and when negation flips the meaning of a mention.

Terminology normalization

Canonical terms and Synonym mapping. To reduce surface-form variation in radiology language, the system normalizes mentions in both the query text and candidate report text to a shared set of canonical concepts using the curated vocabulary introduced in Section 3.1.3. Canonical terms and their synonym variants are treated as

equivalent: if any synonym is matched in text, it is mapped back to the corresponding canonical term before comparison. This way, heterogeneous phrasing contributes to a single underlying concept before polarity is computed in the next step. In the vocabulary resource, synonym variants are extracted cell-by-cell. Multi-valued cells are decomposed by splitting on common separators and each variant is normalized before being stored as an individual synonym entry (optionally capped per canonical term). Further details of vocabulary parsing and storage are provided in Appendix E.
Illustrative example.

> **Canonical term**: Pulmonary Atelectasis
> **Synonym cells**: atelectasis | atelectases | collapsed lung | collapse; pulmonary | lobar collapse
> **Stored synonym entries**: {atelectasis, atelectases, collapsed lung, collapse, pulmonary, lobar collapse}

Negation scope and polarity extraction

After normalization, the system assigns each canonical concept a polarity label within a text. Polarity takes values in $\{+1, -1, 0\}$: $+1$ for affirmed (present) mentions, -1 for negated (absent) mentions, and 0 when the concept is not mentioned or is internally inconsistent within the same text.

Polarity is computed by detecting explicit negation cues and defining a local scope for each cue. Concept mentions inside an active negation scope are labeled -1, while mentions outside any scope are labeled $+1$. If a concept receives both $+1$ and -1 assignments within the same text, it is resolved to 0, indicating an ambiguous or internally conflicting statement.

This yields a concept-level polarity representation for (i) the user query text and (ii) each candidate report text, enabling clinically meaningful comparison of *what* is mentioned and *whether* it is stated as present or absent. The full negation cue inventory, local scope definition and polarity resolution procedure are specified in Appendix F.

Worked example: synonym normalization and negation

To illustrate the interaction between synonym normalization and polarity extraction, consider the following input texts:

Query: *No evidence of pneumonitis or pneumothorax is seen.*

Report: *Pneumonia is identified. Free air in the chest outside the lung.*

After canonical-term matching and polarity extraction, the resulting concept-level polarity maps are:

Query polarity: {*pneumonia*: -1, *pneumothorax*: -1}

Report polarity: {*pneumonia*: $+1$, *pneumothorax*: $+1$, *thorax*: $+1$, *lung*: $+1$}

This example demonstrates how heterogeneous surface forms are consolidated into canonical concepts before polarity comparison. For the concept *pneumonia*, the query uses the synonym *pneumonitis*, which is normalized to the canonical term and assigned negated polarity due to its occurrence within the negation cue "*no evidence of*". In contrast, the report explicitly affirms *pneumonia*, resulting in a polarity mismatch at the concept level.

Similarly, the query negates *pneumothorax*, because it appears within the same local negation scope initiated by the cue "*no evidence of*" , with the coordinating connector *"or"* allowing multiple consecutive concepts to fall under a single negation cue. In contrast, the report uses the synonym phrase "*free air in the chest outside the lung*", which is normalized to the same canonical concept and assigned affirmed polarity. Additional canonical anatomy terms are matched independently; in this example, *chest* is normalized to *thorax*, and *lung* is matched directly, both with affirmed polarity in the report.

Overall, the example shows how synonym normalization and local negation handling produce concept-level polarity representations that reveal clinically meaningful mismatches, even when surface phrasing differs substantially.

Retrieval use: polarity-aware adjustment of Text→Text similarity

The concept-level polarity representations are used as a lightweight post-retrieval signal to refine ranking. The adjustment is applied exclusively to the Text→Text similarity component used in late fusion (Section 3.4.3), leaving other retrieval pathways unaffected.

For each canonical concept, a signed contribution is computed based on the alignment of polarity labels between the query text and a candidate report:

- **Affirmed agreement**: query polarity $= +1$, report polarity $= +1$. A positive bonus is applied, rewarding agreement on the presence of a finding.
- **Negated agreement**: query polarity $= -1$, report polarity $= -1$. A smaller positive bonus is applied, rewarding consistent statements of absence.
- **Polarity mismatch**: query polarity $= +1$ and report polarity $= -1$, or vice versa. A strong negative penalty is applied, discouraging clinically inconsistent matches.
- **Single-sided or unresolved mention**: one text has polarity $\in \{+1, -1\}$ for a canonical concept, while the other has polarity $= 0$ (not mentioned or internally conflicting). A mild negative penalty is applied, reflecting incomplete or unresolved concept alignment.

The resulting contributions are summed over all canonical concepts that are mentioned in at least one of the two texts, normalized by the number of such concepts, scaled, and clamped to a bounded range.

The final adjusted Text$\rightarrow$Text similarity used in late fusion is given by:

$$S_{T2T_{\text{adj}}} = S_{T2T} + A_{\text{pol}}.$$

Here S_{T2T} is the base embedding similarity score, and A_{pol} is the bounded polarity adjustment computed as described above.

Notably, this signal is used to *modulate* similarity rather than to hard-filter candidates: retrieved cases remain visible, but their rank better reflects concept-level agreement in both concept identity (under the system's terminology normalization) and polarity.

The polarity labels and the subsequent term-by-term query–report comparison used in this adjustment are specified in Appendix F, Section F.4.

3.5 VLM evidence conditioning and structured generation

3.5.1 Evidence packaging for the VLM (augmentation inputs)

For the subsequent VLM calls, the system constructs a bounded augmentation bundle that contains only the artifacts required for the downstream pairwise-comparison and final-synthesis calls. The bundle is built from (i) the top-n retrieved cases, where $n \leq K$ is the number of cases for which both an image and a report text are available after alignment, and (ii) the validated patient-only structured finding summary produced by the first VLM call. These two sources are combined to provide per-case evidence for comparisons and compact, deterministic guidance signals.

Patient-only structured finding summary (single bundle item). The first VLM call returns a short list of **4–5** patient findings, where only the main finding carries certain structured tags. This validated summary is retained as-is for later conditioning, and a tag-stripped text view is derived from it to support deterministic text analysis.

Retrieved case evidence (per case). For each aligned retrieved case ($i = 1 \dots n$), the bundle stores exactly: (i) **one** representative case radiograph (selected from fused retrieval outputs with deterministic fallback selection when needed), and (ii) the corresponding **full cleaned report text** (placeholders removed). A full per-report image gallery may also be retained for UI inspection, but only the representative radiograph is used for VLM conditioning.

Deterministic text-derived signals (per case). Using the tag-stripped patient text view (falling back to the original user query if needed) and each retrieved report text, the system computes compact per-case signals under a synonym-enabled canonical term mapping and deterministic negation scope rules. Two signal types are produced: (i) optional term anchors indicating supported overlaps and differences, and (ii) explicit polarity-mismatch warning strings when a concept is affirmed in one text but negated in the other. To control context length, generic anatomy/attribute tokens are filtered and each per-case list is truncated to a small fixed limit before use.

Aggregated warnings (single bundle item). All per-case polarity warnings and validation-related warnings are merged into one ordered list and then truncated to a fixed maximum length before being passed to the final synthesis stage.

3.5.2 Generation with structured prompting (VLM)

Role of the VLM in the proposed system. In the proposed DSS pipeline, vision–language models (VLMs) are used not as free–form generators, but as controlled reasoning components operating under strict structural constraints. The VLM is invoked through three specialized prompts corresponding to distinct reasoning stages in the pipeline. Each prompt operates under a fixed input–output contract and produces structured outputs that are consumed by subsequent stages. All prompts produce machine–validated structured outputs; validation and failure handling are described in Chapter 4.

Runtime model selection. The system supports configurable VLM backends at runtime (defaulting to a primary model, with optional selection of alternative compatible models). This enables comparative experimentation under the same prompt contracts and validation rules. Implementation details of the UI–level model selection are provided in Chapter 5.

The three prompts correspond to distinct reasoning stages:

1. **Patient–only extraction**: Produces a structured summary of patient findings from the patient inputs.
2. **Pairwise patient–case comparison**: Compares the patient to each retrieved case and outputs structured overlap/differences grounded in the retrieved evidence.
3. **Final clinician–facing assessment**: Synthesizes a final structured assessment based on the patient and the retrieved evidence.

Prompt-specific augmentation. The three prompts use different subsets of the packaged evidence: (1) patient-only uses only patient inputs; (2) pairwise receives the per-case image–report plus *case-specific* warning strings and optional term anchors; (3) final assessment receives patient inputs, validated structured outputs from earlier stages, and only *aggregated* signals (counts/summary objects and a capped warning list), not raw retrieval scores or full case text.

1. Patient–only prompt: structured finding extraction

Purpose. This prompt extracts a concise, structured summary of patient findings grounded *only* in the patient chest radiograph and the accompanying clinical summary text. The resulting patient–centric representation is reused by downstream comparison and synthesis stages.

Inputs.

(i) **Patient clinical summary text** (free–form radiology-style description or report fragment).

(ii) **Patient chest X-ray image** (single radiograph).

Structured output (conceptual: `PATIENT_FINDINGS`). The model returns a single JSON object whose key `patient_findings` stores the `PATIENT_FINDINGS` list: a short list of **4–5 distinct finding sentences**. The list consists of one primary abnormality followed by secondary supporting observations, and each finding is expressed as a single, atomic sentence.

Primary finding (tagged). One finding is designated as the primary abnormality and is expressed using a fixed sequence of structured tags followed by the finding sentence:

```
[LAT=...] [SIZE=...] [SHIFT=...] [DIR=...] finding sentence
```

The tags encode clinically salient attributes of the primary abnormality:

- **LAT (laterality)**: `right | left | bilateral | unknown`
- **SIZE (extent/severity)**: `small | moderate | large | unknown`
- **SHIFT (mediastinal/tracheal shift present)**: `yes | no | unknown`
- **DIR (shift direction)**: `left | right | none | unknown`

The value `unknown` must be used only if a tagged attribute cannot be confidently inferred from the available evidence.

Secondary findings (un–tagged). All remaining findings are expressed as short, atomic sentences without structured tags. These findings capture additional observations or contextual details and are presented alongside the primary finding within the same structured output.

Design rationale. Structured tags are applied only to the primary abnormality, while secondary findings remain unconstrained descriptive statements. This avoids forcing unsupported attribute assignments while preserving interpretability.

Appendix reference. The exact prompt template used for this stage is provided in Appendix G, Section G.1.

2. Pairwise patient–case comparison prompt

Purpose. This prompt compares the patient against a single retrieved reference case at a time, integrating visual and textual evidence to assess clinical similarity and identify supported overlaps and differences.

Inputs. For each patient–case comparison, the VLM receives:

(i) **Image 1** (the patient chest X-ray image I_p) and **Text A** (the original patient clinical query text q),

(ii) **Image 2** (the representative radiograph of the current retrieved case) and **Text B** (a fixed-length snippet of the corresponding retrieved case report, with its case identifier),

(iii) **Warnings** (a short list of system-generated warning strings, provided only as guidance for consistency checking and not to be quoted), and

(iv) **Term hints** (optional overlap and case-only term lists provided as anchors; they are cues only and must not be copied as standalone outputs).

The warning strings and term hints are computed deterministically from canonical-term and polarity comparisons between the retrieved report text and a patient-side text view derived from the validated `PATIENT_FINDINGS` output (with fallback to q when needed), and are passed as separate guidance fields rather than as Text A.

Structured output (conceptual). For each retrieved case, the model produces one JSON object containing:

- `case_findings`: a compact list of exactly three visual findings describing the retrieved case image (Image 2 only),
- `similarity_to_patient`: a categorical similarity assessment between the patient and the retrieved case, chosen from {`similar`, `partially similar`, `not similar`}, based on the combined patient–case evidence (Image 1 vs. Image 2 and Text A vs. Text B),
- `common_with_patient`: a short list of overlaps supported by the two report texts (Text A vs. Text B),
- `differences_from_patient`: a short list of case-vs-patient differences supported by the two report texts (Text A vs. Text B), and
- `one_line_rationale`: three short sentences summarizing the basis of the assessment, explicitly referencing both report-text alignment and visual alignment.

Design rationale. The system performs independent pairwise comparisons rather than joint reasoning over all retrieved cases. This mirrors clinical practice, where reference cases are assessed individually, and enables fine-grained validation and error handling at the level of each patient–case comparison.

Appendix reference. The exact prompt template used for this stage is provided in Appendix G, Section G.2.

3. Final clinician-facing assessment prompt

Purpose. This prompt produces a consolidated, clinician-facing assessment of the validated patient findings and the aggregate evidence retrieved from prior cases. The VLM is used in an evaluative role: it synthesizes existing signals, checks for consistency between image and text evidence, and reports a calibrated confidence level without introducing new primary findings.

Inputs. The final assessment stage is conditioned on:

(i) **Image 1** (the patient chest X-ray image I_p) and the **CLINICAL SUMMARY** text (the original patient clinical query text q),

(ii) **PATIENT_FINDINGS**, i.e., the structured findings list produced by the patient-only stage,

(iii) **Aggregated consistency signals** derived from pairwise comparisons (counts of cases labeled `similar`, `partially similar`, and `not similar`; no retrieved case images or case report text are provided), and

(iv) **Aggregated system signals** used only for calibration (a total warning count and a count of canonical terms labeled as negated in the patient-side polarity map; individual terms and case identifiers are not exposed).

Structured output. The model returns a single JSON object with:

- `text`: a clinician-facing narrative with fixed headings (Evidence, Case Consistency, Negation Check, Impression, Confidence Rationale), and
- `confidence`: a discrete confidence label in {`low`, `medium`, `high`}.

Design rationale. This prompt is deliberately constrained to prevent uncontrolled synthesis. The model cannot introduce new primary abnormalities, cannot reference individual retrieved cases, and must base confidence on explicit, interpretable signals such as image support, retrieval consistency, and system warnings. Negation is treated as a routine property of radiology language and is explicitly prevented from acting as a direct confidence penalty.

Appendix reference. The exact prompt template used for this stage is provided in Appendix G, Section G.3.

3.5.3 Sequential VLM orchestration and dependency structure

This section describes the temporal and logical execution order of the Vision–Language Model (VLM) calls in the proposed system. The pipeline follows a strictly sequential orchestration, where later stages depend on validated outputs from earlier stages. This ordering is intentional and reflects a DSS-oriented principle: later reasoning stages must not bypass earlier evidence extraction steps.

Stage 1: Multimodal retrieval and fusion (deterministic, pre–VLM). Before any VLM interaction, the system retrieves the top-K candidate cases using the multimodal retriever and late-fusion reranking (Section 3.4.3). This stage outputs a report-level ranked list with linked case images and report text, together with fused similarity scores used for traceability and diagnostics. Retrieval and reranking are fully deterministic and complete before any VLM call.

Stage 2: Patient–only VLM extraction (anchors downstream reasoning). The VLM is first invoked on the patient inputs to produce a validated, patient-centric finding summary under a fixed schema (Section 3.5.2). This output provides the anchor representation used by subsequent stages.

Stage 3: Deterministic clinical signals (post patient-only, pre-pairwise). After patient-only extraction, the system computes lightweight term-level signals outside the VLM by comparing the patient representation against each retrieved report. This includes canonical-term normalization and polarity comparison (affirmed vs. negated), as described in Section 3.4.4. From these signals, the system derives structured warnings for clinically relevant polarity mismatches. These artifacts are retained for provenance and are optionally provided to the VLM only as constrained guidance in later stages.

Stage 4: Pairwise VLM comparison (per retrieved case). For each retrieved case ($i = 1 \ldots K$), a pairwise VLM call compares the patient against that case using the patient inputs and the case evidence, together with optional structured hints/warnings from Stage 3. Each call returns a validated, case-specific comparison summary (Section 3.5.2). Pairwise outputs are additionally post-processed with lightweight rule-based scrubbing and consistency checks before aggregation. Validation failures trigger a deterministic fallback.

Stage 5: Final assessment VLM call (VLM as an adjudicator). The final VLM call produces a consolidated clinician-facing assessment under strict constraints. It does not receive raw retrieval scores or full case reports; instead it consumes the patient inputs, the validated patient findings, and structured per-case comparison summaries together with aggregated warning signals derived from earlier stages. The model's role is evaluative rather than exploratory: it assesses whether the retrieved evidence collectively supports the patient findings and returns a calibrated confidence level. Output validation and deterministic fallback ensure stability for decision support.

CHAPTER 4

Validation

This chapter describes the validation layer that enforces strict input–output contracts around all VLM interactions. Each VLM call is expected to produce a JSON object under a fixed schema. Patient-only and pairwise contract failures trigger a deterministic fallback response for the request, while final-assessment failures are handled by a local deterministic fallback for the final assessment fields.

Validation is designed to be minimal and structural, enforcing schema-level constraints (required keys, JSON types, cardinality/index alignment where applicable, and enum checks) while leaving clinical semantics to earlier deterministic signals and later prompting constraints.

4.1 Validation interface and outcomes

All validators return a dictionary with the fields: `{ok, errors, warnings, logic_ok}`.

- `errors`: hard contract violations (missing keys, wrong types, cardinality/index misalignment). Any non-empty `errors` makes the output invalid.
- `warnings`: non-fatal irregularities kept for traceability (e.g., unexpected labels, meta phrasing, potential cross-case leakage). Warnings never block execution.
- `ok` and `logic_ok`: boolean validity summaries. Under the current policy they are equivalent and true if `errors` is empty; `ok` is used at orchestration boundaries, while `logic_ok` is reserved for future separation of structural vs. higher-level checks.

4.2 Schema contracts

The system enforces three schema contracts, aligned with the three VLM calls: (i) patient-only extraction, (ii) pairwise patient–case comparison, and (iii) multicase aggregate output. For each contract, we list the required keys together with their intended interpretation, followed by the enforced type/enum/alignment constraints.

In this chapter, "contract" refers to the enforced JSON structure and alignment constraints. Content-shaping requirements such as phrasing style and sentence count are handled by prompt design and downstream normalization rather than by the validator.

4.2.1 Patient-only contract

Required fields and interpretation. The output must be a JSON object containing:

- `patient_findings`: a short list of patient-centric finding sentences extracted from the patient inputs.

Type constraints. `patient_findings` must be a list of strings. Optional tag formatting for the primary finding is accepted but not required for validity. When present, tags follow the form: `[LAT=...][SIZE=...][SHIFT=...][DIR=...]` prefixed to the finding sentence. Missing or malformed tags are permitted and do not invalidate the output under the current policy.

The 4–5 findings count and primary-tag formatting are enforced by prompting and downstream normalization expectations. Under the current validation policy, patient-only checks remain structural (JSON type/field presence) rather than semantic.

4.2.2 Pairwise contract

Required fields and interpretation. The output must be a JSON object containing:

- `case_findings`: a short list of salient findings for the retrieved case, produced by the VLM and then post-processed with deterministic hygiene steps (filtering, deduplication, and length capping) to stabilize the output.
- `similarity_to_patient`: an overall qualitative similarity label for this case relative to the patient.

- `common_with_patient`: a short list of overlaps between patient and case report text, as produced by the VLM under the pairwise prompt constraints and optional deterministic term hints.

- `differences_from_patient`: a short list of patient–case differences, as produced by the VLM and then post-processed with deterministic filters.

- `one_line_rationale`: a short rationale string (stored as a single JSON string field) summarizing the similarity judgment. In the prompt contract, this string is requested to contain three short sentences covering both report-text alignment and visual alignment.

Type constraints. `case_findings`, `common_with_patient`, and `differences_from_patient` must be lists of strings, `one_line_rationale` and `similarity_to_patient` must be strings.

Enum constraints. `similarity_to_patient` must be one of: {`similar`, `partially similar`, `not similar`}, any other value is treated as a contract violation.

4.2.3 Final assessment contract

Required fields and interpretation. The final structured output must be a JSON object containing:

- `patient_findings`: the patient findings list produced by the patient-only stage.

- `retrieved_cases`: a case-aligned list of per-case UI entries (one entry per processed case), each storing the `case_index`, a compact `findings` list, and a `similarity_to_patient` label.

- `comparison_summary`: a case-aligned list of per-case comparison objects (one per processed case) that is passed as structured input to the final assessment call. Each item contains `case_index`, `similarity_to_patient`, `common_with_patient`, `differences_from_patient`, and `one_line_rationale`.

- `final_diagnosis_assessment`: the final clinician-facing assessment object containing `text` and `confidence`.

Cardinality constraints. Let n denote the number of cases actually processed after alignment. Then both `retrieved_cases` and `comparison_summary` must have length exactly n.

Index alignment constraints. For $i = 1, \ldots, n$, `retrieved_cases[i]` and `comparison_summary[i]` must include `case_index=i`. Mismatches are treated as errors because they break case-to-summary correspondence.

Final assessment fields. `final_diagnosis_assessment` must be a JSON object. In the expected format, it includes:

- `text`: the final consolidated narrative assessment.
- `confidence`: a discrete confidence label summarizing overall support.

Enum constraints. `final_diagnosis_assessment.confidence` should be one of {`low`, `medium`, `high`}. Unexpected values are retained as warnings.

Additional warning checks. Non-fatal warnings are emitted for potential cross-case leakage in `one_line_rationale` (i.e., references to a different case index) and for meta-level or absence-style phrasing in `differences_from_patient`.

4.3 Validation checkpoints and enforcement

Validation is enforced at three points in the runtime pipeline:

1. **After patient-only extraction**: The patient-only JSON is parsed and validated immediately. If it fails, the system returns a deterministic fallback response.
2. **After each pairwise comparison**: Each case-specific JSON is validated independently. If any pairwise output fails, the system returns a deterministic fallback response for the full request.
3. **After multicase assembly**: The composed multicase object is validated against the expected case count n and schema constraints. Validation warnings are then augmented with system-generated warning streams for provenance.

4.4 Failure handling and fallback behavior

When a VLM response is returned, the system first applies light JSON parsing repair (e.g., code-block stripping, extraction of the first balanced JSON object, and trailing-comma cleanup). For the patient-only and pairwise stages, the parsed object is then validated against the stage-specific schema. For the final-assessment stage, the system applies field-level checks on the returned `text`/`confidence` values and falls back locally if they are not usable for downstream output.

If the patient-only output fails parsing/validation, or if any pairwise case output fails (including parsing, schema validation, or pairwise-call failure), the system returns a fixed fallback response for the full request. This fallback preserves the expected top-level JSON structure, includes empty `patient_findings`, case-indexed placeholder entries for retrieved cases, and a final assessment with `low` confidence that explicitly includes the failure reason; deterministic comparison entries are included when they are already available.

If the final-assessment stage fails, the system does not discard the already assembled patient-only and pairwise outputs. Instead, it preserves the assembled structured output and replaces only the `text` and `confidence` fields inside `final_diagnosis_assessment` with fallback values.

4.5 Downstream interpretation

Across all stages, orchestration uses only the boolean validity signal (`ok`/`logic_ok`) to decide whether to proceed or to trigger a deterministic fallback response. The full contents of `errors` and `warnings` are preserved and surfaced for traceability and debugging, but are not partially repaired or selectively ignored within the validation layer.

CHAPTER 5

System Implementation and User Interface

This chapter describes the prototype implementation and the Streamlit-based user interface (UI) [19] used to operationalize the proposed multimodal decision support system **ThoraxDSS** in an interactive setting. The UI collects clinician-facing inputs (the clinical summary text q and the patient radiograph I_p), exposes runtime controls for retrieval depth (k), late-fusion retrieval weights, and VLM model selection, forwards requests to the backend pipeline, and renders structured outputs together with evidence and diagnostic views.

5.1 System architecture and UI integration

The prototype separates the Streamlit user interface layer from the core processing pipeline. The backend pipeline performs retrieval, reranking, evidence packaging, VLM inference, and schema validation, while the UI is responsible for input collection, runtime parameter configuration, request submission, and output visualization.

5.2 User interaction workflow

A typical run starts from a new patient query consisting of (i) free-text clinical summary and (ii) an uploaded chest radiograph. The user may also configure retrieval depth and optional runtime settings (advanced model selection and late-fusion weights) before triggering execution. The UI then renders the validated structured output and provides evidence inspection and diagnostics for traceability.

Figure 5.1 shows the initial landing view of the prototype, including the primary input fields, retrieval-depth control, and the expandable panels for advanced model settings and retrieval weights.

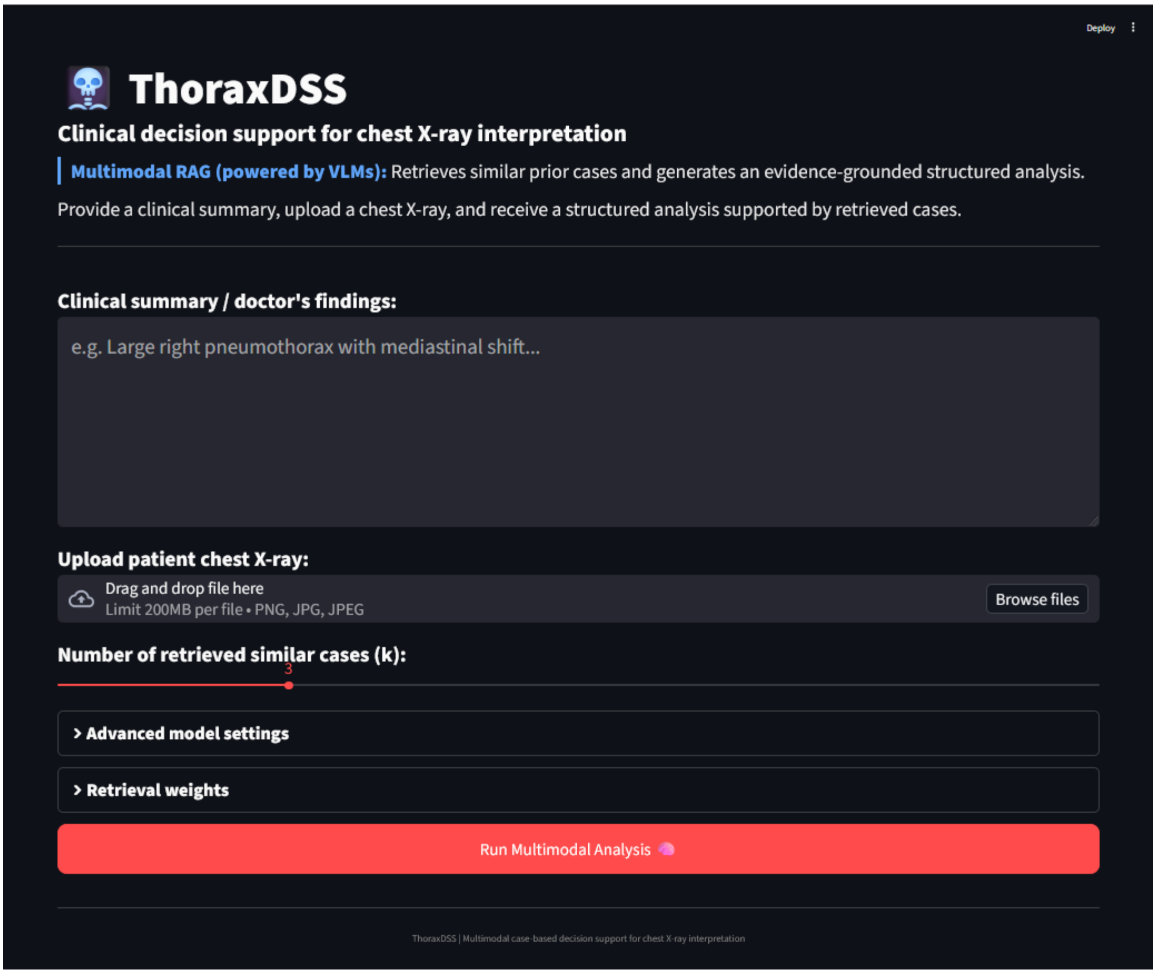


Figure 5.1: Initial UI landing view showing primary patient inputs, retrieval-depth control, and collapsed runtime-configuration panels.

5.2.1 Patient input specification

The primary user inputs are:

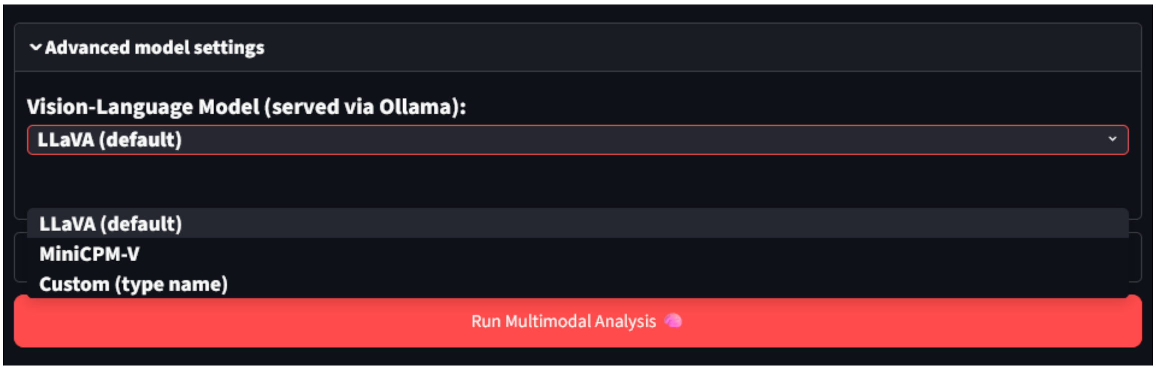
- **Clinical summary text** (q): free-form text describing suspected findings or a draft report fragment.

- **Patient chest radiograph (I_p)**: a single uploaded chest X-ray image (PNG/JPEG), passed to the backend as a temporary file.
- **Number of retrieved cases (k)**: user-selected retrieval depth ($k \in [1, 10]$, default $k = 3$).

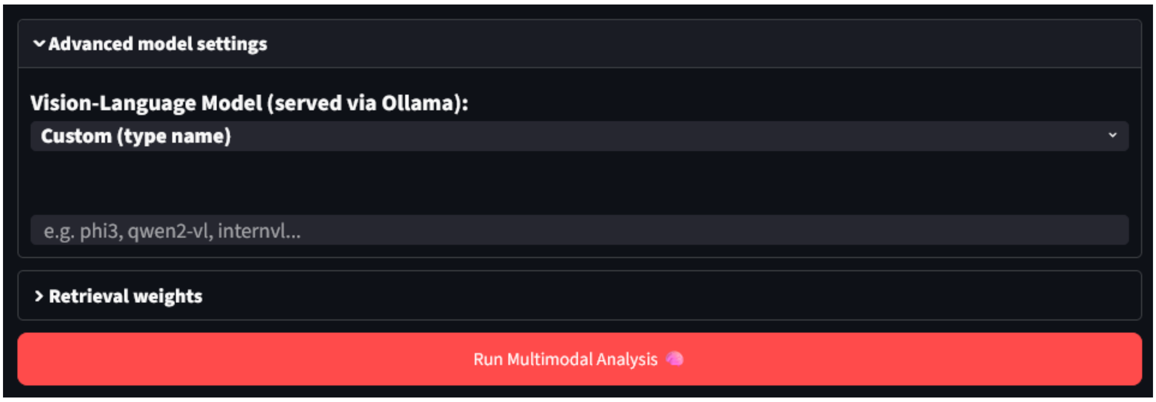
5.2.2 Runtime configuration options

The UI exposes an *Advanced settings* panel that controls: (i) which vision–language model is used for generation, and (ii) how modality scores are weighted during late fusion.

Figure 5.2 shows the Advanced settings panel used to select the vision–language model at runtime, either from built-in options or via a custom Ollama identifier.



(a) Built-in VLM selection (LLaVA / MiniCPM-V).



(b) Custom Ollama model identifier input.

Figure 5.2: Advanced settings for runtime VLM selection.

Vision–language model selection

The system supports the following VLM choices:

- **LLaVA (default)**: mapped to the Ollama model identifier `llava` [20].
- **MiniCPM-V**: mapped to the Ollama model identifier `minicpm-v` [21] .
- **Custom**: the user may enter an arbitrary model identifier, provided that the corresponding model is available locally. If the custom field is empty or the specified model is unavailable,the system falls back to `llava`.

The selected model identifier is passed unchanged to the backend VLM handler and is used consistently for all VLM calls in the pipeline (patient-only extraction, pairwise comparisons, and final assessment).

Late-fusion retrieval weight controls

The weight sliders controlling late-fusion retrieval, allowing interactive adjustment of Text→Image and Image→Image weights, while the Text→Text contribution is computed automatically, are illustrated in Figure 5.3.

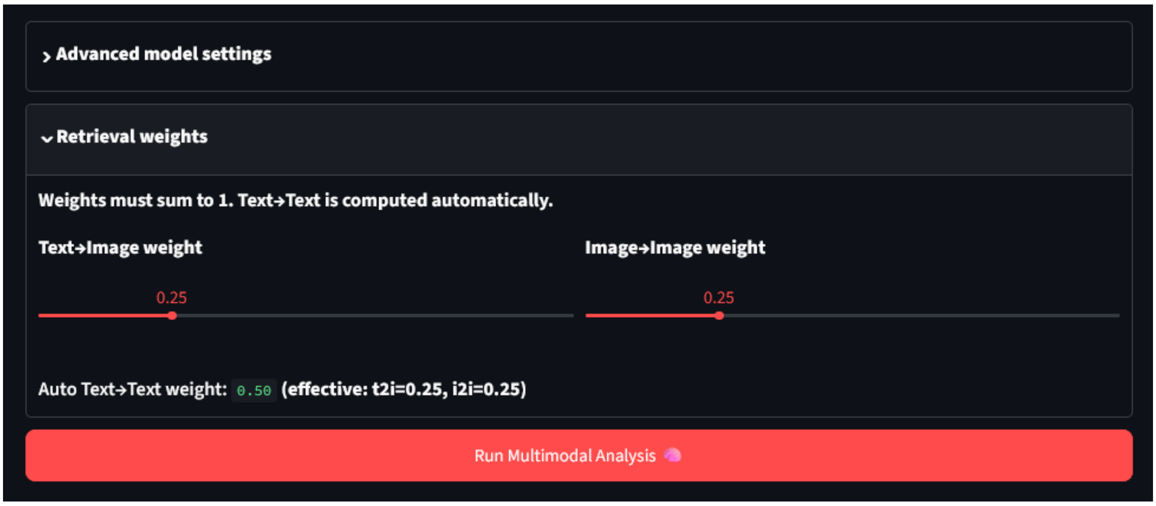


Figure 5.3: Late-fusion retrieval weight controls.

The UI allows interactive adjustment of the late-fusion weights for the two image-involving pathways:

$$w_{t2i} \quad (\text{Text}{\rightarrow}\text{Image}), \qquad w_{i2i} \quad (\text{Image}{\rightarrow}\text{Image}).$$

The Text→Text weight is computed automatically as:

$$w_{t2t} = \max\left(0,\ 1 - w_{t2i} - w_{i2i}\right).$$

To prevent degenerate settings that eliminate the text component, the UI enforces a small reserve for Text→Text scoring by rescaling (w_{t2i}, w_{i2i}) when $w_{t2i} + w_{i2i} > 0.95$, ensuring $w_{t2t} \geq 0.05$.

Defaults. By default, the UI initializes $w_{t2i} = 0.25$ and $w_{i2i} = 0.25$, resulting in $w_{t2t} = 0.50$. The effective weights are written into the backend retrieval configuration before retrieval is executed.

5.2.3 Request execution

Once the clinical summary, radiograph, and retrieval parameters are set, execution is triggered via the *Run Multicase Analysis* control. The interface packages the text input, the uploaded image, and the selected runtime settings (k, model identifier, and fusion weights) into a single request and forwards it to the backend pipeline. During processing, the interface displays a running state indicating that retrieval, reranking, and backend VLM calls are in progress. When processing completes, the UI renders the validated structured output and enables the evidence and diagnostics tabs for inspection.

Figure 5.4 shows the interface immediately after request submission, while the backend pipeline is executing (retrieval, reranking, and VLM calls).

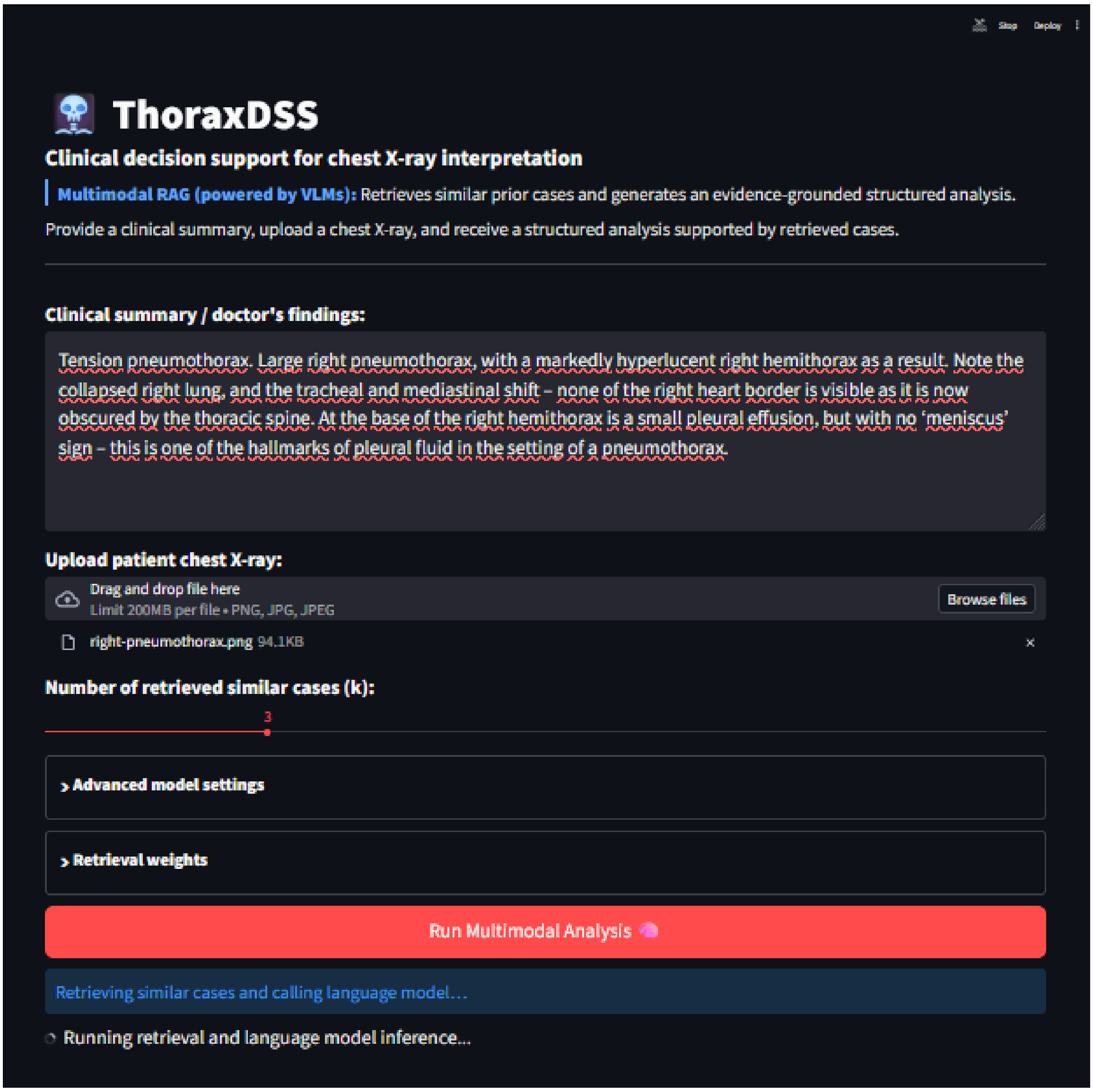


Figure 5.4: Run execution after submitting inputs and settings.

5.2.4 Clinician-facing output

The UI presents the validated structured outputs produced by the backend pipeline in a stage-aligned format, rather than as a single monolithic block. This presentation mirrors the three VLM stages in the pipeline: (i) patient-only extraction, (ii) pair-wise patient–case comparison for each retrieved case, and (iii) final clinician-facing assessment (see Section 3.5.2).

For readability, the interface renders each stage in a dedicated layout. The patient-only stage is shown as a compact findings list, where the primary finding includes the tagged attributes (`LAT`, `SIZE`, `SHIFT`, `DIR`) and the remaining findings are displayed as plain text items. Pairwise and final-stage outputs are presented in separate sections together with their corresponding evidence context.

Figure 5.5 shows the patient-only output as rendered in the UI.

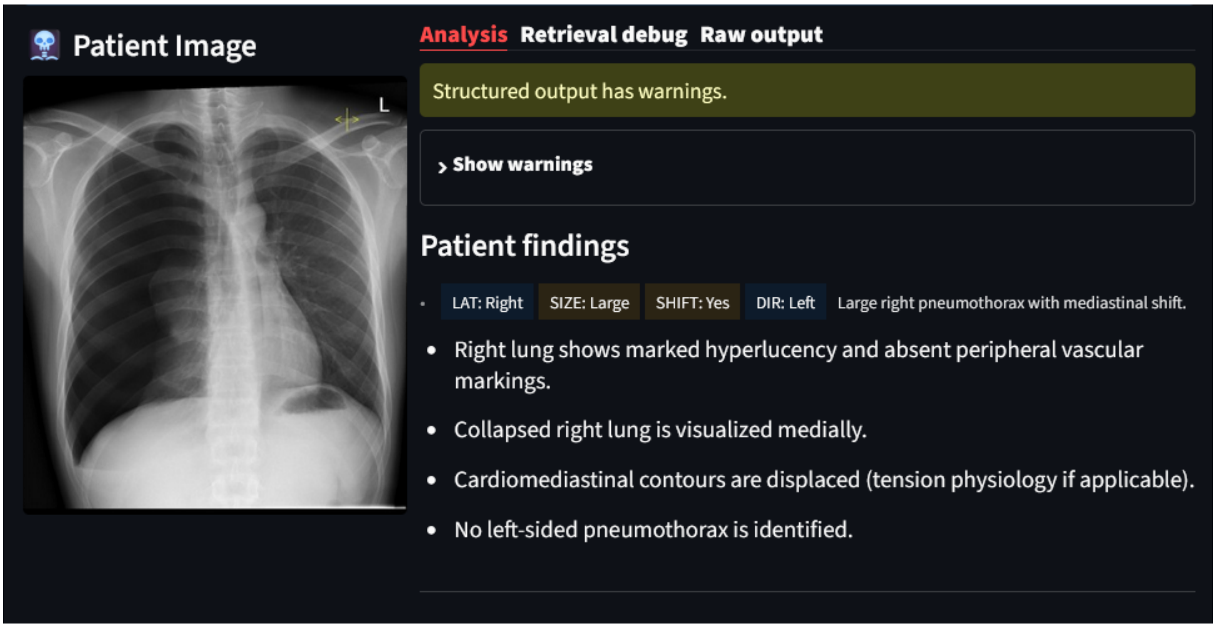


Figure 5.5: Structured patient findings as rendered in the UI (patient-only extraction stage), including the primary tagged finding derived from the patient radiograph and clinical summary text. **Patient image source**: SVUH Radiology, "Tension Pneumothorax" case study (`http://www.svuhradiology.ie/case-study/tension-pneumothorax/`). Reproduced for academic illustration.

The pairwise stage outputs are presented together with the retrieved-case evidence context used for each comparison. In the UI, each retrieved case is shown as a case card with its structured pairwise summary, expandable report text, and linked radiographs from the same report.

Figure 5.6 shows the top-ranked retrieved case as rendered in the UI, with the case report expanded and the linked radiographs displayed for inspection.

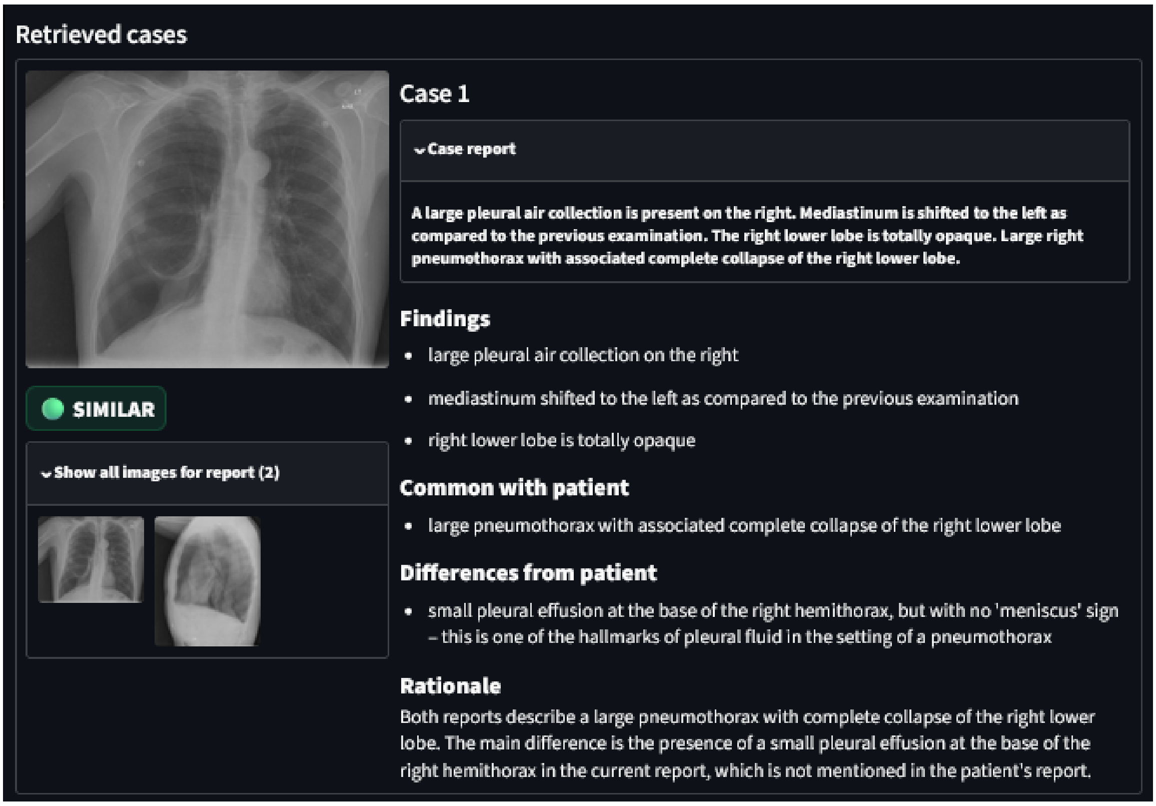


Figure 5.6: Retrieved case card (rank 1) as rendered in the UI, with the case report expanded and the linked radiographs displayed for evidence inspection (pairwise patient–case comparison stage). **Source**: Indiana University Chest X-ray Collection via NLM Open-i. Reproduced unmodified with attribution.

Figure 5.7 shows the second-ranked retrieved case in the same UI format, again with the report text and linked radiographs available for inspection.

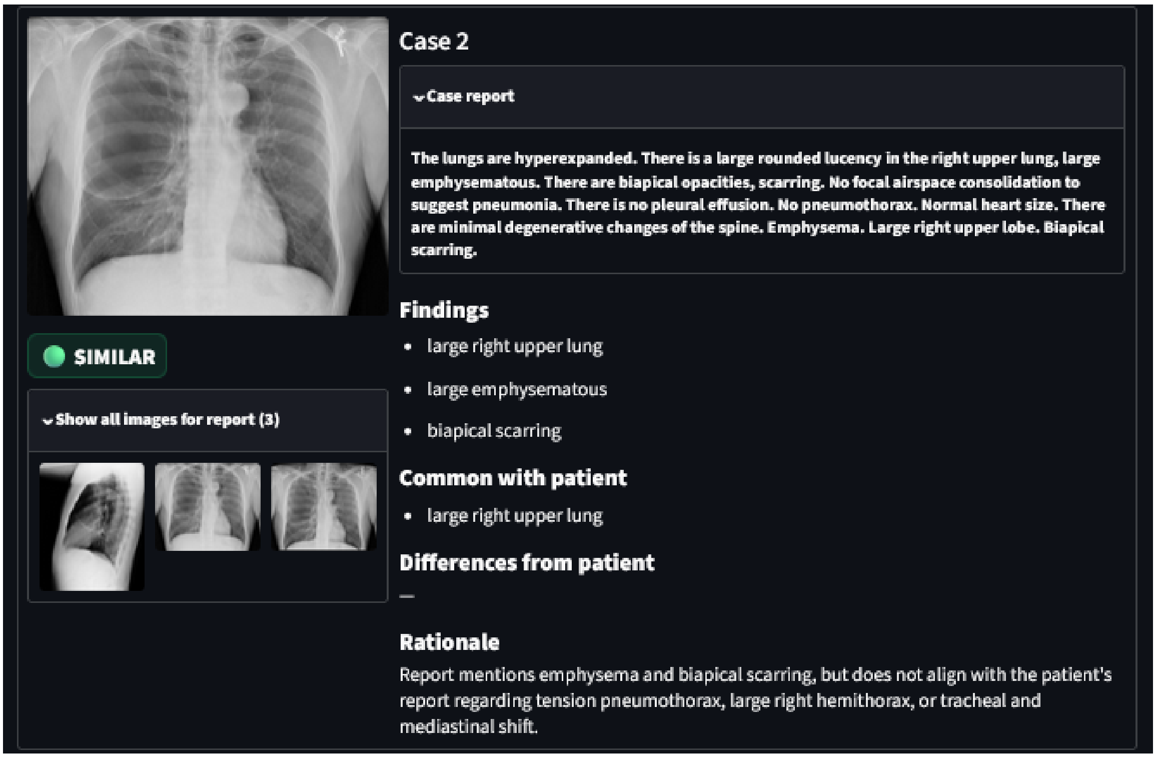


Figure 5.7: Retrieved case card (rank 2) as rendered in the UI, with the case report expanded and the linked radiographs displayed for evidence inspection (pairwise patient–case comparison stage). **Source**: Indiana University Chest X-ray Collection via NLM Open-i. Reproduced unmodified with attribution.

Figure 5.8 shows the third-ranked retrieved case in the same evidence-inspection layout.

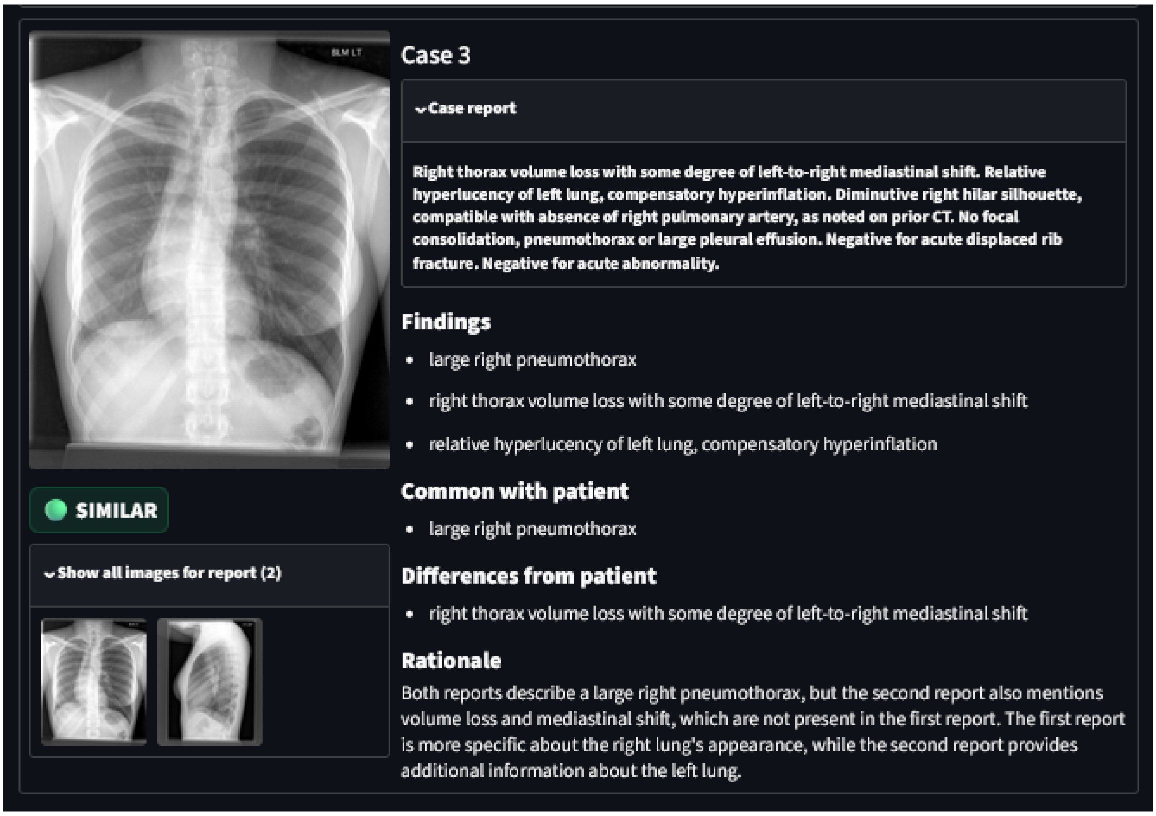


Figure 5.8: Retrieved case card (rank 3) as rendered in the UI, with the case report expanded and the linked radiographs displayed for evidence inspection (pairwise patient–case comparison stage). **Source**: Indiana University Chest X-ray Collection via NLM Open-i. Reproduced unmodified with attribution.

The final stage output is then presented as a separate clinician-facing assessment panel containing the narrative assessment and confidence label.

Figure 5.9 shows the final assessment output as rendered in the UI, including the confidence level and supporting rationale text.

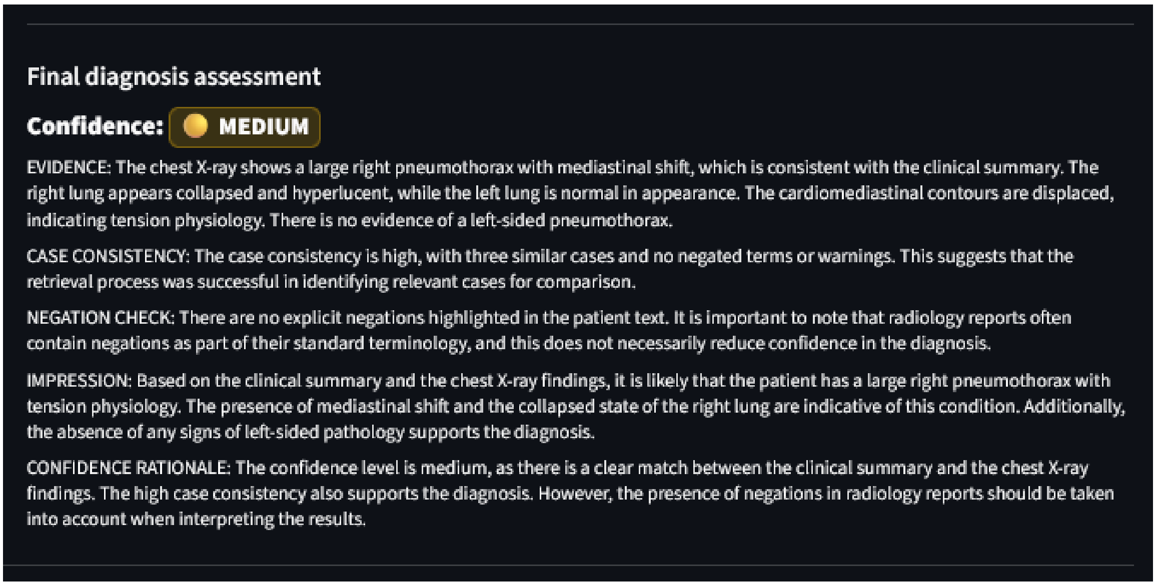


Figure 5.9: Final clinician-facing assessment as rendered in the UI, including the narrative assessment and confidence label (final assessment stage).

5.2.5 Diagnostics and provenance views

Beyond the stage-aligned clinician-facing outputs and case-level evidence inspection, the UI also exposes backend diagnostics for traceability and debugging. These views surface warning signals, retrieval-scoring details, and intermediate processing outputs that are not part of the primary clinical presentation.

The diagnostics and provenance views include:

- warning messages from validation and backend checks (e.g., patient–retrieved report negation conflicts),
- a retrieval debug table with modality scores, negation adjustment, and fused ranking, and
- intermediate and final structured JSON outputs for backend-output inspection and debugging.

Figure 5.10 shows the Analysis tab when warnings are present in the assembled output. In this example, the warning panel surfaces a backend-detected image–report negation conflict, while the validated patient findings and retrieved-case sections remain visible.

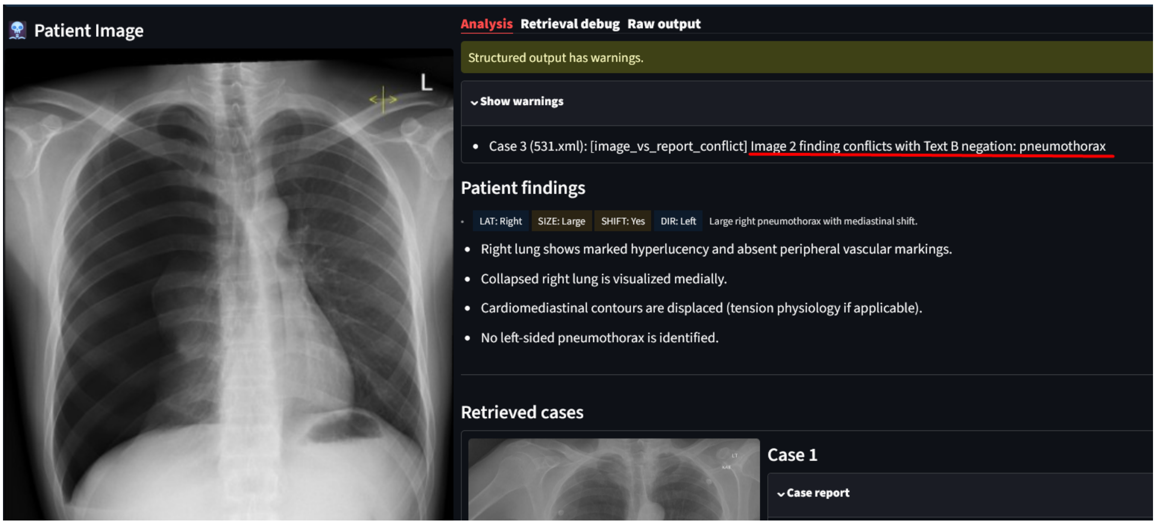


Figure 5.10: Analysis tab with warning panel and validated structured output. **Patient image source**: SVUH Radiology, "Tension Pneumothorax" case study (`http://www.svuhradiology.ie/case-study/tension-pneumothorax/`). Reproduced for academic illustration.

Figure 5.11 shows the retrieval diagnostics table with modality scores, negation-aware adjustment and fused ranking. The table is scrollable, allowing inspection of additional candidates beyond the visible rows.

Analysis **Retrieval debug** **Raw output**

Top retrieved candidates with fusion + negation-aware adjustment.

rank	score_text2text	score_text2image	score_image2image	negation_adjustment	score_fused
1	0.9483	0.46	0.9188	(0.10)	0.83
2	0.9423	0.47	0.9304	(0.18)	0.79
3	0.9387	0.00	0.8765	(0.12)	0.75
4	0.9452	0.45	0.8714	(0.23)	0.74
5	0.9332	0.00	0.8584	(0.20)	0.70
6	0.9364	0.00	0.8623	(0.24)	0.68
7	0.9416	0.00	0.857	(0.28)	0.66
8	0.9486	0.00	0.8828	(0.35)	0.62
9	0.9427	0.00	0.8578	(0.35)	0.61
10	0.939	0.00	0.8596	(0.37)	0.60

Figure 5.11: Retrieval diagnostics table (scores, adjustments, fused ranking).

The UI exposes raw intermediate and final JSON objects; Figure 5.12 shows the corresponding diagnostics view.

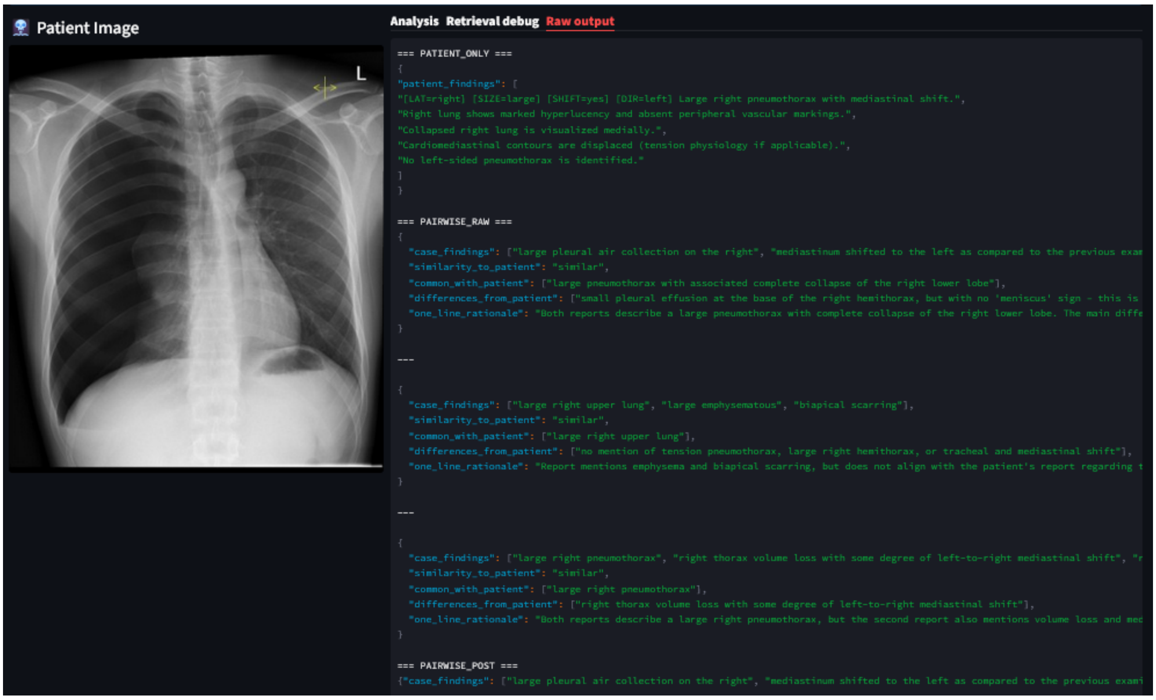


Figure 5.12: Diagnostics: raw backend outputs (intermediate and final JSON). Only an excerpt is visible in the screenshot; the UI provides a scrollable view to access the complete JSON objects.

5.3 Summary

The prototype implementation and Streamlit-based interface enables operation of the proposed multimodal DSS pipeline. The interface supports the full execution workflow, from patient input submission and runtime parameter configuration to result rendering and evidence inspection. In alignment with the validation layer described in Chapter 4, the UI presents validated structured outputs from the backend pipeline, together with supporting retrieval evidence and diagnostic views (retrieved case reports and images, fused retrieval scores, negation-related warnings, and raw/intermediate JSON objects). This design supports traceability by allowing inspection of both the final clinician-facing assessment and the intermediate artifacts used to construct it. The next chapter builds on this implementation by defining the experimental setup used to evaluate the system components and overall pipeline behavior.

CHAPTER 6

Experimental Setup

6.1 End-to-end evaluation setup

In this chapter, we evaluate the multimodal CXR-RAG system under the same operating conditions as in the Streamlit user interface. Each run starts from a new patient input consisting of (i) a free-text patient report/description (q) and (ii) a chest X-ray image (I_p). The system then executes the full retrieval and reasoning pipeline and returns a clinician-facing structured output together with the reference report texts used for evaluation.

For a single patient run, the system returns:

- **Clinician-facing structured output**: a structured patient findings summary, the retrieved cases with per-case findings and similarity labels, per-case comparison summaries (commonalities, differences, and a one-line rationale), and a final consolidated assessment with a discrete confidence level.

- **Reference report texts**: the patient input text (used as the patient-side reference) and the report texts corresponding to the retrieved database cases (used as case-side references).

The structured fields above follow the staged prompting specification in Section 3.5.2.

6.2 Data and evaluation signals

This section summarizes corpus properties of the Open-i subset ([3]) that motivate negation- and synonym-aware processing. All statistics are computed on the cleaned Findings+Impression text used throughout the pipeline (over the 7,430 image–report associations), using the same term resource and polarity rules as in Section 3.1.4 and Appendix F.

6.2.1 Negation patterns and polarity distributions

Radiology reports frequently describe not only what is present, but also what is *absent*. As shown previously in Section 3.1.4, explicit negation cues occur in the majority of reports, motivating explicit handling of polarity at the concept level. Here we further characterize (i) which cue phrases dominate the dataset and (ii) which canonical concepts most often end up resolved as negated/absent under the implemented polarity-resolution rules (Appendix F).

Most frequent negation cues

Figure 6.1 ranks negation cue phrases by the number of times they occur in the corpus (total cue occurrences across all report-text instances). The cue "*no*" dominates by a large margin, followed by multi-word patterns such as "*there is no*" and "*without*". These frequencies are high relative to the dataset size: across all of the image–report associations used by the retrieval pipeline, the most common cue appears thousands of times, indicating that explicit negation is a routine component of chest X-ray reporting rather than an edge case.

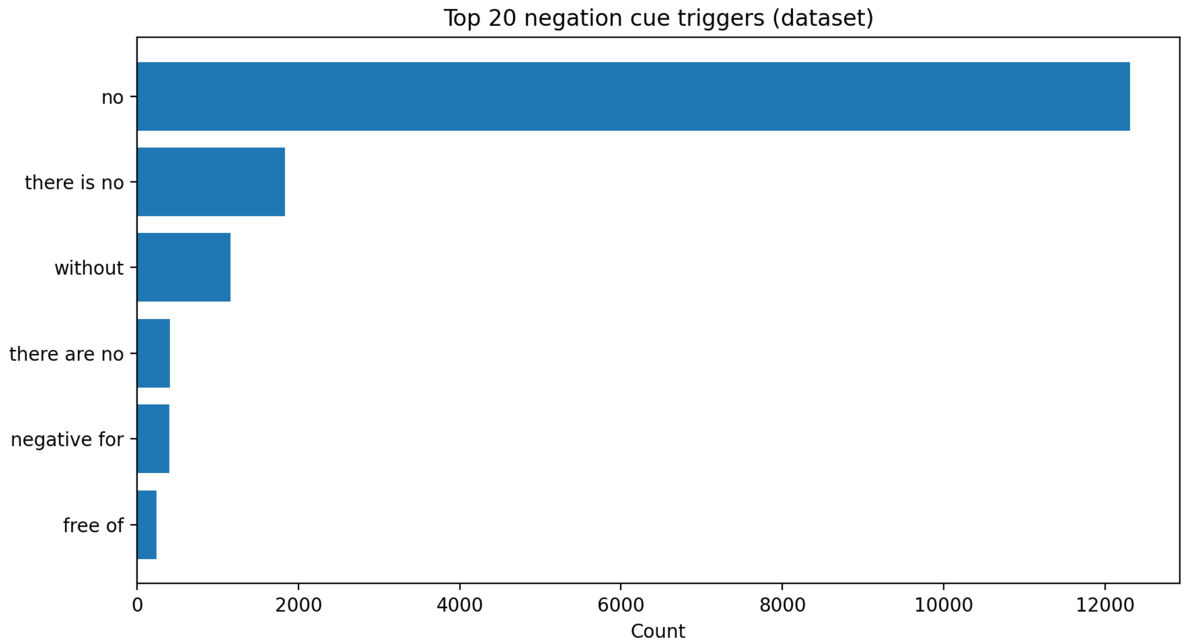


Figure 6.1: Top negation cue phrases in the Open-i Indiana subset, ranked by frequency (total cue occurrences across all report texts). The distribution is highly skewed, with "no" accounting for the majority of cue occurrences.

Why it matters. A compact, explicit cue inventory captures a large fraction of negation usage in this dataset, making rule-based cue detection a strong and interpretable baseline for downstream polarity-aware processing (Appendix F).

Most frequent negated canonical terms

Figure 6.2 ranks canonical terms by how often they are resolved as *negated/absent* under the implemented negation-scope rules. Frequent items include common CXR findings (e.g., *pneumothorax*, *pleural effusion*, *consolidation*) as well as broader descriptors (e.g., *acute*, *large*, *lung*, *focal*).

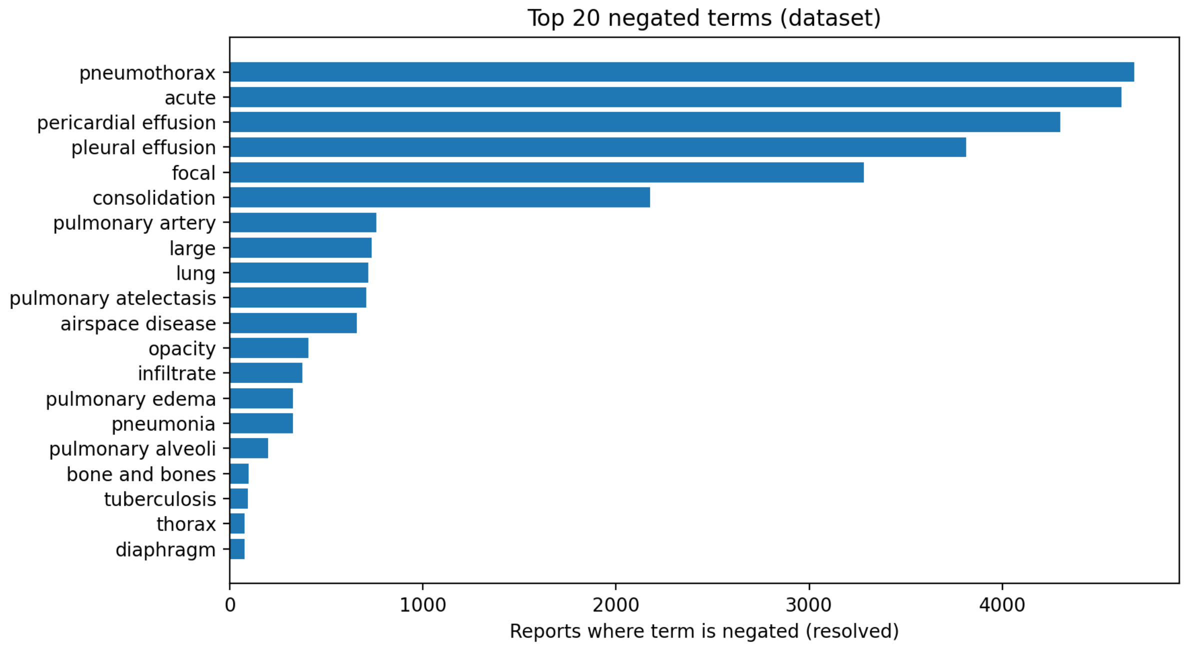


Figure 6.2: Top canonical terms most frequently resolved as negated/absent in the Open-i subset after applying the negation-scope rules (Appendix F).

Beyond cue frequency, we also summarize how often specific findings are ultimately labeled as absent vs. present under these rules.

Corpus grounding of polarity signals. We compute an offline term-level summary using the same canonical vocabulary, synonym mapping, tokenization, and negation-scope rules as the runtime pipeline.

Table 6.1 provides an excerpt of the term-level polarity statistics, reporting how often canonical terms occur in negated vs. affirmed contexts and how often each term's *final* report-level polarity is negated, affirmed, or conflicting. For readability we present only the top terms by negated usage to illustrate typical corpus patterns.

Table 6.1: Top terms by absent/negated usage (excerpt).

term	negated mentions	affirmed mentions	negated reports	affirmed reports	conflict reports	total mentions
acute	5770	42	4617	24	13	5812
pneumothorax	4743	163	4682	137	5	4906
pericardial effusion	4361	430	4300	286	24	4791
pleural effusion	3841	328	3813	240	6	4169
focal	3397	276	3283	204	38	3673
consolidation	2224	84	2176	71	4	2308
lung	1242	3738	717	2429	437	4980

Column definitions: **negated/affirmed mentions** count matches inside vs. outside a negation scope across all report texts (e.g., “no X” vs. “X is present”). **negated/affirmed reports** count how many reports end up with final term polarity -1 or $+1$ after within-report aggregation. **conflict reports** count reports where the same term is both negated and affirmed at least once (final polarity 0). **total mentions** is the sum of negated and affirmed mentions. For the full cue inventory, scope terminators, and the within-report aggregation used to produce these counts and final polarities, see Appendix F.

These counts highlight typical reporting patterns in the Open-i subset: *pneumothorax* is overwhelmingly described in negated contexts (4,743 mentions) compared to affirmed contexts (163). In contrast, broader concepts such as *lung* exhibit higher within-report conflict rates (437 reports), consistent with reports that mix exclusionary and descriptive statements (e.g., “no focal lung consolidation” alongside “lungs are clear”).

Implication for retrieval. In this corpus, many high-frequency concepts appear predominantly under negation and it is typical for some to exhibit within-report polarity conflicts. A polarity-aware similarity signal therefore provides a more clinically consistent notion of overlap than surface matching, discouraging retrieval of cases that share terms but disagree on presence versus absence.

6.2.2 Lexical variability and synonym coverage

Beyond negation, radiology language exhibits substantial surface-form variability (abbreviations, near-synonyms, and alternative phrasing). While Section 3.1.4 quantified the corpus-level *synonym gain* when enabling synonym matching, here we focus on the concepts for which synonym variability is largest.

Concepts with the largest synonym sets

Figure 6.3 ranks canonical terms by the number of curated synonym variants. These concepts represent high-variability targets where the same underlying finding may be expressed through many alternative surface forms.

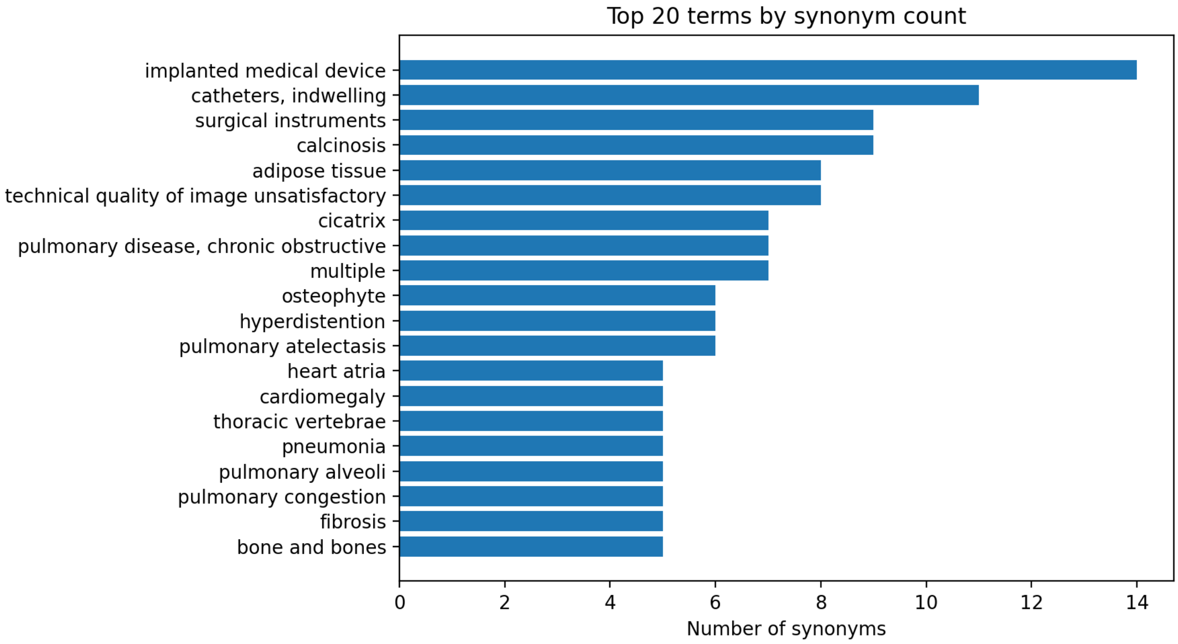


Figure 6.3: Top canonical terms by synonym set size in the curated vocabulary resource. Larger synonym sets indicate concepts with high lexical variability in reporting.

Why it matters. Large synonym sets mark concepts with high lexical variability; synonym normalization reduces sensitivity to surface-form variation and helps align mentions of the same concept across reports.

Implication for retrieval. These corpus characteristics motivate the polarity-aware and synonym-normalized reranking used in the retrieval pipeline (Section 3.4.4).

6.3 Evaluation blocks (A–E): metrics and scoring

Evaluation is performed on the structured clinician-facing output using the patient input text and the retrieved-case report texts as references. The evaluation interface (how predictions and references are formed), the reported scalar identifiers, and aggregation conventions are documented in Appendix H, Section H.1. Definitions of the text-similarity metrics used in Blocks A, B, and E are given in Section H.2, while the weak-label similarity and term-set metrics used in Blocks C and D are described in Section H.3.

A) Patient findings faithfulness. We evaluate whether the structured patient findings summary reflects the patient input text.

- **Prediction**: concatenation of the sentences in the patient findings summary.
- **Reference**: the patient input text provided for the run.

We report BLEU, ROUGE-L, METEOR, chrF, and BERTScore.

B) Retrieved-case findings faithfulness. For each retrieved case, we evaluate whether the generated retrieved-case findings summary is consistent with the corresponding ground-truth report text for that retrieved database case.

- **Prediction**: concatenation of the retrieved-case findings sentences.
- **Reference**: the ground-truth report text of the same retrieved case.

We compute BLEU, ROUGE-L, METEOR, chrF, and BERTScore per retrieved case and report the mean across the k cases.

C) Similarity label quality (weak supervision). No ground-truth patient–case similarity labels are available for evaluating `similarity_to_patient`. We therefore construct a weak reference label using semantic similarity between the patient input text and each retrieved-case report text. Similarity is computed as cosine similarity of L2-normalized sentence embeddings from
`sentence-transformers/all-MiniLM-L6-v2` [22].[1] Weak labels are assigned with fixed thresholds:

$$\texttt{similar} \text{ if } s \geq 0.70, \quad \texttt{partially similar} \text{ if } 0.45 \leq s < 0.70, \quad \texttt{not similar} \text{ if } s < 0.45.$$

Agreement is summarized using accuracy and macro-F1.

[1] Model card: `https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2`.

D) Comparison summary correctness (term-set agreement). We test whether the comparison fields `common_with_patient` and `differences_from_patient` are supported by the patient input text and each retrieved-case report text. Using synonym-normalized, negation-aware concept extraction, we form *present* canonical term sets for the patient (P) and each retrieved case (C_i), and define silver reference sets:

$$\text{gold\_common}_i = P \cap C_i, \qquad \text{gold\_diff}_i = C_i \setminus P.$$

Predicted term sets are extracted from the corresponding model output fields using the same term processor. We compute set precision/recall/F1 per case and report mean F1 across cases.

E) Final assessment coherence with patient report. We evaluate whether the final assessment narrative is consistent with the patient input text.

- **Prediction**: the final assessment text.
- **Reference**: the patient input text.

We report BLEU, ROUGE-L, METEOR, chrF, and BERTScore.

What the text metrics capture (interpretation). We report multiple complementary text agreement metrics because each emphasizes a different notion of agreement:

(i) **BLEU**: focuses on local n-gram precision and is sensitive to exact phrasing.

(ii) **ROUGE-L**: captures sequence overlap via longest-common-subsequence structure and is more recall-oriented.

(iii) **METEOR**: allows more flexible matching and typically correlates better with sentence-level paraphrases than pure n-gram precision.

(iv) **chrF**: measures character n-gram overlap and is more robust to small lexical/inflectional variations.

(v) **BERTScore**: measures semantic similarity using contextual embeddings and is less sensitive to surface wording.

6.4 Experiments

6.4.1 Retrieval fusion weight sweep (settings)

To study how retrieval fusion affects the end-to-end behavior of the system, we perform a small sweep over the late-fusion weights used to combine (i) text→text, (ii) text→image and (iii) image→image similarity signals . We denote these weights as $(w_{t2t}, w_{t2i}, w_{i2i})$, enforcing $w_{t2t} + w_{t2i} + w_{i2i} = 1$.

Figure 6.4 visualizes the tested configurations. This figure is shown first to clarify the evaluation logic: in the subsequent analysis, each row of results corresponds to one of these weight settings, while the patient inputs (the free-text query q and patient radiograph I_p), the number of retrieved cases k, and the reasoning model remain fixed.

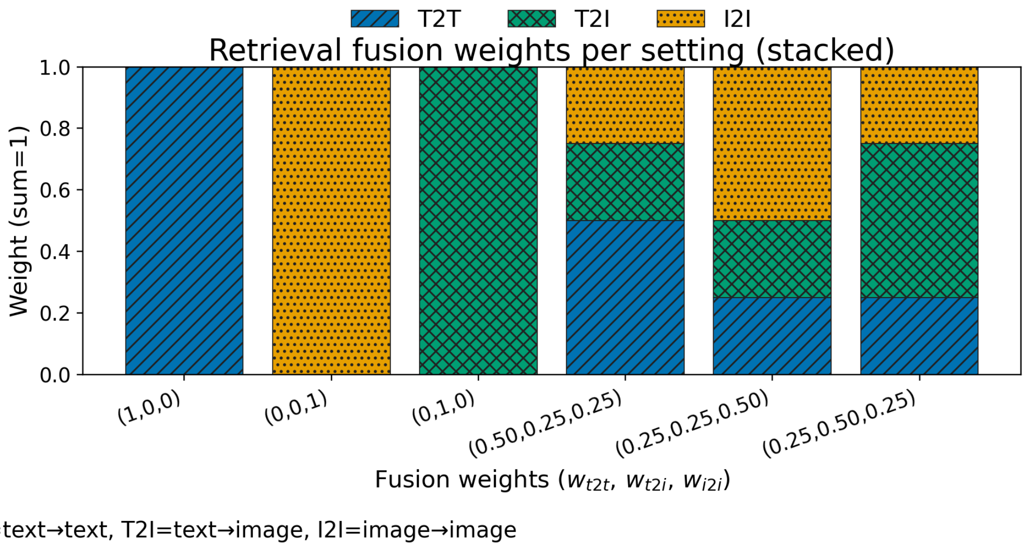


Figure 6.4: Retrieval fusion weights per sweep setting. T2T denotes text→text retrieval, I2I image→image retrieval, and T2I text→image retrieval. Each stacked bar sums to 1.

Table 6.2: Fusion settings used throughout all weight-sweep experiments. We report results using the short setting labels below.

Setting label	w_{t2t}	w_{t2i}	w_{i2i}	Description
T2T-only	1.00	0.00	0.00	text→text retrieval only
I2I-only	0.00	0.00	1.00	image→image retrieval only
T2I-only	0.00	1.00	0.00	text→image retrieval only
Mixed (T2T-heavy)	0.50	0.25	0.25	mixed fusion, more weight on text→text
Mixed (I2I-heavy)	0.25	0.25	0.50	mixed fusion, more weight on image→image
Mixed (T2I-heavy)	0.25	0.50	0.25	mixed fusion, more weight on text→image

6.4.2 Retrieval fusion weight sweep (single-case example)

Experimental setup. To assess how retrieval fusion weights affect end-to-end behavior while keeping all other components fixed, we run a weight sweep over the six fusion settings in Table 6.2 (Figure 6.4). The sweep is performed on a single pneumothorax query with $k = 3$ retrieved cases.

Metrics. We report the Block A–E scalar metrics defined in Appendix H. For this single-case experiment, values are reported directly per fusion setting without aggregation. Block A is reported once because it is computed from the patient-only VLM call and is independent of the retrieval fusion weights.

Single-case sweep results. Tables 6.3–6.7 report the Block A–E metrics and runtime flags for each fusion setting on this pneumothorax query.

Table 6.3: Patient-side metrics (Block A) for the pneumothorax example ($k = 3$). These values are identical for all weight settings.

`patient_bleu`	`patient_rougeL`	`patient_meteor`	`patient_chrf`	`patient_bertscore`
4.9441	0.2093	0.2588	45.1448	0.8275

Note: Block A is generated by the patient-only VLM call using the same patient image and the same clinical summary; retrieval fusion weights do not affect this step, hence the scores are identical for all weight combinations $(w_{t2t}, w_{t2i}, w_{i2i})$.

Table 6.4: Retrieved-case faithfulness metrics (Block B) for the pneumothorax sweep ($k = 3$). Each row corresponds to a fusion weight configuration $(w_{t2t}, w_{t2i}, w_{i2i})$, reported using the setting labels defined in Table 6.2.

$(w_{t2t}, w_{t2i}, w_{i2i})$	mean_case_bleu	mean_case_rougeL	mean_case_meteor	mean_case_chrf	mean_case_bertscore
T2T-only	1.7146	0.1771	0.1583	24.3439	0.8161
I2I-only	5.7083	0.3505	0.2407	29.0357	0.8493
T2I-only	6.0300	0.3642	0.2652	31.1424	0.8493
Mixed (T2T-heavy)	**8.3760**	**0.4612**	**0.3294**	**34.1214**	**0.8757**
Mixed (I2I-heavy)	**8.3760**	**0.4612**	**0.3294**	**34.1214**	**0.8757**
Mixed (T2I-heavy)	**8.3760**	**0.4612**	**0.3294**	**34.1214**	**0.8757**

Observation. In this single-case sweep, the mixed fusion settings yield identical Block B scores, suggesting that different weightings can converge to the same top-k retrieved set for this query. Among all settings, the **mixed configurations** achieve the highest retrieved-case faithfulness across all Block B metrics, indicating that retrieval fusion is beneficial for this example.

Table 6.5: Weak-label agreement (Block C) and term-set agreement (Block D) for the pneumothorax sweep ($k = 3$).

$(w_{t2t}, w_{t2i}, w_{i2i})$	similarity _accuracy	similarity _macro_f1	mean_common_f1	mean_diff_f1
T2T-only	0.3333	0.2500	0.4452	**0.3286**
I2I-only	0.3333	0.2500	0.4444	0.0000
T2I-only	**0.6666**	**0.4000**	**0.6944**	0.1333
Mixed (T2T-heavy)	0.3333	0.2500	0.5396	0.1333
Mixed (I2I-heavy)	0.3333	0.2500	0.5396	0.1333
Mixed (T2I-heavy)	0.3333	0.2500	0.5396	0.1333

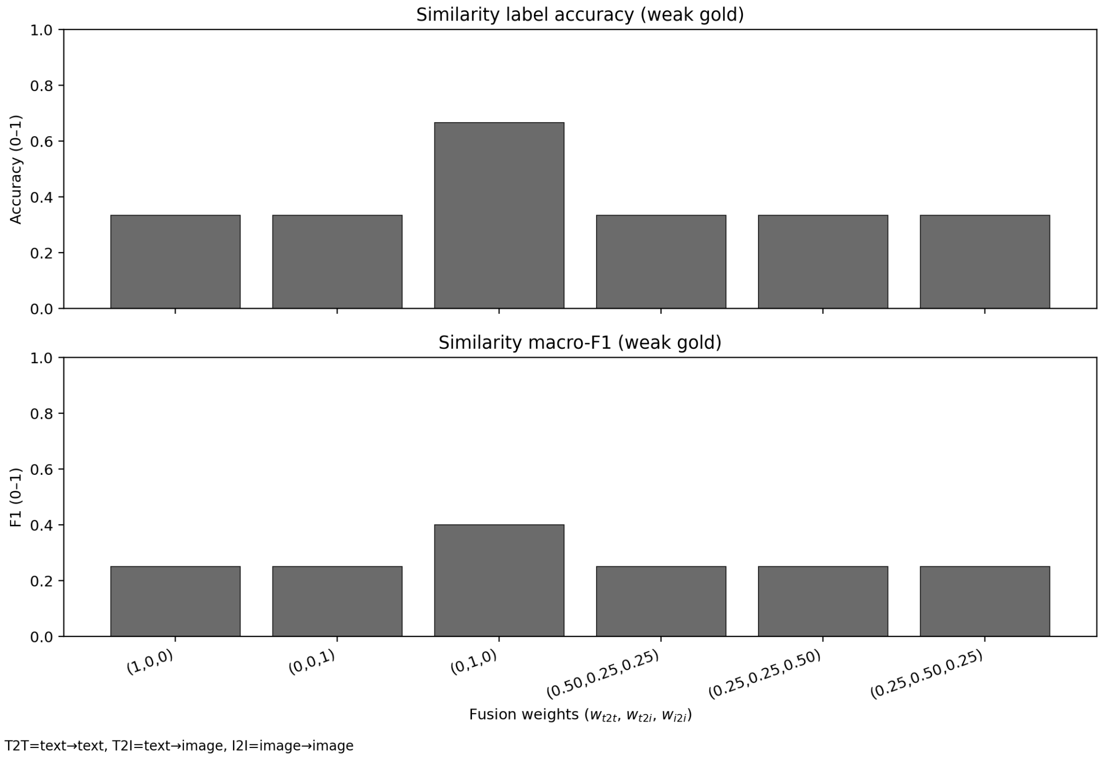


Figure 6.5: Block C: weak-label agreement across fusion weights for the pneumothorax sweep ($k = 3$). Top: `similarity_accuracy`. Bottom: `similarity_macro_f1`.

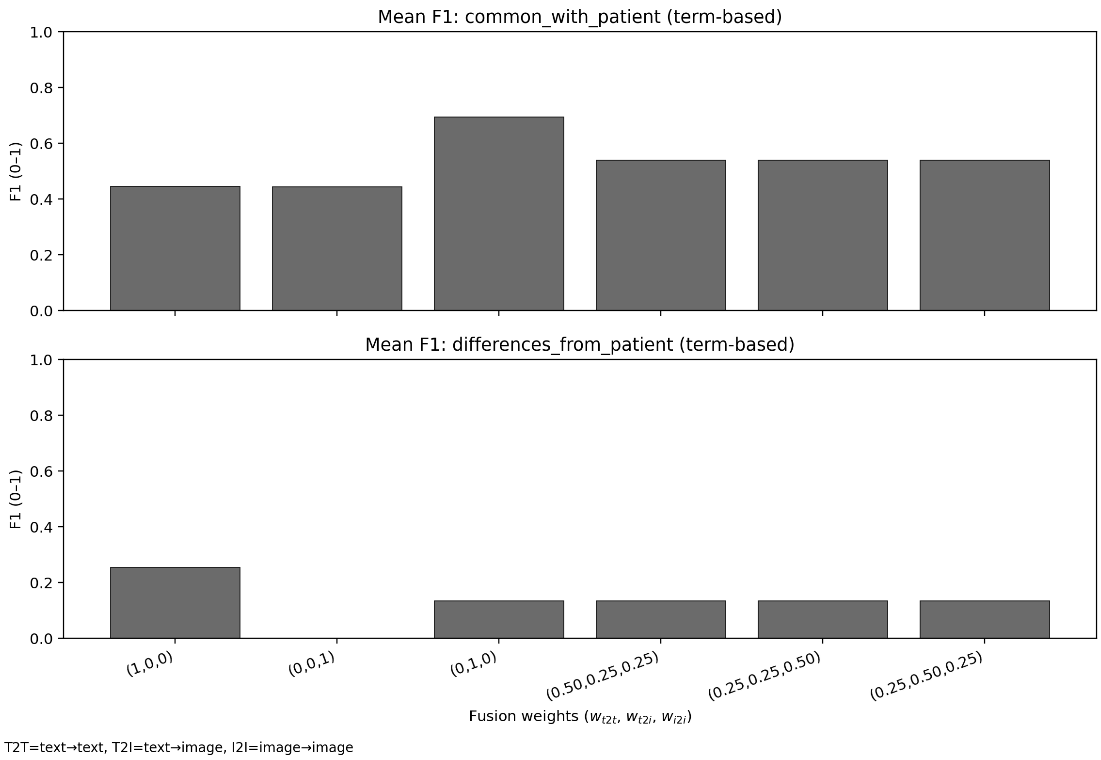


Figure 6.6: Block D: term-set agreement across fusion weights for the pneumothorax sweep ($k = 3$). Top: `mean_common_f1`. Bottom: `mean_diff_f1`.

Observation. For this single case, **T2I-only** achieves the highest weak-label agreement and the strongest overlap in shared terms (`mean_common_f1`). In contrast, **T2T-only** yields the highest `mean_diff_f1`, indicating better separation on the differing term set for this query. As in Block B, the mixed fusion settings produce identical values, suggesting that multiple weightings can lead to the same effective retrieval outcome in this example.

Table 6.6: Final assessment metrics (Block E) for the pneumothorax sweep ($k = 3$).

$(w_{t2t}, w_{t2i}, w_{i2i})$	`final_bleu`	`final_rougeL`	`final_meteor`	`final_chrf`	`final_bertscore`
T2T-only	3.8444	0.1433	0.3438	34.9205	0.8065
I2I-only	4.1883	0.1465	0.3850	36.5494	0.8063
T2I-only	3.7109	0.1390	0.3186	33.5316	0.8022
Mixed (T2T-heavy)	3.8444	0.1433	0.3438	34.9205	0.8065
Mixed (I2I-heavy)	3.8444	0.1433	0.3438	34.9205	0.8065
Mixed (T2I-heavy)	3.8444	0.1433	0.3438	34.9205	0.8065

Runtime validation flags. In addition to the Block A–E evaluation metrics, we report runtime validation indicators derived from the validation outputs described in Chapter 4. Specifically, `n_warnings` and `n_errors` denote the number of entries in the validator's `warnings` and `errors` lists, respectively, while `validation_ok` and `logic_ok` are the corresponding boolean pass flags returned by the validation layer.

Table 6.7: Runtime flags and validation status for the pneumothorax sweep ($k = 3$).

$(w_{t2t}, w_{t2i}, w_{i2i})$	`n_warnings`	`n_errors`	`validation_ok`	`logic_ok`
T2T-only	1.0	0.0	True	True
I2I-only	2.0	0.0	True	True
T2I-only	0.0	0.0	True	True
Mixed (T2T-heavy)	1.0	0.0	True	True
Mixed (I2I-heavy)	1.0	0.0	True	True
Mixed (T2I-heavy)	1.0	0.0	True	True

Takeaways from the single-case sweep. Across fusion settings, execution is stable and the final outputs are broadly similar; the main differences appear in the *retrieved* *evidence*. In this pneumothorax example, prioritizing text→image retrieval produces the most query-consistent supporting cases and the strongest overlap with clinically relevant patient terms, yielding the most coherent evidence for downstream reasoning. Pure text→text retrieval places relatively more emphasis on contrastive cues (i.e., what differs from the patient), which can sharpen separation but does not necessarily improve overall evidence support. Image→image retrieval is less consistent here: it can return plausible visual neighbors, but is more prone to evidence–narrative mismatches in this run. Finally, the mixed fusion variants behave very similarly in this example, indicating that different weight combinations can lead to nearly the same retrieved set.

Single-case model swap sanity check (MiniCPM-V)

To assess whether the single-case weight-sweep observations are specific to the reasoning VLM, we repeated the same pneumothorax sweep (k=3, same inputs and retrieval index) using `minicpm-v` in place of `llava`. We report only the retrieval-side agreement plots in Figures 6.7–6.8, which summarize how the retrieved evidence and the generated comparison fields behave across fusion settings under this alternative model.

Practical limitation. In this run, the final structured assessment was not consistently produced for all fusion settings. Inspection of the raw outputs indicates that the failure mode occurs at the final model call, likely due to input-length and/or formatting sensitivity when the model is provided the full multi-case context. Because this affects the downstream stage rather than retrieval itself, we treat this experiment as a *sanity check on retrieval behavior* under a smaller VLM, and we do not include downstream output comparisons for MiniCPM-V.

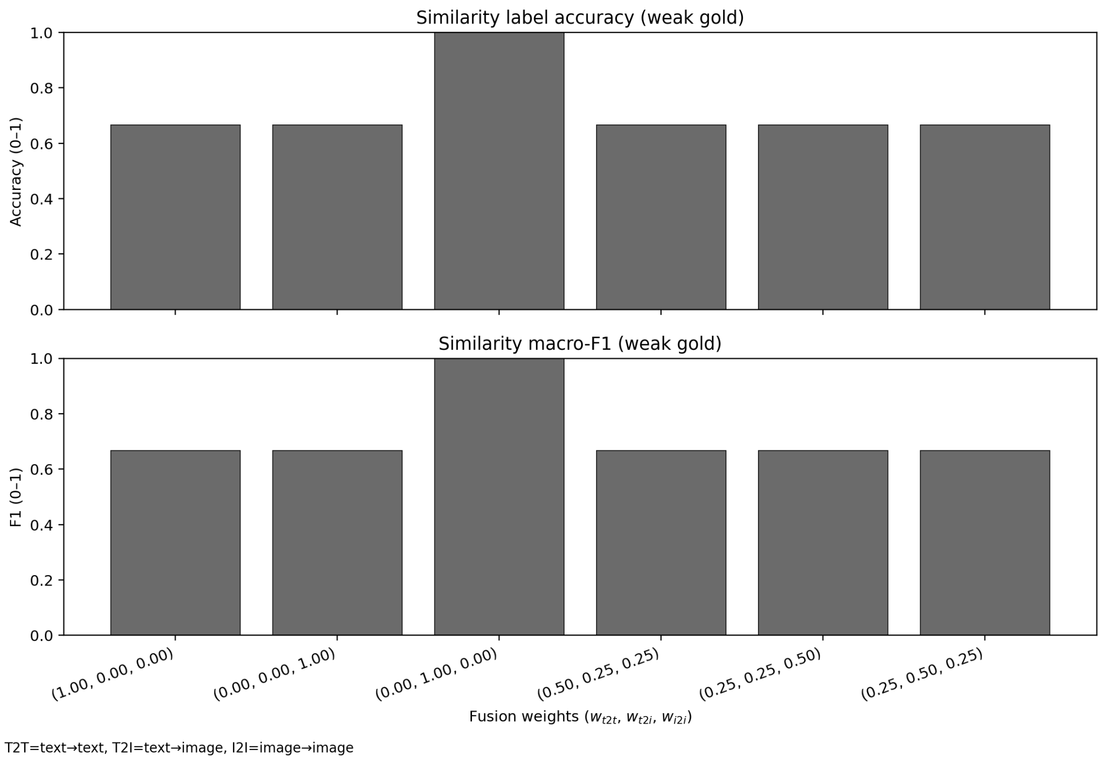


Figure 6.7: MiniCPM-V single-case sweep: agreement of the model's similarity labeling with the weak reference across fusion settings (k=3).

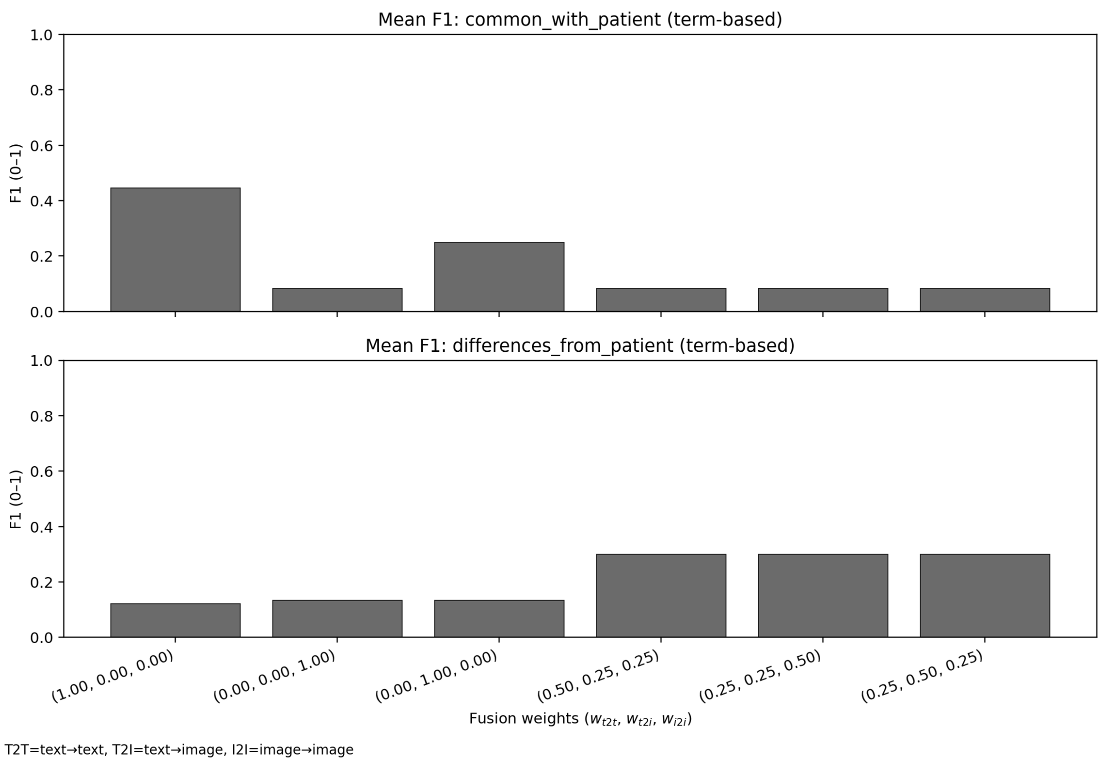


Figure 6.8: MiniCPM-V single-case sweep: agreement of the comparison summaries with term-set references across fusion settings (k=3).

Takeaways (MiniCPM-V vs. LLaVA, single-case). MiniCPM-V preserves the same overall sensitivity to fusion weights as LLaVA: changing the weights mainly changes *which evidence is retrieved* and how the model expresses similarity versus contrast. For similarity labeling, the trend is stable across models: text→image again performs best, and under MiniCPM-V it reaches a perfect score for this run, while the remaining settings follow a similar pattern at *uniformly higher agreement than LLaVA*. For the comparison summaries, however, the best setting shifts: under MiniCPM-V, pure text→text yields the strongest *shared-term overlap* with the patient, whereas the strongest *difference-focused* signal is achieved by the mixed fused settings (rather than a single modality). Finally, we note a practical limitation of this MiniCPM-V run: the final assessment stage was not reliably produced for all weight settings (likely due to input-length/format constraints at the final VLM call), so we only interpret the upstream evidence-level behaviors shown in Figures 6.5–6.6 for this comparison.

These single-case findings motivate repeating the sweep across multiple patients, where aggregate patterns are more reliable than any individual run.

6.4.3 Retrieval fusion weight sweep (10-patient evaluation)

Experimental setup. To test whether the trends observed in the single-case sweep generalize, we repeat the same retrieval-fusion weight sweep on $n = 10$ archived patient study pairs. We keep the retrieval depth fixed at $k = 3$ and use the same reasoning model and prompting configuration as in the single-case experiment. Fusion settings follow the six weight triplets shown in Figure 6.4.

Retrieval protocol (leave-one-out). For each patient query, the corresponding patient report ID is removed from the retrieval index. This prevents trivial self-retrieval from the archived pool and ensures that the top-k retrieved cases come from other patients.

Metrics and aggregation. We report the same Block A–E scalar metrics defined in Appendix H. For each fusion setting, metrics are summarized by the arithmetic mean over the $n = 10$ patient runs (excluding missing values). Block A is reported once because it is computed from the patient-only generation step and is independent of retrieval fusion weights. Accordingly, the Block A metric values are identical across all fusion settings for each patient.

Results overview. Tables 6.8–6.12 report the $n = 10$ sweep results per fusion setting. Figures 6.9–6.12 visualize the corresponding aggregated metrics.

Table 6.8: Patient-side metrics (Block A) for the 10-patient weight-sweep experiment ($k = 3$).

patient_id	patient_bleu	patient_rougeL	patient_meteor	patient_chrf	patient_bertscore
patient_001	2.1008	0.1356	0.1025	20.2506	0.7826
patient_002	2.1008	0.1356	0.1025	20.2506	0.7826
patient_003	16.5624	0.3125	0.2466	27.6768	0.8023
patient_004	16.5624	0.3125	0.2466	27.6768	0.8023
patient_005	54.6024	0.7429	0.9295	87.2572	0.9255
patient_006	67.3319	0.7879	0.9431	90.7250	0.9536
patient_007	11.2951	0.3465	0.2674	30.6560	0.8209
patient_008	8.8594	0.3175	0.2248	26.2849	0.8102
patient_009	11.2951	0.3465	0.2674	30.6560	0.8209
patient_010	26.1440	0.4500	0.4853	47.4071	0.8433
mean	**21.6855**	**0.3887**	**0.3816**	**40.8841**	**0.8344**

Note: Block A is computed from the patient-only VLM call using the same patient input (image and `query_text`). Retrieval fusion weights do not affect this step, so Block A is invariant to $(w_{t2t}, w_{t2i}, w_{i2i})$ and is reported once for the sweep.

Observation. Patient-side scores vary substantially across the 10 patients, indicating that patient difficulty and content dominate Block A. In contrast to n-gram overlap measures (BLEU/ROUGE-L/METEOR/chrF), BERTScore remains relatively stable and consistently high across patients, suggesting that the generated patient findings tend to preserve the *overall semantic content* even when wording and phrasing differ.

Table 6.9: Retrieved-case faithfulness metrics (Block B), averaged across $n = 10$ patients ($k = 3$).

$(w_{t2t}, w_{t2i}, w_{i2i})$	mean_case_bleu	mean_case_rougeL	mean_case_meteor	mean_case_chrf	mean_case_bertscore
T2T-only	18.0779	0.4778	0.4039	42.4514	0.8609
I2I-only	12.9634	0.3660	0.3408	37.9553	0.8484
T2I-only	5.7907	0.3427	0.2550	28.7050	0.8487
Mixed (T2T-heavy)	9.1098	0.3389	0.2731	33.7301	0.8433
Mixed (I2I-heavy)	9.5516	0.3696	0.2950	34.8436	0.8523
Mixed (T2I-heavy)	7.2377	0.3278	0.2642	33.0541	0.8440

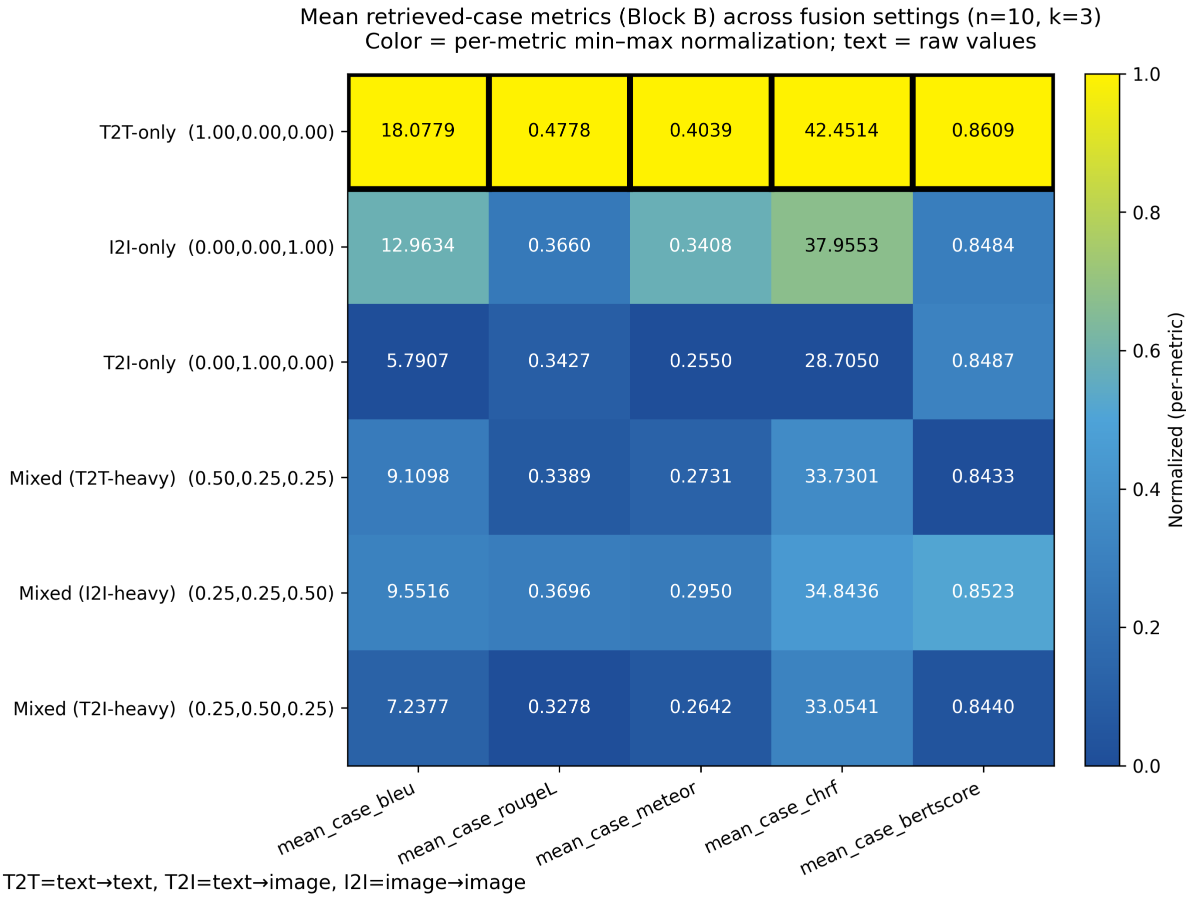


Figure 6.9: Retrieved-case faithfulness metrics (Block B) across fusion settings ($n = 10$, $k = 3$). Cell color shows *per-metric min–max normalization* across settings (higher is better), and cell text shows the raw mean value. Thick borders mark the best setting per metric.

Observation. Across all retrieved-case faithfulness metrics, *T2T-only* yields the highest mean scores in this $n = 10$, $k = 3$ sweep, suggesting that text-to-text retrieval retrieves more faithful supporting cases than image-driven variants for this dataset.

Table 6.10: Weak-label agreement (Block C) and term-set agreement (Block D), averaged across $n = 10$ patients ($k = 3$).

$(w_{t2t}, w_{t2i}, w_{i2i})$	`similarity_accuracy`	`similarity_macro_f1`	`mean_common_f1`	`mean_diff_f1`
T2T-only	0.8333	0.7450	0.5574	0.6759
I2I-only	0.2333	0.1550	0.3778	0.2492
T2I-only	0.1333	0.0800	0.3852	0.2610
Mixed (T2T-heavy)	0.3667	0.3500	0.5115	0.3000
Mixed (I2I-heavy)	0.3667	0.3500	0.4782	0.3000
Mixed (T2I-heavy)	0.2667	0.2150	0.4856	0.2833

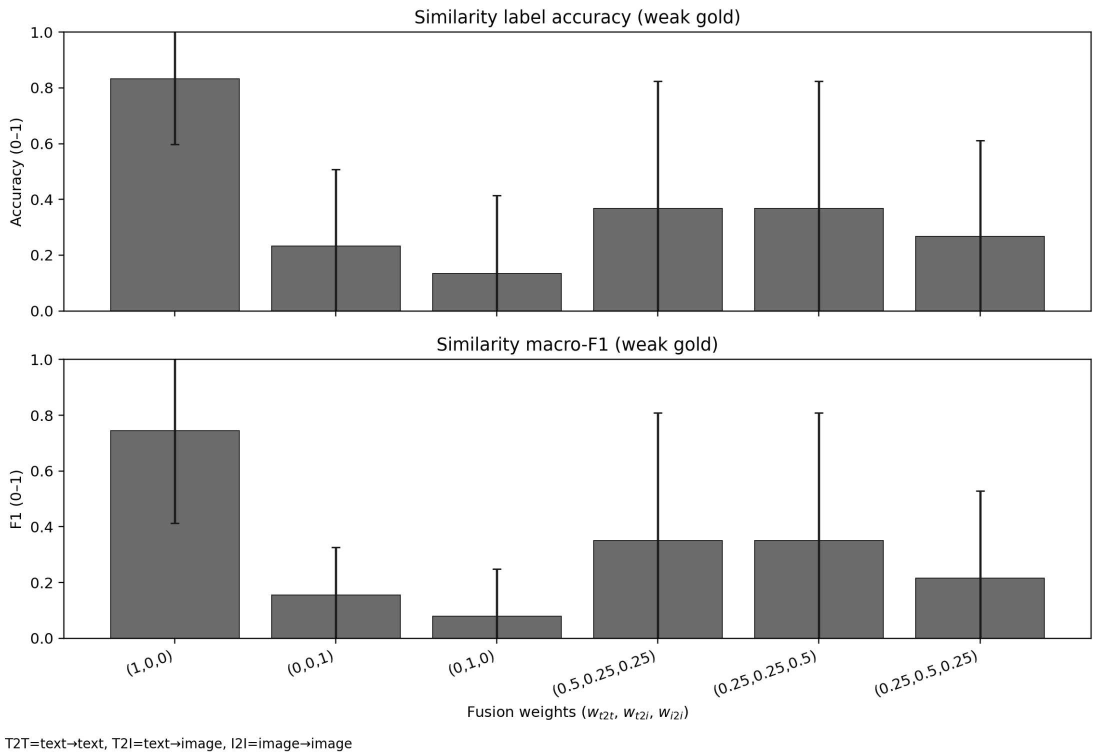


Figure 6.10: Weak-label agreement (Block C) across fusion settings ($n = 10$, $k = 3$). Higher is better (0–1). X-axis weights are ordered as $(w_{t2t}, w_{t2i}, w_{i2i})$.

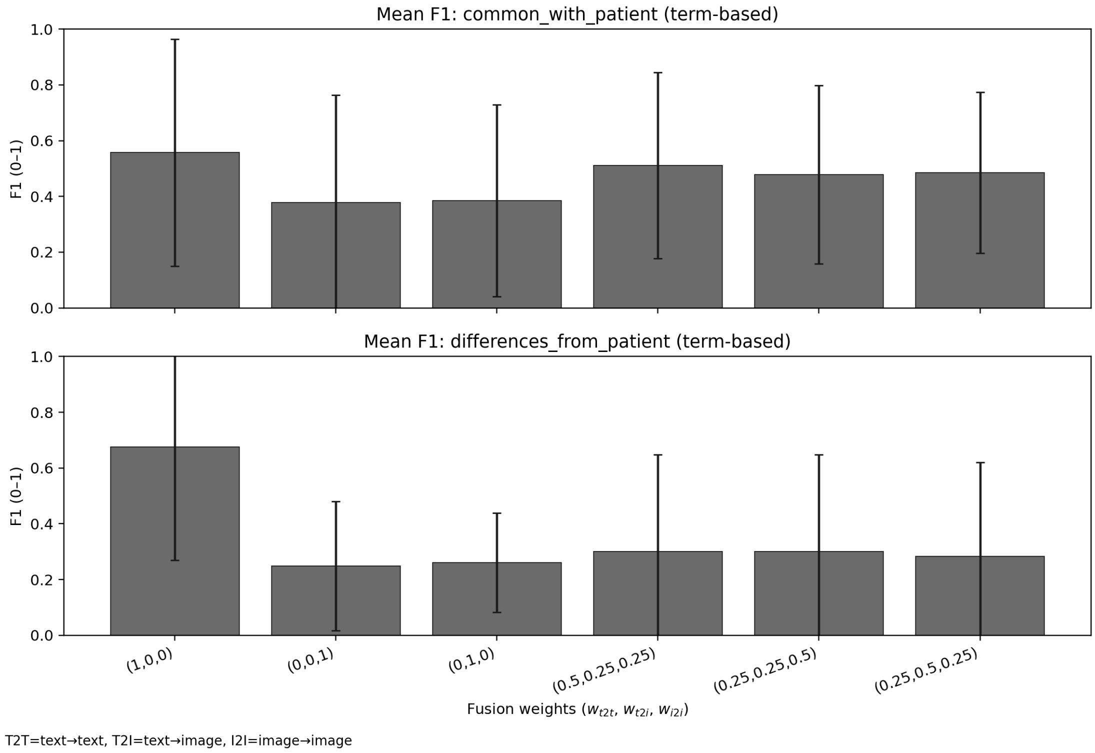


Figure 6.11: Term-set agreement (Block D) across fusion settings ($n = 10$, $k = 3$). Higher is better (0–1). X-axis weights are ordered as $(w_{t2t}, w_{t2i}, w_{i2i})$.

Observation. *T2T-only* achieves the highest agreement across all four metrics (similarity accuracy, similarity macro-F1, mean common-F1, and mean diff-F1), indicating the most consistent alignment between retrieved cases and weak-label/term-set evaluations under text-to-text retrieval.

Table 6.11: Final assessment metrics (Block E), averaged across $n = 10$ patients ($k = 3$).

$(w_{t2t}, w_{t2i}, w_{i2i})$	`final_bleu`	`final_rougeL`	`final_meteor`	`final_chrf`	`final_bertscore`
T2T-only	10.2490	0.2038	0.4310	40.2066	0.7965
I2I-only	10.1445	0.1971	0.4312	40.6237	0.7942
T2I-only	9.5980	0.2091	0.4201	39.5981	0.7947
Mixed (T2T-heavy)	9.9265	0.2077	0.4364	40.2191	0.7940
Mixed (I2I-heavy)	9.9265	0.2077	0.4364	40.2191	0.7940
Mixed (T2I-heavy)	9.8573	0.2080	0.4370	40.3506	0.7957

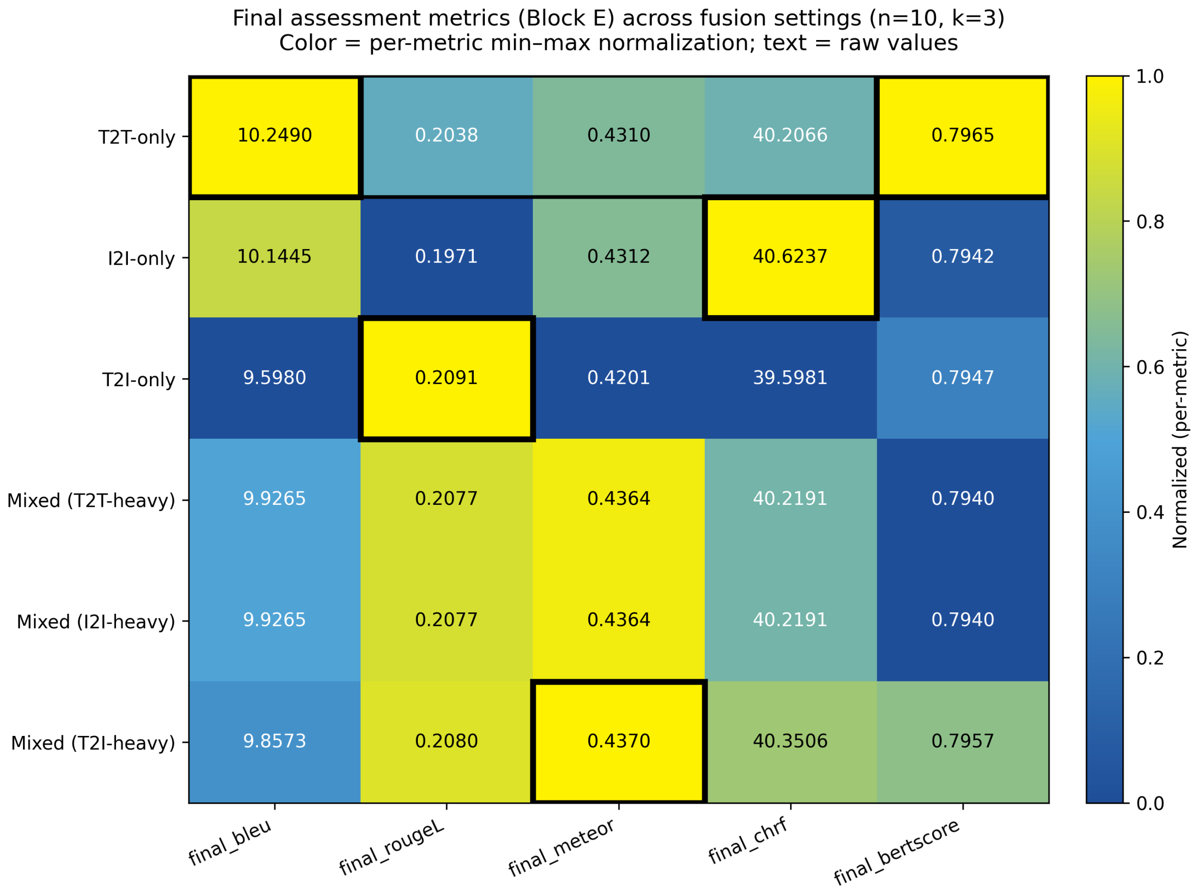


Figure 6.12: Final-assessment metrics (Block E) across fusion settings (n = 10, k = 3). Cell text shows the raw mean value. Cell color uses *per-metric min–max normalization* across settings (higher is better). Thick borders mark the best setting per metric.

Observation. Final assessment performance is similar across fusion settings, and the best setting depends on the metric. This suggests that while fusion weights strongly affect retrieved-case alignment (Blocks B–D), their impact on the final generation metrics is smaller in this sweep.

Table 6.12: Runtime flags and validation status, aggregated across $n = 10$ patients ($k = 3$).

$(w_{t2t}, w_{t2i}, w_{i2i})$	`n_warnings`	`n_errors`	`validation_ok`	`logic_ok`
T2T-only	3.1	0.0	1.0	1.0
I2I-only	3.5	0.0	1.0	1.0
T2I-only	2.4	0.0	1.0	1.0
Mixed (T2T-heavy)	2.8	0.0	1.0	1.0
Mixed (I2I-heavy)	2.8	0.0	1.0	1.0
Mixed (T2I-heavy)	2.7	0.0	1.0	1.0

Note: `n_warnings` and `n_errors` are reported as the mean count per patient (over $n = 10$ runs) for each fusion setting. `validation_ok` and `logic_ok` are reported as pass rates in $[0, 1]$ (fraction of patients for which the flag was `True`); thus, `1.0` indicates that all patients (10/10) satisfied the corresponding check.

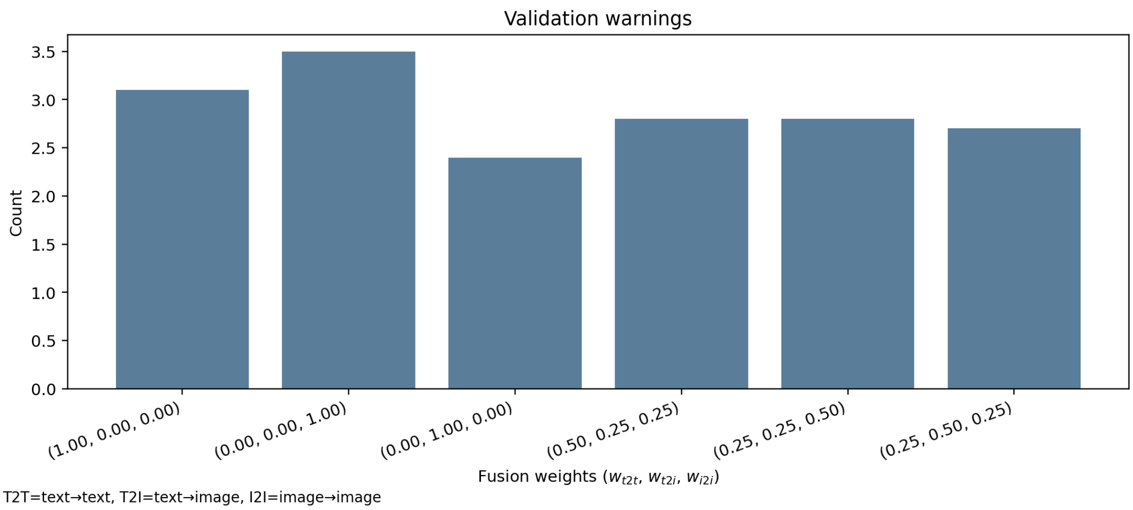


Figure 6.13: Mean validation warnings per fusion setting across $n = 10$ patients ($k = 3$). Lower is better; differences are minor.

Observation. All fusion settings completed without runtime errors (`n_errors`=0.0) and achieved perfect validation and logic pass rates (`validation_ok`=`logic_ok`=1.0), i.e., 10/10 patients passed the checks. The mean warning count varies modestly (2.4–3.5): *T2I-only* yields the fewest validation warnings on average, whereas *I2I-only* yields the most, suggesting slightly cleaner schema/metadata consistency under text-to-image retrieval.

Takeaways from the 10-patient sweep. Across all six fusion settings, the system executes reliably: there are no runtime failures and all runs satisfy the validation and logic checks, with only modest variation in warning frequency. When performance is assessed at the level of the *retrieved evidence*, text→text pathway stands out: on average it retrieves supporting cases that are the most consistent and best-aligned with the patient inputs, and it yields the strongest overall agreement signals in the auxiliary evaluations. In contrast, image-driven retrieval (image→image and text→image) is less consistently aligned in this n=10 sample, and the mixed settings fall between the extremes rather than improving upon the text→text baseline. Despite these clear differences upstream, downstream outputs remain broadly comparable across settings, indicating that the reasoning stage partially compensates for retrieval variation when k=3 is fixed. Overall, this multi-patient sweep suggests that, for this dataset and retrieval depth, emphasis on the text→text retrieval produces the most dependable evidence package, while alternative fusions mainly affect retrieval behavior without translating into large changes in final outputs.

CHAPTER 7

Conclusions and Future Work

7.1 Summary of Findings

Dataset properties motivate polarity- and synonym-aware retrieval. Corpus statistics show that radiology reports frequently encode clinically important *absence* statements (negated findings) and exhibit substantial surface-form variability for the same underlying concepts (abbreviations and synonyms). As a result, naive lexical overlap can be misleading (e.g., conflating or missing matches across equivalent phrasing), which reinforces the need for polarity-aware and synonym-normalized processing in the retrieval and comparison stages.

End-to-end stability. Across experiments, the system executes reliably across fusion settings and produces broadly similar clinician-facing outputs, indicating that the overall pipeline is stable under the tested configurations.

Where fusion weights matter. The primary effect of fusion weights is on the *retrieved evidence*: changing the weighting alters which archived cases are surfaced, which in turn changes how strongly the retrieved evidence supports the downstream reasoning and comparisons.

Model sensitivity (LLaVA vs. MiniCPM-V). A targeted single-case rerun with MiniCPM-V suggests that the choice of VLM can change the *measured agreement patterns* under the same retrieval settings. In this comparison, MiniCPM-V exhibits similar qualitative sensitivity to fusion weights, but produces different relative strengths across settings in the evidence-level evaluations. Because the final assessment stage was not reliably produced for all MiniCPM-V weight settings in this run, this model comparison is interpreted only at the upstream evidence level.

Single-case versus multi-patient behavior. The single pneumothorax example illustrates that different weightings can lead to noticeably different supporting evidence for a given set of query-time patient inputs. When the same sweep is repeated across $n = 10$ patients, the aggregate trends differ from the single-case result, highlighting that conclusions drawn from one patient may not generalize and that patient-level variability plays a major role.

7.2 Limitations

Evaluation scope and sample size. This study uses a limited number of patient inputs for aggregated experiments and includes a single-case analysis; larger-scale evaluation is required for stronger statistical confidence.

Automatic metrics and weak supervision. The evaluation relies on automatic similarity metrics and weak-label proxies, which do not guarantee clinical correctness and can disagree with human judgement.

Model-specific output constraints. The MiniCPM-V comparison exposed practical constraints at the final generation stage for some fusion settings, limiting the completeness of cross-model comparisons under identical conditions.

Dataset and reporting-style dependence. Outcomes may depend on corpus composition, reporting conventions, and the distribution of pathologies represented in the archived retrieval pool.

Archived-versus-novel input gap. The single-case sweep uses newly provided inputs, whereas the n=10 sweep evaluates on archived studies drawn from the same dataset that supplies the retrieval pool (using a leave-one-out technique to avoid self-retrieval). Therefore, the aggregated results mainly reflect *in-distribution* behavior, and performance may shift when the system is applied to new incoming cases with different imaging characteristics or reporting styles.

7.3 Future Work

7.3.1 Multi-view case representation in embedding space

An important next step is to support retrieval over multi-view studies (e.g., frontal (PA/AP) and lateral projections), treating each study as a *single retrievable case* rather than a set of independent images. Specifically, both the archived database and the query input would represent a study as a *bundle of view-specific embeddings* with an explicit study-level matching rule. It would be interesting to explore three complementary designs: (i) a *single study embedding* formed by combining the view embeddings into one vector (with fixed or learned view weighting) so the index remains one entry per study, (ii) a *multi-embedding study representation* that keeps one vector per view and retrieves studies using a view-matching score (e.g., best-view match or top-m view matches), and (iii) a two-stage retrieval strategy that first retrieves candidate studies using a fast study-level representation and then re-ranks candidates using explicit view-to-view alignment. Overall, the goal is an embedding-space design that preserves complementary information across views while keeping the retrieval unit aligned with the clinical notion of a study.

7.3.2 Uncertainty-aware concept handling

A practical extension would be to treat radiology uncertainty as an explicit state distinct from affirmed and negated findings. In routine reporting, hedging cues such as "possible", "may represent", and "suspicious for" often encode clinically meaningful ambiguity rather than true absence or presence. In the current system, term polarity is represented as $\{+1, -1, 0\}$, where 0 covers both *unmentioned* terms and *within-text polarity conflicts* (the same concept appearing as both affirmed and negated).

Future work could therefore extend the term-level representation beyond $\{+1, -1, 0\}$ to a four-level encoding $\{+1, -1, 0, +0.5\}$, where $+0.5$ denotes uncertainty, and 0 is reserved for unmentioned concepts (with conflicts tracked separately). This would require (i) a fixed uncertainty cue inventory and scope rules analogous to negation scope, including a clear interaction policy when negation and uncertainty cues co-occur, and (ii) an adjustment function that treats uncertainty as *partial agreement* rather than full agreement or full conflict. For retrieval, this would enable softer ranking behavior where, for example, "possible atelectasis" aligns more strongly with "atelectasis" than with "no atelectasis", while remaining distinguishable from a definitive positive finding.

Finally, the same uncertainty signal can be surfaced in the clinician-facing traceability view, allowing the UI to highlight which overlaps are definitive versus hedged and reducing the risk that downstream generation presents uncertain findings as categorical claims.

7.3.3 Patient subgroup analysis via clustering

A useful extension would be to analyze patient-level variability through *cluster-based subgrouping*, rather than studying fusion-weight behavior only at the global level. In particular, clustering methods (e.g., hierarchical clustering, k-means, or density-based clustering) could be applied to a joint feature space that combines image embeddings, report/text embeddings, term-level concept signals (including polarity), and available patient metadata (e.g., age, sex, and clinical indication/history, when available).

This would enable the identification of clinically meaningful patient subgroups and support retrieval/fusion analysis within each subgroup. Additionally, cluster-aware analysis could clarify whether a single default fusion setting is adequate across case types, or whether some subgroups consistently benefit from different retrieval weight profiles. This direction is also consistent with broader recent work showing that clustering can support representative data selection and retrieval-oriented augmentation in LLM pipelines, suggesting a useful methodological precedent for cluster-aware analysis in retrieval systems [23].

7.3.4 Clinician-facing decision support features

An extension for the clinician could be to propagate the *structured image-derived* tags used in the patient-only stage (`LAT`, `SIZE`, `SHIFT`, `DIR` for the primary abnormality) to the retrieved cases as well, by applying the same constrained tag extraction to the representative image of each retrieved case. This would enable a lightweight attribute-level comparison (e.g., laterality agreement or shift-direction mismatch) alongside the existing patient–case comparison outputs, without changing the retrieval or the evidence shown.

A complementary direction would be to introduce a lightweight *knowledge graph view* for each run. The system could extract clinical concepts (findings and anatomy) with polarity (present/absent/uncertain) and simple relations (e.g., *located-in*, *associated-with*), forming a patient graph and per-retrieved-case graphs. The User Interface could then show: (a) a *shared-evidence* subgraph capturing concepts and relations that agree between the patient and the retrieved cases, (b) a *disagreement* subgraph highlighting concepts/relations that are missing on one side or appear contradictory, and (c) a *provenance* overlay that links each node/edge back to the exact supporting text spans in the patient description and retrieved reports. This graph view could make the evidence grounding more traceable by showing where the retrieved cases support, differ from, or only partially align with the patient description.

7.3.5 Evaluation extensions

Future work would benefit from larger multi-patient sweeps and broader pathology coverage to reduce sensitivity to individual cases and to test generalization beyond the current sample. In addition, human evaluation would complement automatic scores, focusing on clinical correctness as well as structured assessment of omissions and contradictions.

It would also be informative to explicitly compare system behavior on *archived* versus *novel* inputs by running matched evaluations where (i) queries come from the indexed pool (with leave-one-out safeguards) and (ii) queries are held-out or externally sourced studies, to quantify how retrieval quality and downstream agreement shift under distribution change.

Closing perspective. This thesis began from a fundamental characteristic of chest radiography: a chest X-ray is a two-dimensional projection of three-dimensional thoracic structures, in which overlapping anatomy can obscure clinically important findings. ThoraxDSS was designed around this constraint by combining complementary retrieval signals across textual, cross-modal, and visual similarity signals, while refining the text pathway with synonym-aware and polarity-aware adjustments to improve clinical consistency. In this way, the system approaches CXR interpretation as a multimodal evidence-integration problem, assembling a traceable account of why prior cases are retrieved and how they support downstream structured analysis. The future directions outlined above extend this same principle toward richer study-level retrieval, uncertainty-aware reasoning, and more transparent clinician-facing evidence inspection.

BIBLIOGRAPHY

[1] W. B. Gefter, B. A. Post, and H. Hatabu, "Commonly missed findings on chest radiographs: Causes and consequences," *Chest*, vol. 163, no. 3, pp. 650–661, Mar. 2023. [Online]. Available: https://pubmed.ncbi.nlm.nih.gov/36521560/

[2] T. Drew, M. L.-H. Vô, and J. M. Wolfe, "The invisible gorilla strikes again: Sustained inattentional blindness in expert observers," *Psychological Science*, vol. 24, no. 9, pp. 1848–1853, Sep. 2013.

[3] National Library of Medicine, "Open-i: An open access biomedical image search engine," https://openi.nlm.nih.gov, 2024, indiana University Chest X-ray Collection; accessed via Open-i. [Online]. Available: https://openi.nlm.nih.gov/faq?download=true

[4] J. Broder, "Imaging the chest: The chest radiograph," 2011, chapter 5 in Diagnostic Imaging for the Emergency Physician.

[5] J.-B. Delbrouck, J. Xu, J. Moll, A. Thomas, Z. Chen, S. Ostmeier, A. Azhar, K. Z. Li, A. Johnston, C. Bluethgen, E. Reis, M. Muneer, M. Varma, and C. Langlotz, "Automated structured radiology report generation," 2025. [Online]. Available: https://arxiv.org/abs/2505.24223

[6] A. Wright, S. Phansalkar, M. Bloomrosen, R. A. Jenders, A. M. Bobb, J. D. Halamka, G. Kuperman, T. H. Payne, S. Teasdale, A. J. Vaida, and D. W. Bates, "Best practices in clinical decision support: The case of preventive care reminders," *Applied Clinical Informatics*, vol. 1, no. 3, pp. 331–345, 2010. [Online]. Available: https://pubmed.ncbi.nlm.nih.gov/21991299/

[7] Z. Yi, T. Xiao, and M. V. Albert, "A multimodal multi-agent framework for radiology report generation," arXiv preprint, 2025. [Online]. Available: https://arxiv.org/abs/2505.09787

[8] E. Asgari, N. Montaña-Brown, M. Dubois, S. Khalil, J. Balloch, J. A. Yeung, and D. Pimenta, "A framework to assess clinical safety and hallucination rates of llms for medical text summarisation," *npj Digital Medicine*, vol. 8, no. 1, p. 274, May 2025.

[9] L. Sun, J. Zhao, M. Han, and C. Xiong, "Fact-aware multimodal retrieval augmentation for accurate medical radiology report generation," 2025. [Online]. Available: https://arxiv.org/abs/2407.15268

[10] A. Stubbs and □. Uzuner, "New approaches to cohort selection," *Journal of the American Medical Informatics Association*, vol. 26, no. 11, pp. 1161–1162, 10 2019. [Online]. Available: https://doi.org/10.1093/jamia/ocz174

[11] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, "Retrieval-augmented generation for large language models: A survey," 2024. [Online]. Available: https://arxiv.org/abs/2312.10997

[12] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive nlp tasks," 2021. [Online]. Available: https://arxiv.org/abs/2005.11401

[13] Z. Wang, C. Gao, C. Xiao, Y. Huang, S. Si, K. Luo, Y. Bai, W. Li, T. Duan, C. Lv, G. Lu, G. Chen, F. Qi, and M. Sun, "Document segmentation matters for retrieval-augmented generation," in *Findings of the Association for Computational Linguistics: ACL 2025*, W. Che, J. Nabende, E. Shutova, and M. T. Pilehvar, Eds. Vienna, Austria: Association for Computational Linguistics, Jul. 2025, pp. 8063–8075. [Online]. Available: https://aclanthology.org/2025.findings-acl.422/

[14] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, "Learning transferable visual models from natural language supervision," 2021. [Online]. Available: https://arxiv.org/abs/2103.00020

[15] S. Geng, H. Cooper, M. Moskal, S. Jenkins, J. Berman, N. Ranchin, R. West, E. Horvitz, and H. Nori, "Jsonschemabench: A rigorous benchmark of structured outputs for language models," arXiv preprint, 2025. [Online]. Available: https://arxiv.org/abs/2501.10868

[16] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with gpus," 2017. [Online]. Available: https://arxiv.org/abs/1702.08734

[17] P. Deka, A. Jurek-Loughrey, and P. Deepak, "Improved methods to aid unsupervised evidence-based fact checking for online health news," *Journal of Data Intelligence*, vol. 3, no. 4, pp. 474–504, 2022.

[18] S. Zhang, Y. Xu, N. Usuyama, H. Xu, J. Bagga, R. Tinn, S. Preston, R. Rao, M. Wei, N. Valluri, C. Wong, A. Tupini, Y. Wang, M. Mazzola, S. Shukla, L. Liden, J. Gao, A. Crabtree, B. Piening, C. Bifulco, M. P. Lungren, T. Naumann, S. Wang, and H. Poon, "A multimodal biomedical foundation model trained from fifteen million image-–text pairs," *NEJM AI*, vol. 2, no. 1, 2024. [Online]. Available: https://ai.nejm.org/doi/full/10.1056/AIoa2400640

[19] Streamlit Inc., "Streamlit: A fast way to build and share data apps," https://streamlit.io, 2024.

[20] Ollama, "Llava: An end-to-end trained multimodal model for general-purpose vision–language understanding," https://ollama.com/library/llava, 2024, lLaVA model card as served by the Ollama runtime; updated to version 1.6.

[21] ——, "A series of multimodal llms (mllms) designed for vision–language understanding," https://ollama.com/library/minicpm-v, 2024, miniCPM-V model card as served by the Ollama runtime.

[22] N. Reimers and I. Gurevych, "Sentence-bert: Sentence embeddings using siamese bert-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, 2019.

[23] Y. Y. Shan Zhong, Jiahao Zeng and B. Lin, "Clustering algorithms and rag enhancing semi-supervised text classification with large llms," Nov. 2024, arXiv:2411.06175v1 [cs.CL], submitted 9 Nov 2024.

[24] z uo, "llava-med-v1.5-mistral-7b_q8_0 (ollama model library entry)," Ollama Model Library, model tag: z-uo/llava-med-v1.5-mistral-7b_q8_0:latest. [Online]. Available: https://ollama.com/z-uo/llava-med-v1.5-mistral-7b_q8_0

[25] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "Bleu: a method for automatic evaluation of machine translation," in *Proceedings of the 40th Annual Meeting of the*

Association for Computational Linguistics, P. Isabelle, E. Charniak, and D. Lin, Eds. Philadelphia, Pennsylvania, USA: Association for Computational Linguistics, Jul. 2002, pp. 311–318. [Online]. Available: https://aclanthology.org/P02-1040/

[26] M. Post, "A call for clarity in reporting BLEU scores," in *Proceedings of the Third Conference on Machine Translation: Research Papers*, O. Bojar, R. Chatterjee, C. Federmann, M. Fishel, Y. Graham, B. Haddow, M. Huck, A. J. Yepes, P. Koehn, C. Monz, M. Negri, A. Névéol, M. Neves, M. Post, L. Specia, M. Turchi, and K. Verspoor, Eds. Brussels, Belgium: Association for Computational Linguistics, Oct. 2018, pp. 186–191. [Online]. Available: https://aclanthology.org/W18-6319/

[27] C.-Y. Lin, "ROUGE: A package for automatic evaluation of summaries," in *Text Summarization Branches Out*. Barcelona, Spain: Association for Computational Linguistics, Jul. 2004, pp. 74–81. [Online]. Available: https://aclanthology.org/W04-1013/

[28] S. Banerjee and A. Lavie, "METEOR: An automatic metric for MT evaluation with improved correlation with human judgments," in *Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization*, J. Goldstein, A. Lavie, C.-Y. Lin, and C. Voss, Eds. Ann Arbor, Michigan: Association for Computational Linguistics, Jun. 2005, pp. 65–72. [Online]. Available: https://aclanthology.org/W05-0909/

[29] M. Popović, "chrF: character n-gram F-score for automatic MT evaluation," in *Proceedings of the Tenth Workshop on Statistical Machine Translation*, O. Bojar, R. Chatterjee, C. Federmann, B. Haddow, C. Hokamp, M. Huck, V. Logacheva, and P. Pecina, Eds. Lisbon, Portugal: Association for Computational Linguistics, Sep. 2015, pp. 392–395. [Online]. Available: https://aclanthology.org/W15-3049/

[30] M. Hanna and O. Bojar, "A fine-grained analysis of BERTScore," in *Proceedings of the Sixth Conference on Machine Translation*, L. Barrault, O. Bojar, F. Bougares, R. Chatterjee, M. R. Costa-jussa, C. Federmann, M. Fishel, A. Fraser, M. Freitag, Y. Graham, R. Grundkiewicz, P. Guzman, B. Haddow, M. Huck, A. J. Yepes, P. Koehn, T. Kocmi, A. Martins, M. Morishita, and C. Monz, Eds. Online: Association for Computational Linguistics, Nov. 2021, pp. 507–517. [Online]. Available: https://aclanthology.org/2021.wmt-1.59/

APPENDIX A

Open-i dataset schema

This appendix defines the fields stored in the image–report pairing dataset.

- `report`: the source report file name (in our implementation, `xml_path.name`, i.e., the XML basename used as the report identifier).
- `image`: the radiograph filename associated with the source report. The loader reads the `id` attribute of each `<parentImage>` element and maps it to a local PNG using the convention `{id}.png`; a row is kept only if the corresponding PNG exists locally.
- `findings`: the narrative text extracted from all `AbstractText` elements with `Label="FINDINGS"`; if multiple such elements are present, their texts are connected with spaces.
- `impression`: the narrative text extracted from all `AbstractText` elements with `Label="IMPRESSION"` in the XML; if multiple such elements are present, their texts are concatenated with spaces.
- `text`: the combined retrieval text used downstream, formed as `(findings + " " + impression).strip()`, i.e., Findings and Impression joined into a single string with leading/trailing whitespace removed.

APPENDIX B

Precomputed retrieval artifacts

To support efficient repeated experiments and reproducible runs, the system stores intermediate artifacts generated during data preparation, embedding, and indexing. When these artifacts are available, the pipeline loads them directly instead of rebuilding them from raw data.

- `df_sample.parquet`: sampled (or full) report–image table used as the retrieval archive during experiments.
- `text_emb.npy`: dense embeddings for report text (`text` field).
- `image_emb.npy`: dense embeddings for archive radiographs.
- FAISS indices: serialized similarity-search indices built from the stored text and image embeddings.
- `openi_pairs.csv`: the full parsed report–image association table produced from the Open-i XML reports (used when `df_sample.parquet` is not already available).

Artifact-first loading order (`load_or_build_data`). The data loader follows a "prefer precomputed" policy. It first checks whether `df_sample.parquet` exists by calling `load_df_sample_if_exists`. If present, that file is loaded and returned immediately, and the pipeline proceeds without rebuilding the full dataset. When loading this parquet file, the `img_path` column is converted back to `Path` objects to support later file access.

If `df_sample.parquet` is not available, the loader falls back to the full table. It loads `openi_pairs.csv` if it already exists. Otherwise, it rebuilds the table from the raw Open-i XML reports.

Rebuilding the report–image table (`build_openi_pairs`). When reconstruction is required, `build_openi_pairs` iterates over all XML report files under `reports_dir`. For each report, it parses the XML and extracts the `FINDINGS` and `IMPRESSION` sections from `AbstractText` elements. If multiple elements exist for the same label, their text is concatenated with spaces. The combined retrieval text is then formed as `(findings + " " + impression).strip()`.

The same XML file is also used to recover report–image links through the `id` attribute of each `<parentImage>` element. Each identifier is mapped to a local PNG filename using the convention `{id}.png`. If a report references multiple images, the loader emits one row per linked image, duplicating the report text fields across rows. Rows are kept only when the corresponding PNG file exists locally.

The resulting table is saved as `openi_pairs.csv` and contains the columns `report`, `image`, `findings`, `impression`, and `text`.

Sampling and file-validity checks (`prepare_sample`). Before embedding and indexing, the loader optionally creates a working table (`df_sample`) for experimentation. It first removes rows with missing report text in the configured text column. It then constructs `img_path` by joining `images_dir` with each `image` filename and drops rows whose resolved image path does not exist.

If sampling is enabled and the dataset exceeds the requested size, the loader draws a random sample using a fixed seed (`random_state=42`) for reproducibility. Otherwise, the full filtered table is used.

Serialization details (`save_df_sample`, `save_artifacts`). The working table is persisted as `df_sample.parquet`. Before saving, the `img_path` column is converted to strings for parquet compatibility. On load, it is converted back to `Path` objects.

The pipeline also stores precomputed embedding arrays as `text_emb.npy` and `img_emb.npy`. The corresponding FAISS index files are serialized separately by the indexing stage.

Loading precomputed embeddings (`load_precomputed_artifacts`). The helper `load_precomputed_artifacts` checks that all required files exist (`df_sample.parquet`, text/image embeddings, and both FAISS index files). If any are missing, it returns `None` values so the pipeline can rebuild what is needed.

If all files are present, it loads `df_sample.parquet`, restores `img_path` to `Path` objects, and loads the text and image embeddings from disk. This allows repeated runs to skip the most expensive preprocessing steps.

APPENDIX C

Reports linked to multiple images and representative image selection

One-to-many report–image relations. In the Open-i collection, a single report may reference multiple radiographs through multiple `<parentImage>` entries. During dataset construction, we therefore materialize the archive at the image level: for each referenced image identifier `img_id` within a report, a separate table row is created that shares the same report identifier and extracted report text (`findings`, `impression`, `text`), but is paired with a different image filename. As a result, the pairing table contains repeated report text whenever a report is linked to more than one radiograph.

Report-level galleries for retrieval and UI. Although the base table is image-level, retrieval and presentation operate at the report level. To preserve the complete visual context of a retrieved report, the system aggregates all image paths associated with the same report into a report-level gallery. Concretely, image paths are collected by grouping rows with the same report identifier and storing the list of their corresponding image paths; the resulting gallery is exposed as `all_image_paths`, together with its size `n_images`. This ensures that, when a report is returned as evidence, all linked radiographs remain available for inspection rather than only the single image that entered the candidate list.

Representative image selection during fusion. Some downstream components require a single image per retrieved report (e.g., a comparison panel). For such cases, the system selects one representative radiograph using a fixed, provenance-aware priority order:

$$\texttt{path\_i2i} \rightarrow \texttt{path\_t2i} \rightarrow \texttt{all\_image\_paths}[0].$$

Here, `path_i2i` denotes the path of the database image through which the report entered the candidate set via the image→image pathway, and `path_t2i` denotes the corresponding path from the text→image pathway. If neither modality provides an explicit image path for that report (e.g., the report is included only via text→text retrieval or via union deduplication), the system falls back to the first image in the report's gallery (i.e., the first entry of `all_image_paths`). This policy preserves traceability, by preferring the modality-specific image that triggered retrieval when available, while also maintaining completeness of evidence access through the full report-level gallery.

APPENDIX D

Retrieval Implementation Details

D.1 Hyperparameters and controls

The following parameters control the behavior of `MultimodalRetriever.run_query` and the modality-specific search routines it calls:

- k: Target number of results returned *per modality* (Text→Text, Text→Image and Image→Image). After late fusion, the fused output is ranked and displayed with the highest-scoring candidates first.
- **overfetch**: Candidate pool size requested from the FAISS index *before* filtering. Each modality search retrieves max(k, `overfetch`) nearest neighbors, then applies exclusion and uniqueness constraints and keeps the first k valid results.
- **unique_by**: Uniqueness key for image-returning pathways (Text→Image and Image→Image). If `"report"`, results are deduplicated by report identifier, returning at most one image per report. If `"path"`, results are deduplicated by image path, allowing multiple images from the same report to be returned.
- **exclude_report**: Optional leave-one-out control used during evaluation. When set, any candidate whose report identifier matches `exclude_report` is removed in each modality before fusion, preventing trivial self-retrieval.

D.2 Output Data Schema

Each retrieval pathway returns a ranked list of dictionaries. For the purpose of the decision support system (DSS) and downstream fusion, these are converted to `pandas.DataFrames`. As shown in the retrieval methods (`search_text2text`, `search_text2image`, and `search_image2image`), the schema varies based on the target index.

Field	Pathways	Source	Description
`rank`	All	Python Loop	The sequential position ($1 \ldots k$) after seen-set deduplication.
`score`	All	FAISS	The inner-product similarity score between the query and archive embedding.
`report`	All	Metadata	The unique identifier for the clinical case.
`text`	All	Metadata	The clinical report content (Findings + Impression).
`image`	T2I, I2I	Metadata	The specific image filename associated with the visual match.
`path`	T2I, I2I	Metadata	The absolute filesystem path to the radiograph.

Table D.1: Metadata schema for individual pathway outputs prior to fusion.

D.3 Late Fusion Implementation

The `_fuse_results` method serves as the structural bridge between the parallel retrieval streams, conveying them into a single ranked list. The following implementation details explain the logic used to handle data sparsity and visual representation.

D.3.1 Candidate union and missing-modality scores

Late fusion starts from three pathway outputs (T2T, T2I, I2I), each returning a ranked list with a per-hit similarity score and a `report` identifier. The image-based pathways

(T2I and I2I) additionally return the matched image filename and filesystem path (`image`, `path`). The fused candidate set is defined as the union of all retrieved `report` identifiers across pathways, deduplicated so that each report appears once.

For each candidate report, modality-specific score fields are attached when available: `score_text2text_initial`, `score_text2image`, and `score_image2image`. If a report is not returned by a given pathway, the corresponding score is missing after the merge and is set to 0.0 (i.e., `NaN` $\to 0.0$), so that every candidate has a complete set of modality score fields for fusion.

During the same merge step, image-path provenance from the image-based pathways is preserved in modality-specific columns (`path_t2i` and `path_i2i`), which are later used by the representative-image selection rule described in Appendix C.

D.3.2 Visual Metadata Enrichment

The fusion module enriches the candidate set with metadata to support the Decision Support interface:

- **Complete Gallery**: The field `all_image_paths` is populated by mapping each `report` to its full list of associated archive image paths; `n_images` stores the gallery size.
- **Selection Logic (`_choose_image_path`)**: A single representative image path `image_path_for_plot` is assigned using a fixed priority rule:
    1. `path_i2i`: the retrieved image path returned by Image-to-Image for this report (if present).
    2. `path_t2i`: the retrieved image path returned by Text-to-Image for this report (if present).
    3. `all_image_paths[0]`: fallback to the first image in the report-level gallery.

D.3.3 Text-score completion over the fused candidate set

Because each retrieved image in the archive is linked to a radiology report, every fused candidate has associated report text. After candidate union, the system recomputes a Text→Text similarity score for *every* candidate report, producing a completed text score field `score_text2text`. This defines S_{T2T} over the full fused set, including reports that entered only through image-based retrieval (T2I or I2I).

In the implementation, this completion step obtains the stored report embedding for each candidate and scores it against the query embedding (yielding an inner-product similarity).

D.3.4 Final reranking logic (implementation)

The fusion module computes the diagnostic fused score $S_{\text{fused,raw}}$ and the final ranking score S_{fused} exactly as defined in Section 3.4.3 (Eqs. 3.1 and 3.2). In the implementation, these are stored as `score_fused_raw` and `score_fused`, and the final `rank` is assigned by sorting `score_fused` in descending order.

Weight derivation from configuration. Weights are derived from configuration values as $w_{t2i} = \max(0, \texttt{weight\_text2image})$, $w_{i2i} = \max(0, \texttt{weight\_image2image})$, and $w_{t2t} = \max(0, 1 - w_{t2i} - w_{i2i})$.

APPENDIX E

Term resource parsing and synonym mapping details

This appendix summarizes the implementation details used to construct the canonical-term list and synonym mapping for terminology normalization (Section 3.4.4).

String normalization. All term strings are normalized with the same preprocessing function before storage and matching: leading/trailing whitespace is removed, text is lowercased, and repeated whitespace is collapsed to a single space. The normalized canonical term is then used as the internal concept identifier in downstream matching and polarity extraction.

Canonical term loading. Canonical concepts are loaded from the `Term` column of the Open-i term spreadsheet (sheet `synonyms`) by retaining non-empty string entries. In the current implementation, canonical terms are loaded as provided (i.e., no deduplication is applied at load time).

Synonym column handling (duplicate headers). The spreadsheet may contain multiple columns with the same header name (e.g., repeated `Synonym` columns). To avoid losing values under duplicate headers, the loader identifies synonym columns by scanning all column names and selecting every column whose header begins with `Synonym` (prefix match), then accesses them by column index rather than by a single column name.

Row matching for synonym extraction. For each canonical term, the loader first matches the corresponding spreadsheet row using normalized equality on the `Term` column. If no exact normalized match is found, it applies a fallback case-insensitive `contains` match and uses it only when that fallback returns a unique row.

Cell parsing and synonym storage. Synonym values are read cell-by-cell from all detected synonym columns in the matched row. If a cell contains multiple variants, it is split on `;` and `,` (regex `[;,]`). Each fragment is normalized and stored as an individual synonym entry. Fragments that are empty or identical to the normalized canonical term are discarded.

Synonym lists are stored *without deduplication* in the current implementation and are capped at `max_synonyms_per_term = 50` entries per canonical concept for efficiency.

Fallback behavior when the spreadsheet is unavailable. If the spreadsheet path cannot be resolved or loading fails, the system falls back to a built-in default term list (`DEFAULT_TERMS`) and uses an empty synonym map. This list was generated with LLM assistance using the Ollama-hosted quantized model `z-uo/llava-med-v1.5-mistral-7b_q8_0:latest` [24] and is used only as a fallback resource. This ensures that the polarity and term-matching pipeline remains operational even when the external vocabulary file is missing.

Use in terminology normalization. During term matching, both canonical terms and synonym entries are treated as valid phrase variants. However, any matched synonym is mapped back to its canonical concept before comparison, so that downstream polarity extraction and report-query comparison operate at the canonical concept level.

APPENDIX F

Negation cue inventory and polarity scope rules

This appendix specifies the rule inventory used to assign concept-level polarity labels (*affirmed*, *negated*, *not mentioned/conflicting*) after canonical-term matching.

F.1 Negation cue inventory

Negation is triggered only by an explicit cue phrase, represented as a token sequence. The cue inventory used in this thesis is:

- Single-token cues: {`no`, `without`}
- Multi-token cues: {`free of`, `negative for`, `no evidence of`, `no sign of`, `no signs of`, `there is no`, `there are no`, `is no`, `are no`}

A cue *triggers* when its full token sequence matches contiguously in the tokenized text.

Representation and matching of negation cues In the implemented system, each negation cue is represented not as a raw string, but as an ordered sequence of normalized tokens. Formally, the negation cue inventory is stored as a list of token lists, where each inner list corresponds to a single cue phrase. This representation allows both single-token and multi-token cues to be handled uniformly during matching.

More precisely, the cue inventory takes the form:

```
negation_cues = [ ["no"], ["without"], ["free", "of"], ["negative", "for"],
["no", "evidence", "of"], ["no", "sign", "of"], ["no", "signs", "of"],
["there", "is", "no"], ["there", "are", "no"], ["is", "no"], ["are", "no"], ]
```

All text is lowercased and tokenized prior to matching. A negation cue is set to *trigger* at position i in a token sequence only if the entire cue token list matches contiguously starting at that position. Partial or non-contiguous matches do not activate negation.

During polarity extraction, the tokenized sentence is scanned from left to right. At each token position, the system attempts to match every cue in the inventory. If a cue matches, its endpoint defines the start of a local negation scope, which is subsequently bounded by predefined scope terminators (see Section F.2). Only canonical concept mentions whose token spans fall fully within this scope are assigned negated polarity.

This explicit token-sequence representation ensures that negation handling remains precise and deterministic, avoids over-negation of entire sentences, and allows clinically meaningful constructs such as "*no evidence of pneumothorax, but pleural effusion is present*" to be handled correctly.

F.2 Scope terminators and local negation scope

In the implementation, scope termination is governed by a fixed set of tokens evaluated during token-level scanning. Termination occurs immediately upon encountering any token belonging to the following set:

{`.`, `\n`, `;`, `but`, `however`, `though`, `although`, `except`, `nevertheless`, `yet`, `-`, `—`}

All terminators are matched as individual normalized tokens produced by the same tokenization procedure used for negation cue detection and canonical-term matching. Once a terminator is encountered, the current negation scope is closed and no further concept mentions are considered negated under that cue.

This design enforces *local* negation: a negation cue affects only concept mentions occurring after the cue and before the first terminating token, rather than propagating across the entire sentence or report.

F.3 List connectors and matching punctuation

Within a negation scope, radiology findings are frequently expressed as list-alike constructions are common (e.g., “no pneumothorax, pleural effusion, or consolidation”).

To support this structure, the system treats the following tokens as list connectors, defining a dedicated set of *list connector tokens*:

{`,`, `and`, `or`, `/`, `-`}.

Immediately after a negation cue is matched, any number of consecutive list connector tokens are skipped before determining the start of the negation scope. This allows concept mentions that follow connectors directly to be interpreted as falling under the same negation cue, without prematurely closing or shifting the scope. List connectors do not terminate negation scope.

Separately, to ensure robust canonical-term and synonym matching across heterogeneous punctuation styles, a fixed set of punctuation tokens is ignored during phrase matching. These tokens are skipped when aligning token sequences against canonical terms and synonym variants:

{`,`, `:`, `;`, `-`, `/`, `(`, `)`, `[`, `]`}.

Skipping these tokens allows multi-token concepts to be matched correctly even when interrupted by punctuation, while preserving the original token stream for negation scope detection and termination.

F.4 Concept-level polarity assignment and conflict resolution

Polarity is assigned per input text (query or candidate report) at the level of canonical concepts. Processing is sentence-local: the text is split into sentence-like units using punctuation boundaries, and each unit is handled independently to prevent negation cues from propagating across boundaries.

Within each sentence, polarity assignment proceeds as follows.

Negated mentions. The token sequence is scanned from left to right. Whenever a negation cue is detected, a forward scope is constructed according to the rules defined in Sections F.1–F.3. All canonical concept mentions whose matched token span lies fully within this scope are assigned negated polarity (-1). Canonical concepts negated within a sentence are recorded in a sentence-local set.

Affirmed mentions. Independently of negation scope boundaries, the sentence is evaluated for all canonical concept mentions. Any concept that is mentioned in the sentence and is not present in the sentence-local negation set is assigned affirmed polarity ($+1$).

Aggregation and conflict resolution within a single text. For each *single input text* (processed independently for the query and for each candidate report), the system builds a polarity set *per canonical term* by accumulating all polarity assignments for that term across sentences. A final polarity label is then assigned *for that term in that text* as follows:

- If the term's polarity set contains only $+1$, assign $+1$ (affirmed).
- If the term's polarity set contains only -1, assign -1 (negated).
- If the term's polarity set contains both $+1$ and -1, assign 0 (ambiguous/conflicting mention within the same text).
- If the term's polarity set is empty, assign 0 (term not mentioned in that text).

This produces one polarity map per text (query or report), mapping each canonical term to a single label in $\{+1, -1, 0\}$.

Cross-text polarity comparison (query vs. candidate report). After polarity maps are computed separately for the query text and the candidate report text, the system compares them *term by term* at the canonical-term level. For each canonical term, it examines the polarity pair formed by the query label and the report label, and uses this pair in three ways.

First, it contributes to a scalar polarity-aware adjustment used in retrieval reranking: matching affirmed mentions and matching negated mentions contribute positive evidence (with different weights), polarity mismatches contribute a strong penalty, and single-sided mentions contribute a mild penalty. Contributions are accumulated over canonical terms and then normalized, scaled, and clamped as described in Section 3.4.4.

Second, the system generates explicit warning strings only for true polarity conflicts, i.e., when the same canonical term is affirmed in one text and negated in the other.

Third, the same term-by-term comparison is used to build structured concept lists for downstream UI/DSS use: a `common` list (canonical terms mentioned in both texts with the same nonzero polarity), a `patient_only` list, and a `report_only` list (both including one-sided mentions and polarity-disagreeing mentions, each with polarity tags).

APPENDIX G

Prompt templates and output schemas

G.1 Patient-only prompt

This appendix provides the *exact* prompt template used for the patient-only VLM stage (Section 3.5.2).

```
You will analyze a PATIENT chest X-ray.

IMAGE MAPPING:
- PATIENT: Image 1

CLINICAL SUMMARY (PATIENT REPORT TEXT):
{query_text}

TASK:
Analyze the patient image and the clinical summary.
Return ONLY a single valid JSON object with this schema:

{
"patient_findings": [
    "One short sentence per finding."
]
}
```

CRITICAL OUTPUT FORMAT (must follow exactly):

- You MUST return 4-5 DISTINCT patient findings (4-5 list items).
- ONLY the FIRST finding string MUST start with tags in this exact order:

"[LAT=<laterality>] [SIZE=<size>] [SHIFT=<yes/no/unknown>]
[DIR=<left/right/none/unknown>] <finding sentence>"

- Findings 2-5 MUST NOT include any tags. They must be plain short sentences.

Where the enums are:

- laterality (LAT): right | left | bilateral | unknown
- size (SIZE): small | moderate | large | unknown
- shift present (SHIFT): yes | no | unknown
- shift direction (DIR): left | right | none | unknown

RULES:

- If unsure about any enum, use "unknown" (do NOT guess).
- Tags should refer to the MAIN abnormality (e.g., pneumothorax/effusion/mass).
- SHIFT/DIR should refer to mediastinal/tracheal shift suggesting tension physiology (if applicable).
- Avoid unsupported findings. Be faithful to image and consistent with the clinical summary.
- Output MUST be valid JSON, double quotes only.
- No comments, no extra text.
- Do NOT use placeholders like "XXXX". If uncertain, omit that finding.
- One atomic finding per sentence.
- Do NOT repeat the same finding in different words.

EXAMPLE (format only):

```
{
"patient_findings": [
    "[LAT=right] [SIZE=large] [SHIFT=yes] [DIR=left] Large right pneumothorax
    with mediastinal shift.",
    "Right lung shows marked hyperlucency and absent peripheral vascular
```

```
    markings.",
    "Collapsed right lung is visualized medially.",
    "Cardiomediastinal contours are displaced (tension physiology if
    applicable).",
    "No left-sided pneumothorax is identified."
]
}

Return ONLY the JSON object.
```

G.2 Pairwise patient–case comparison prompt

This appendix provides the exact prompt template used for the pairwise patient–case VLM stage described in Section 3.5.2. The pairwise stage returns a single JSON object with five required keys: `case_findings`, `similarity_to_patient`, `common_with_patient`, `differences_from_patient`, and `one_line_rationale`.

```
Analyze TWO chest X-rays: PATIENT (Image 1) and RETRIEVED CASE (Image 2).

TEXT A (patient report):
{query_text}

TEXT B (case report snippet, id={case_id}):
{snippet}

WARNINGS (only for overlap filtering; do NOT quote):
{safe_warnings_json}

TERM HINTS (anchors only; do NOT copy as standalone items):
overlap_terms_hint={hint_common_json}
case_only_terms_hint={hint_diffs_json}

OUTPUT RULES:
- Output ONE JSON object ONLY. No extra text.
```

```
- Keys must be exactly:
case_findings, similarity_to_patient, common_with_patient,
differences_from_patient, one_line_rationale.
- All list items must be NON-EMPTY strings.
- Do NOT output any of these anywhere: "No meaningful overlap found", "<", ">",
"...", "TODO".

case_findings (Image 2 ONLY):
- MUST be a list of EXACTLY 3 strings.
- Each string MUST be a short VISUAL radiology finding about Image 2.
- ABSOLUTELY FORBIDDEN inside case_findings:
  * any tag text like "LAT=right" or "[LAT=right]"
  * meta phrases like "No mention", "Text A", "Text B", "Image 1", "Image 2",
  "report does not mention"

common_with_patient / differences_from_patient (TEXT A vs TEXT B ONLY):
- Use only clinician-friendly radiology phrases supported by the texts.
- If no explicit support: output [] (do NOT invent).
- Do NOT output meta phrases like "No mention of..." in these lists.

similarity_to_patient:
- Choose exactly one: "similar", "partially similar", "not similar" (lowercase).

one_line_rationale:
- Exactly 3 short sentences.
- Must mention BOTH report-text alignment AND visual alignment.

Return JSON:
{
  "case_findings": ["string","string","string"],
  "similarity_to_patient": "similar",
  "common_with_patient": ["string"],
  "differences_from_patient": ["string"],
  "one_line_rationale": "sentence. sentence. sentence."
```

```
}
```

G.3 Final clinician-facing assessment prompt

This appendix provides the exact prompt template used for the final clinician-facing assessment stage described in Section 3.5.2.

```
You are producing a FINAL clinician-facing assessment for a chest X-ray system.

YOU SEE ONLY ONE IMAGE HERE:
- Image 1 = PATIENT chest X-ray

OUTPUT JSON ONLY (one JSON object). No markdown. No extra text.
If you cannot comply, output exactly:
{"text":"", "confidence":"low"}

ALLOWED EVIDENCE SOURCES:
1) CLINICAL SUMMARY text (below)
2) PATIENT_FINDINGS list (below)
3) Image 1 (for verification / calibration)

CRITICAL CONTENT RULES:
A) PRIMARY ABNORMALITIES:
- You may state as "present/likely" ONLY abnormalities that are already mentioned
  in PATIENT_FINDINGS or CLINICAL SUMMARY.
- Do NOT introduce new abnormality categories as definite.

B) IMAGE-ONLY SUGGESTIONS (LIMITED, OPTIONAL):
- You MAY add at most 1 sentence with up to 1-2 uncertain "image-suggested" items,
  but ONLY phrased as "may suggest / possible / cannot exclude".
- You MUST NOT state any such item as present/likely unless it already appears in
  PATIENT_FINDINGS or CLINICAL SUMMARY.
- If unsure, omit image-suggested items entirely.
- If an item is NOT explicitly in PATIENT_FINDINGS/CLINICAL SUMMARY, you MUST
```

```
  phrase it only as "possible/may suggest" and NEVER as present/likely.

C) MAIN FINDING CALIBRATION:
- Set confidence="low" ONLY if:
  (a) you explicitly wrote "image-report mismatch" due to CLEAR contradiction,
  OR
  (b) warnings_count is very high AND the retrieved cases are broadly
  unsupportive (mostly not similar).
- Do NOT set low due to "uncertainty" alone.

INPUTS
CLINICAL SUMMARY (patient report text):
{query_text}

PATIENT_FINDINGS (primary evidence):
{safe_findings_json}

CONSISTENCY SIGNALS (aggregated only; NO case text is provided):
cases_seen={n_cases}; similar={n_similar}; partially_similar={n_partial};
not_similar={n_not}; negated_terms_count={n_neg_terms}; warnings_count={n_warn}

REQUIRED "text" FORMAT (headings and order MUST match exactly):

EVIDENCE:
<1 paragraph summarizing the main abnormality and key supporting signs, grounded in
PATIENT_FINDINGS and Image 1. Paraphrasing is allowed.>

CASE CONSISTENCY:
<1 paragraph describing overall retrieval support using only the aggregated counts.
No case-level details.>

NEGATION CHECK:
<1-2 sentences:
- If negated_terms_count == 0, write:
```

```
  "No explicit negations highlighted in the patient text."
- Else, write:
  "Patient text contains explicit negations; this is normal in radiology
  reports and does NOT reduce confidence by itself. Use it only to avoid
  contradicting negated statements."
Do NOT list individual terms. Do NOT mention case identifiers.>

IMPRESSION:
<1 paragraph stating final impressions using ONLY abnormalities from
PATIENT_FINDINGS or CLINICAL SUMMARY as "present/likely".>
<OPTIONAL: add one sentence with up to 1-2 uncertain image-suggested items,
clearly phrased as possible.>
<If applicable, include the exact phrase "image-report mismatch".>

CONFIDENCE RATIONALE:
<1 paragraph explaining the confidence level using only: image-report mismatch
presence, warnings_count, and similarity counts. Explicit negations may be
mentioned as requiring caution but
MUST NOT be used as a direct penalty.>

CONFIDENCE RULES:
- Default confidence is "medium".
- Output "low" ONLY if the exact phrase "image-report mismatch" appears above.
- Otherwise, do NOT output "low".
- Output "high" only if Image 1 clearly supports the main abnormality, warnings_count
  is minimal, and there is at least some retrieval support.

OUTPUT JSON ONLY:
{
  "text": "EVIDENCE:\n...\n\nCASE CONSISTENCY:\n...\n\nNEGATION CHECK:\n...
          \n\nIMPRESSION:\n...\n\nCONFIDENCE    RATIONALE:\n...",
  "confidence": "low|medium|high"
}
```

APPENDIX H

Text metrics

This appendix documents the evaluation quantities reported in Chapter 6 and how they are computed by the evaluation code. Unless stated otherwise, higher values indicate closer agreement.

H.1 Evaluation interface, reported scalars, and aggregation

Prediction–reference formation. Text-based metrics are computed on a single *prediction–reference* string pair. When a system output is a list of sentences (e.g., findings lists), it is converted into a single prediction string by removing the placeholder token `XXXX` from each sentence and concatenating the sentences with spaces. Reference texts are used as provided.

Reported scalar identifiers (used in Chapter 6). The evaluation script summarizes the per-run outputs into the following named scalars:

- **A) Patient findings faithfulness**:
  `patient_bleu`, `patient_rougeL`, `patient_meteor`, `patient_chrf`, `patient_bertscore`.
- **B) Retrieved-case findings faithfulness (mean over evaluated retrieved cases)**:
  `mean_case_bleu`, `mean_case_rougeL`, `mean_case_meteor`, `mean_case_chrf`, `mean_case_bertscore`.
- **C) Similarity label quality (weak supervision)**:
  `similarity_accuracy`, `similarity_macro_f1`.

- **D) Comparison summary term-set agreement (mean F1)**: `mean_common_f1`, `mean_diff_f1`.
- **E) Final assessment coherence with patient text**: `final_bleu`, `final_rougeL`, `final_meteor`, `final_chrf`, `final_bertscore`.

Aggregation across retrieved cases. When a block produces one score per retrieved case, the corresponding mean scalar is computed as the arithmetic mean over available numeric values. Missing values (`None`) are excluded; if no numeric values are available, the mean is recorded as missing.

H.2 Text-similarity metrics

Reporting scales and missing values (code-aligned). BLEU and chrF follow SacreBLEU's 0–100 reporting scale, while ROUGE, METEOR, and BERTScore are reported on a 0–1 scale, when their dependencies are available.

BLEU (sentence-level). BLEU [25] is computed using [26]: `sacrebleu.sentence_bleu(pred, [ref]).score`. If `sacrebleu` is unavailable at runtime, BLEU is recorded as missing (`None`).

ROUGE (F1; ROUGE-L reported). ROUGE [27] is computed using the ROUGE F1 formulation. While the evaluation code can produce ROUGE-1, ROUGE-2, and ROUGE-L, this thesis reports only ROUGE-L in Chapter 6.

METEOR. METEOR [28] is computed using NLTK's `meteor_score` with pre-tokenized inputs. Both texts are tokenized using the same alphanumeric tokenization (`[a-z0-9]+`). If either side tokenizes to an empty sequence, METEOR returns 0.0. If the dependency is unavailable at runtime, METEOR is recorded as missing (`None`).

chrF (sentence-level). chrF [29] is computed using SacreBLEU [26]: `sacrebleu.sentence_chrf(pred, [ref]).score`. If `sacrebleu` is unavailable at runtime, chrF is recorded as missing (`None`).

BERTScore (F1). BERTScore is computed using the `bert_score` package with `distilbert-base-uncased`, reporting the F1 component on a 0–1 scale. If the dependency is unavailable (or an error occurs), the value is recorded as missing (`None`). We cite an analysis of BERTScore in [30].

H.3 Weak-label similarity and term-set metrics

Weak semantic similarity for similarity labels (Block C). For each retrieved case i, we construct a *silver* similarity label y_i by comparing the patient input text with the retrieved case report text using cosine similarity s_i of L2-normalized sentence embeddings from `sentence-transformers/all-MiniLM-L6-v2` [22].[1] Embeddings are computed with `normalize_embeddings=True`. If either text is empty, s_i is undefined and the case is excluded from Block C scoring. Silver labels are assigned with fixed thresholds:

$$y_i = \begin{cases} \texttt{similar} & \text{if } s_i \geq 0.70 \\ \texttt{partially similar} & \text{if } 0.45 \leq s_i < 0.70 \\ \texttt{not similar} & \text{if } s_i < 0.45 \end{cases}$$

The *predicted* label $\hat{y}_i$ is the system output field `similarity_to_patient` for the same retrieved case. Predictions are lowercased and normalized to the label set {`similar`, `partially similar`, `not similar`}; any other value is mapped to `not similar`.

Agreement metrics. Agreement between $\hat{y}_i$ and y_i is computed over retrieved cases with defined silver labels: (i) *accuracy* is the fraction of cases where $\hat{y}_i = y_i$; (ii) *macro-F1* is the unweighted mean of per-label F1 scores computed one-vs-rest over the label set above. We report `similarity_accuracy` and `similarity_macro_f1`.

Term-set precision/recall/F1 for comparison summaries (Block D). For each retrieved case i, reference term sets are obtained by extracting *present* (positive-polarity) canonical concepts from the patient input text (P) and the retrieved-case report text (C_i) using the same synonym-normalized, negation-aware term extraction used throughout the pipeline. We then define $\text{gold\_common}_i = P \cap C_i$ and $\text{gold\_diff}_i = C_i \setminus P$. Predicted term sets are extracted from the corresponding model output fields

[1] Model card: `https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2`.

`common_with_patient` and `differences_from_patient` using the same extractor. For each set comparison, letting G be the relevant reference set (either gold_common$_i$ or gold_diff$_i$) and $\hat{G}$ the corresponding predicted set, we compute:

$$\text{Precision} = \frac{|G \cap \hat{G}|}{|\hat{G}|}, \qquad \text{Recall} = \frac{|G \cap \hat{G}|}{|G|}, \qquad \text{F1} = \frac{2\,\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}.$$

Following the implementation, when both sets are empty ($G = \emptyset$ and $\hat{G} = \emptyset$), precision and recall are undefined (0/0), so we define Precision = Recall = F1 = 1 to reflect perfect agreement (no false positives and no false negatives). If exactly one of G or $\hat{G}$ is empty, we define Precision = Recall = F1 = 0. Per-case F1 is computed for `common_with_patient` and `differences_from_patient` separately. The reported `mean_common_f1` and `mean_diff_f1` are arithmetic means of the per-case F1 values.

Illustrative example (one retrieved case). For a single retrieved case, suppose the extracted term sets are:

$$\begin{aligned}
\text{gold_common} &= \{\texttt{lung}, \texttt{pneumothorax}, \texttt{right}, \texttt{shift}\},\\
\text{gold_diff} &= \{\texttt{left}, \texttt{pulmonary atelectasis}\},\\
\text{pred_common} &= \{\texttt{lung}, \texttt{pneumothorax}, \texttt{right}, \texttt{pulmonary atelectasis}\},\\
\text{pred_diff} &= \{\texttt{left}, \texttt{right}, \texttt{shift}\}.
\end{aligned}$$

Then common-set precision/recall are $3/4$ and $3/4$ (F1 = 0.75), while diff-set precision/recall are $1/3$ and $1/2$ (F1 = 0.40). This procedure is applied independently to every retrieved case i.